

2D Finite-Difference Wavefield Modelling

Jan Thorbecke

August 7, 2025

Contents

0	Getting Started	2
0.1	Installation	2
0.2	Compilation and Linking	3
0.3	Running examples	3
1	Introduction to Finite-Difference	4
1.1	Finite-difference algorithm	6
1.2	Stability and Dispersion	8
2	Acoustic	9
2.1	Staggered scheme	10
2.2	Computing SH-waves with acoustic wave equation	11
3	Visco-Acoustic	12
4	Elastic	15
5	Visco-Elastic	17
6	Parameters in program fdelmodc	18
6.1	Modelling parameters	21
6.2	Medium parameters	21
6.3	Boundaries	22
6.3.1	Recursive Integration PML: acoustic	26
6.3.2	Complex frequency shifted RIPML: acoustic	27
6.4	Source signature parameters	28
6.5	Source type and position parameters	33
6.5.1	Source type	34
6.5.2	Source positions	36
6.6	Receiver, Snapshot and Beam parameters	38
6.6.1	Receiver, Snapshot and Beam type	38
6.6.2	Receiver positions	39
6.6.3	Interpolation of receiver positions	40
6.6.4	Snapshots and Beams	41
6.7	Verbose	42
7	Examples to run the code	43
7.1	Example for plane waves: fdelmodc_plane.scr	43
7.2	Example for viscoelastic media: fdelmodc_visco.scr	45
7.3	Example for different source distributions: fdelmodc_sourcepos.scr	45
7.4	Example with receivers on a circle: fdelmodc_circ.scr	45
7.5	Example with topography: fdelmodc_topgraphy.scr	46
7.6	Example verification with analytical results: FigureGreenDxAppendixA.scr	48
7.6.1	Acoustic	48
7.6.2	Elastic	49

7.7	Verification with Scattered field of a cylinder	52
7.7.1	1D scattering by a slab	54
7.7.2	2D scattering by a circular cylinder	56
7.7.3	3D scattering by a sphere	56
A	Source and directory structure	59
B	Differences in parameter use compared with DELPHI's fdacmod	62
C	Makewave	62
C.1	Parameters	62
C.2	Examples	66
D	Makemod	66
D.1	Parameters	66
D.2	Examples	68
E	fdemmodc	68

0 Getting Started

0.1 Installation

The software, downloaded as a gzipped tar archive, can be extracted in a directory of your choice, e.g., by typing

```
> tar -xvfz OpenSource.tgz
```

at the terminal command line of a Unix based operating system. You can also directly pull the source code from its GitHub repository: <https://github.com/JanThorbecke/OpenSource> and make a pull or clone request:

```
> git clone https://github.com/JanThorbecke/OpenSource.git
```

The OpenSource directory also contains other programs (Marchenko and utilities) that are not discussed in this manual.

The code is designed to run on current Unix-based or Unix-like system, such as Linux, Sun's Solaris, Apple's OS-X or IBM's AIX. However, the code has not been tested on any version of Windows. It is certainly possible to make it run on a Windows platform using the appropriate tools, but do not expect it to work simply out-of-the-box. For running the code in a Windows environment one could make use of Cygwin. Cygwin is a virtual Linux environment running in a Windows operating system.

The package extracts itself into a directory **OpenSource**, with the following sub-directories:

- bin
- lib
- doc
- include
- fdelmodc
- utils
- FFTlib

The README file in that directory contains some of the quick-start information given here in condensed format. The FFTlib directory does not contain a main program and contains source code to build a library (`libgenfft.a`) with Fourier transformation functions. The `fdelmodc` and `utils` directories include all files needed to compile and link the executables build in that directory. This means that some of the source files in the `fdelmodc` and `utils` directory are the same. This has been done to make the compilation procedure less complicated. Section A of this manual contains a brief (one-sentence) explanation of the meaning of all the files in the source tree of this package.

The source code is in continuous development to add new features and solve bugs. The latest version of the source code and manual can always be found at:

<https://github.com/JanThorbecke/OpenSource>

<http://www.xs4all.nl/~janth/Software/Software.html>.

The code is used by many different people and when somebody requests a new option for the code (for example place receivers in a circle) then I will try to implement and test the new functionality and put the updated source (and manual) on the website as soon as it is ready and tested.

0.2 Compilation and Linking

1. To compile and link the code you first have to set the `ROOT` variable in the `Make_include` file which can be found in the directory where you have found this README.
2. Check the compiler and `CFLAGS` options in the file `Make_include` and adapt to the system you are using. The default options are set for a the GNU C compiler on a Linux system. A Fortran or g++ compiler is not needed to compile the code. The compilation of the source code has been tested with several versions of GNU and Intel compilers.
3. If the compiler options are set in the `Make_include` file you can type

```
> make
```

and the Makefile will compile and link the source code in the directories:

- FFT library
- `fdelmodc`
- `utils`

The compiled FFT library will be placed in the `lib/` directory, the executables in the `bin/` directory and the include file of the FFT library in the `include/` directory.

To use the executables don't forget to include the pathname in your `PATH`:

```
bash:
export PATH='path_to_this_directory'/bin:$PATH:
csh:
setenv PATH 'path_to_this_directory'/bin:$PATH:
```

On Linux systems using the bash shell you can put the `export PATH='path_to_this_directory'/bin:$PATH:` setting in `$HOME/.bashrc`, to set it every time you login.

Other useful make commands are:

- `make clean`: removes all object files, but leaves libraries and executables
- `make realclean`: removes also object files, libraries and executables.

0.3 Running examples

Important note: The examples and demo scripts make use programs of Seismic Unix (SU). Please make sure that SU is compiled without XDR: the XDR flag (`-DSUXDR`) in `$CWPROOT/Makefile.config` must **NOT** be set in compiling SU. The SU output files of `fdelmodc` are all base on local IEEE data. When the XDR flag is set in SU you have to convert the output files of `fdelmodc` (and all the programs

in the utils directory: basop, fconv, extendmodel, makemod, makewave) with `suoldtonew`, before using SU programs.

If the compilation has finished without errors and produced an executable called `fdelmodc` you can run the demo programs by running. For example the script

```
> ./fdelmodc_plane.scr
```

in the directory `fdelmodc/demo/`. The results of this script are discussed in section 7.1. The `fdelmodc/demo/` directory contains scripts to demonstrate the different possibilities of the modeling program. Most of the scripts in the demo directory can re-produce the figures used in this manual. The examples section 7 contains also detailed explanations of the other demo scripts.

To reproduce the Figures shown in the GEOPHYSICS manuscript "Finite-difference modeling experiments for seismic interferometry" (Thorbecke and Draganov, 2011) the scripts in `fdelmodc/FiguresPaper/` directory can be used. Please read the README in the FiguresPaper directory for more instructions and guidelines.

To clean-up all the produced output files in the `fdelmodc/demo/` and `fdelmodc/FiguresPaper/` directory you can run the `clean` script in those directories.

To read in the SU files `*.su` into MATLAB you can do the following:

```
> sustrip < file.su > file.bin
```

This strips the SU headers (first 240 Bytes from each trace) from the SU files and the output contains only IEEE little endian floating points. To read this `*.bin` file into MATLAB you can do:

```
ns=751 %number of samples per trace =>n1 of the sustrip message
ntr=31 %number of traces =>n2 of the sustrip message
nshot=1
file='file.bin'
fid=fopen(file,'r');
data= fread(fid,[ns,ntr*nshot],'float32');
fclose(fid)
data2 = reshape(data,ns,ntr,nshot);
```

1 Introduction to Finite-Difference

The program `fdelmodc` can be used to model waves conforming the 2D wave equation in different media. This manual does not give a detailed overview about finite-difference modelling and only briefly explains the four different Finite-Difference (FD) schemes implemented in the program `fdelmodc`. More important are the (im)-possibilities of the program, and a detailed explanation is given how to use the parameters together with certain specific implementation issues a user must be aware of. There are already many programs available to model the 2D wave equation, and one might ask why write another one? The program `fdelmodc` is open source, makes use of the Seismic Unix (SU) parameter interface and output files, and specially aims at the modelling of measurements used for Seismic Interferometry. This means that noisy source signals at random source positions can be modeled for very long recording times using only one program.

The first four sections after the introduction describe the four implemented schemes; acoustic, visco acoustic, elastic, and visco elastic. In section 6 the program parameters are described and in section 7 examples are given how the program can be applied and demonstrates the possibilities of the program. The remainder of this introduction explains the finite-difference approximations for the derivatives used in the first-order systems governing the wave equation, and how the discretization must be chosen for stable and dispersion-free modelling results.

The program `fdelmodc` computes a solution of the 2D wave equation by approximating the derivatives in the wave equation by finite-differences. The wave equation is defined through the first-order linearized

systems of Newton's and Hooke's law. For an acoustic medium the equations are given by;

$$\begin{aligned}\frac{\partial V_x}{\partial t} &= -\frac{1}{\rho} \frac{\partial P}{\partial x}, \\ \frac{\partial V_z}{\partial t} &= -\frac{1}{\rho} \frac{\partial P}{\partial z}, \\ \frac{\partial P}{\partial t} &= -\frac{1}{\kappa} \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\},\end{aligned}\tag{1}$$

where V_x, V_z are the particle velocity components in the x and z -direction, respectively, P the acoustic pressure, ρ is the density of the medium and κ the compressibility.

The first-order derivatives in the spatial coordinates (lateral position x and depth position z) are approximated by a so-called centralized 4'th order Crank-Nicolson approximation,

$$\frac{\partial P}{\partial x} \approx \frac{-P((i + \frac{3}{2})\Delta x) + 27P((i + \frac{1}{2})\Delta x) - 27P((i - \frac{1}{2})\Delta x) + P((i - \frac{3}{2})\Delta x)}{24\Delta x}\tag{2}$$

the first order derivative in time is approximated by a 2th order scheme:

$$\frac{\partial P}{\partial t} \approx \frac{P((i + \frac{1}{2})\Delta t) - P((i - \frac{1}{2})\Delta t)}{\Delta t}.\tag{3}$$

These approximations can be derived from linear combination of different Taylor expansions (Fornberg, 1988):

$$P(x + \Delta x) \approx P(x) + \frac{\Delta x}{1!} \frac{\partial P}{\partial x} + \frac{\Delta x^2}{2!} \frac{\partial^2 P}{\partial x^2} + \frac{\Delta x^3}{3!} \frac{\partial^3 P}{\partial x^3} + \mathcal{O}\Delta x^4\tag{4}$$

For example, a 4th order approximation of a first-order derivative, used in the implemented staggered grid, can be derived from four Taylor expansions on 4 points centered around $x = 0$:

$$\begin{aligned}P(x + \frac{\Delta x}{2}) &\approx P(x) + \frac{\Delta x}{2} \frac{\partial P}{\partial x} + \frac{\Delta x^2}{8} \frac{\partial^2 P}{\partial x^2} + \frac{\Delta x^3}{24} \frac{\partial^3 P}{\partial x^3} + \frac{\Delta x^4}{96} \frac{\partial^4 P}{\partial x^4} + \mathcal{O}\Delta x^5 \\ P(x - \frac{\Delta x}{2}) &\approx P(x) - \frac{\Delta x}{2} \frac{\partial P}{\partial x} + \frac{\Delta x^2}{8} \frac{\partial^2 P}{\partial x^2} - \frac{\Delta x^3}{24} \frac{\partial^3 P}{\partial x^3} + \frac{\Delta x^4}{96} \frac{\partial^4 P}{\partial x^4} + \mathcal{O}\Delta x^5 \\ P(x + \frac{3\Delta x}{2}) &\approx P(x) + \frac{3\Delta x}{2} \frac{\partial P}{\partial x} + \frac{9\Delta x^2}{8} \frac{\partial^2 P}{\partial x^2} + \frac{27\Delta x^3}{24} \frac{\partial^3 P}{\partial x^3} + \frac{81\Delta x^4}{96} \frac{\partial^4 P}{\partial x^4} + \mathcal{O}\Delta x^5 \\ P(x - \frac{3\Delta x}{2}) &\approx P(x) - \frac{3\Delta x}{2} \frac{\partial P}{\partial x} + \frac{9\Delta x^2}{8} \frac{\partial^2 P}{\partial x^2} - \frac{27\Delta x^3}{24} \frac{\partial^3 P}{\partial x^3} + \frac{81\Delta x^4}{96} \frac{\partial^4 P}{\partial x^4} + \mathcal{O}\Delta x^5\end{aligned}$$

Subtracting the expansions of $x - \frac{\Delta x}{2}$ from $x + \frac{\Delta x}{2}$ and subtracting $x - \frac{3\Delta x}{2}$ from $x + \frac{3\Delta x}{2}$ already eliminates the second and fourth order terms (or more general all even-power terms) :

$$\begin{aligned}D_1 &= P(x + \frac{\Delta x}{2}) - P(x - \frac{\Delta x}{2}) \approx \Delta x \frac{\partial P}{\partial x} + \frac{2\Delta x^3}{24} \frac{\partial^3 P}{\partial x^3} + \mathcal{O}\Delta x^5 \\ D_2 &= P(x + \frac{3\Delta x}{2}) - P(x - \frac{3\Delta x}{2}) \approx 3\Delta x \frac{\partial P}{\partial x} + \frac{54\Delta x^3}{24} \frac{\partial^3 P}{\partial x^3} + \mathcal{O}\Delta x^5\end{aligned}$$

Using a linear combination of D_1 and D_2 , to eliminate the third order term, gives the 4th order approximation:

$$\frac{27D_1 - D_2}{24\Delta x} \approx \frac{\partial P}{\partial x} + \mathcal{O}\Delta x^4 \approx \frac{27(P(x + \frac{\Delta x}{2}) - P(x - \frac{\Delta x}{2})) - P(x + \frac{3\Delta x}{2}) + P(x - \frac{3\Delta x}{2})}{24\Delta x} + \mathcal{O}\Delta x^4.\tag{5}$$

The implemented Finite-Difference codes make use of a staggered grid and is following the grid layout as described in Virieux (1986). In the sections for the specific solutions the staggered grid is explained in detail. The implementation of equation 1 is also called a stencil, since it forms a pattern of four grid

point needed to compute the partial derivative at one grid point. To compute the spatial derivative on all grid points the stencil is 'shifted' through the grid.

The medium parameters used in the FD program are

$$(\lambda + 2\mu) = c_p^2 \rho = \frac{1}{\kappa} \quad (6)$$

$$\mu = c_s^2 \rho \quad (7)$$

where ρ is the density of the medium, c_p the P-wave velocity, c_s the S-wave velocity, λ and μ the Lamé parameters and κ the compressibility. The program reads the P (and S-wave for elastic modelling) velocity and medium density as gridded input model files. From these files the program calculates the Lamé parameters used in the first order equations 1 to calculate the wavefield at next time steps.

1.1 Finite-difference algorithm

To simulate passive seismic measurements we have chosen to use a two-dimensional finite-difference (FD) approach based on the work of Virieux (1986) and Robertsson et al. (1994). The main reason for choosing the finite-difference method is that it runs well on standard X86 and multi-core hardware (including graphical cards) and is easy to implement. For the moment, only the two-dimensional case is implemented to gain experience and be able to run many experiments within a short computation time. For reading input parameters and access files on disk, use is made of the Seismic Unix (SU) parameter interface and SU-segy header format with local IEEE floating point representation for the data. Four different schemes are implemented : acoustic, visco-acoustic, elastic, and visco-elastic. We will not go into all the implementation details and only explain the specific aspects related to the modeling of measurements that can be used to study seismic interferometry (SI). The main difference with other finite-difference codes is the possibility to use band-limited noise signatures positioned at random source positions in the subsurface and model the combined effect of all those sources in only one modeling step. Existing modeling codes are able to model the same result, but are less efficient or less user friendly (more than one program is required to do the modeling off all the passive sources). More details about the used algorithm and the other options within the program can be found in the manual distributed with the code. There are not that many good FD codes available as open source, and we hope that by making the code freely available we would receive requests from users to add new options and keep on expanding and improving the functionality of the code.

Following the flow chart of Figure 1 the algorithm is explained step by step. The program starts by reading in the given parameters and together with default values sets up a modeling experiment. The velocity and density models are read in together with the grid spacing. Using the model grid spacing and the defined time sampling a check is made for the stability and dispersion criteria. The random source positions and signature lengths are computed and all arrays are allocated. The source signatures are calculated in advance and is explained in more detail in section 6.4.

The algorithm contains two loops: the outer loop is for the number of shots and the inner loop for the number of time steps to be modeled for each shot. For seismic interferometry modeling with random source positions the number of shots in the outer loop is set to one, all sources will become active within the inner time loop.

Every time step, the FD kernel is called to update the wavefields and inject source amplitudes, followed by storing of wavefield components on the defined receiver positions and, if requested, a snapshot of the wavefield components is written to disk. The last task within one time step is suppressing reflections from the sides of the model by tapering the edges of the wavefields with an exponentially decaying function. After all time steps are calculated, the stored wavefield components at the receiver positions are written to disk.

In summary FD modeling computes a wavefield (at all gridded x,z positions) at time step $T = t + \Delta t$ given the wavefield at the current time step $T = t$. If during the time stepping of the algorithm a start time of a source is encountered the source amplitude is added to the wavefield at the position of the source.

In the inset of Figure 1 the acoustic FD kernel is sketched in more detail. Inside the kernel, the particle-velocity fields V_x and V_z are updated first. If there are sources active on the particle-velocity fields, these source amplitudes are added to the V_x and V_z fields after the update. This is done for all the defined source positions. Free or rigid boundary conditions are then handled. The pressure field P is updated

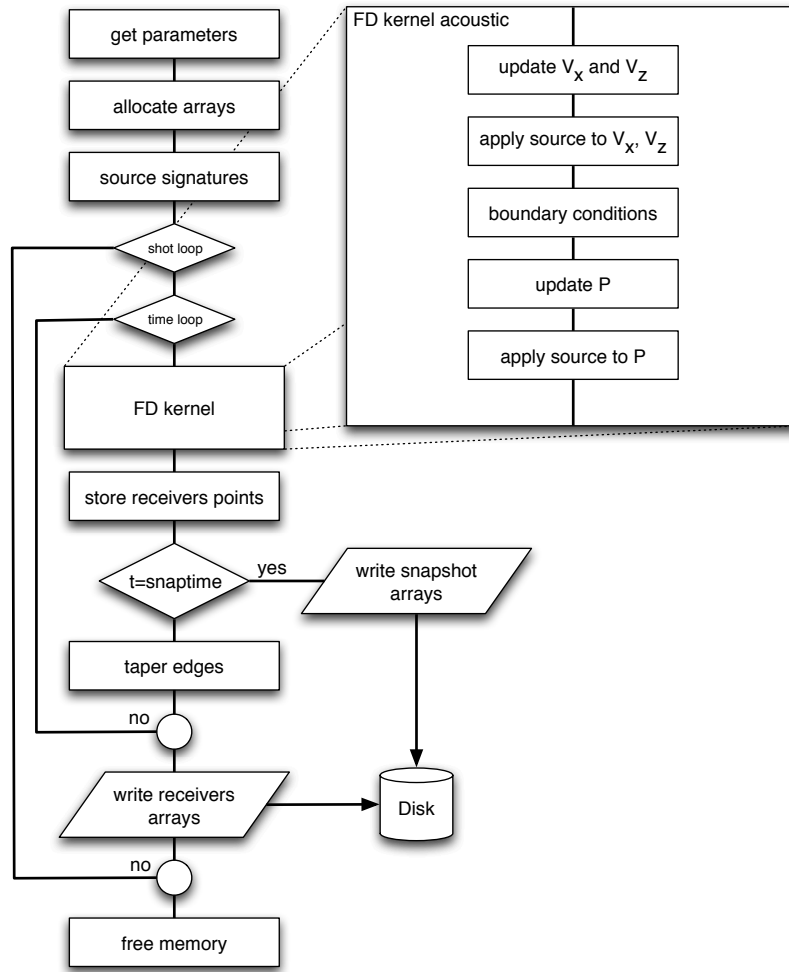


Figure 1: Flow chart of the finite-difference (FD) algorithm. The FD kernel of the acoustic scheme is explained in the onset in more detail. The two decision loops are for the number of shot positions and the number of time steps to be modeled. In the chart, t represents time, V_x and V_z the horizontal and vertical particle-velocity, respectively, and P the acoustic pressure.

next and after the update pressure-source amplitudes are added to the pressure field. This last step completes the FD kernel.

The update of the stress fields (P) is done after the updates of the particle-velocity fields, hence the P field is calculated also staggered with time ($+\frac{1}{2}\Delta t$) compared to the particle-velocity fields. Numerical particle velocity (V_x, V_z) are computed at time $(k + \frac{1}{2})\Delta t$, and numerical stress ($\sigma_{zz} = P, \sigma_{xz}, \sigma_{xx}$) at time $(k + 1)\Delta t$ are computed explicitly from stress at time $k\Delta t$ and from velocity at time $(k - \frac{1}{2})\Delta t$ (Virieux, 1986).

The kernel operators (stencils) are shown in Figure 2. They are the implementation of the finite-difference approximation of a first order derivative as represented in equation 1. A staggered grid implementation has been used. This means that the grids of the V_x and V_z wavefields are positioned in between the P grid.

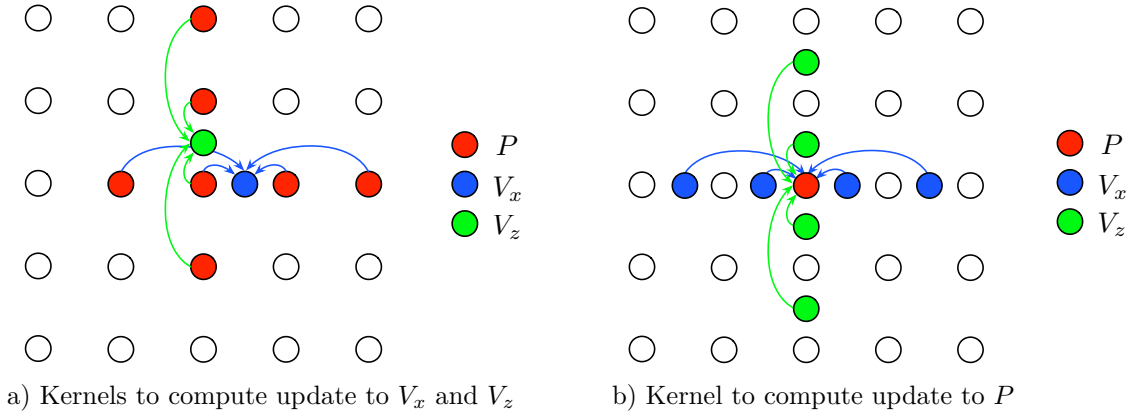


Figure 2: The compute kernels showing the grid points needed to update the V_x and V_z (a) and P (b) wavefields. The wavefields all have a unique grid position. A staggered grid implementation has been used. This means that the grids of the V_x and V_z wavefields are positioned in between the P grid.

1.2 Stability and Dispersion

The first order differential equations are approximated by the finite-difference operators of equations 2 and 3. When explicit time-marching schemes are used for the numerical solution the Courant (Courant et al., 1967) number gives a condition for convergence. The Courant number is used to restrict the time-step in explicit time-marching computer simulations. For example, if a wave is crossing a discrete grid distance (Δx), then the time-step must be less than the time needed for the wave to travel to an adjacent grid point, otherwise the simulation will produce incorrect results. As a corollary, when the grid point separation is reduced, the upper limit for the time step must also decrease. For 4'th order spatial derivatives the Courant number is 0.606 (Sei, 1995) and for stability the discretization must satisfy:

$$\sqrt{\frac{\lambda + 2\mu}{\rho}} \frac{\Delta t}{h} \leq 0.606 \quad (8)$$

$$(9)$$

This approximation requires that

$$\Delta t < \frac{0.606 \Delta h}{c_{max}} \quad (10)$$

with $\Delta h = \Delta x = \Delta z$ being the discretization step in the spatial dimensions. If equation 10 is not satisfied unstable results will be calculated if, within a time step Δt , the wavefront has travelled a distance larger than $0.606 \Delta x$. This will typically occur at high velocities when $\Delta t c_{max}$ is large. The unstable solution will propagate though the whole model and can end up with large numbers and the out-of-range representation NaN (Not a Number).

Besides unstable solutions wavefield dispersion can also occur. Unfortunately, finite-difference schemes are intrinsically dispersive and there is no fixed grid points per wavelength rule that can be given to avoid dispersion. The widespread rule of thumb "5 points per wavelength" for a (2,4) scheme (Alford et al., 1974) has to be understood in the sense "5 points per wavelength for an average geophysical medium," and for the propagation of a 100 wavelengths through the medium (Sei, 1995). Dispersion for the 2D wave-equation will occur more strongly and clearly visible if the following relation is not obeyed,

$$\Delta h < \frac{c_{min}}{5f_{max}}, \quad (11)$$

$$\Delta h < \frac{\lambda_{min}}{5}$$

and will occur at small wavelengths (λ_{min} , low velocities and/or high frequencies). In the case of dispersion, the program will keep on running but will give dispersive waves. Do not confuse numerical dispersion with the physical dispersion of visco-elastic waves discussed later.

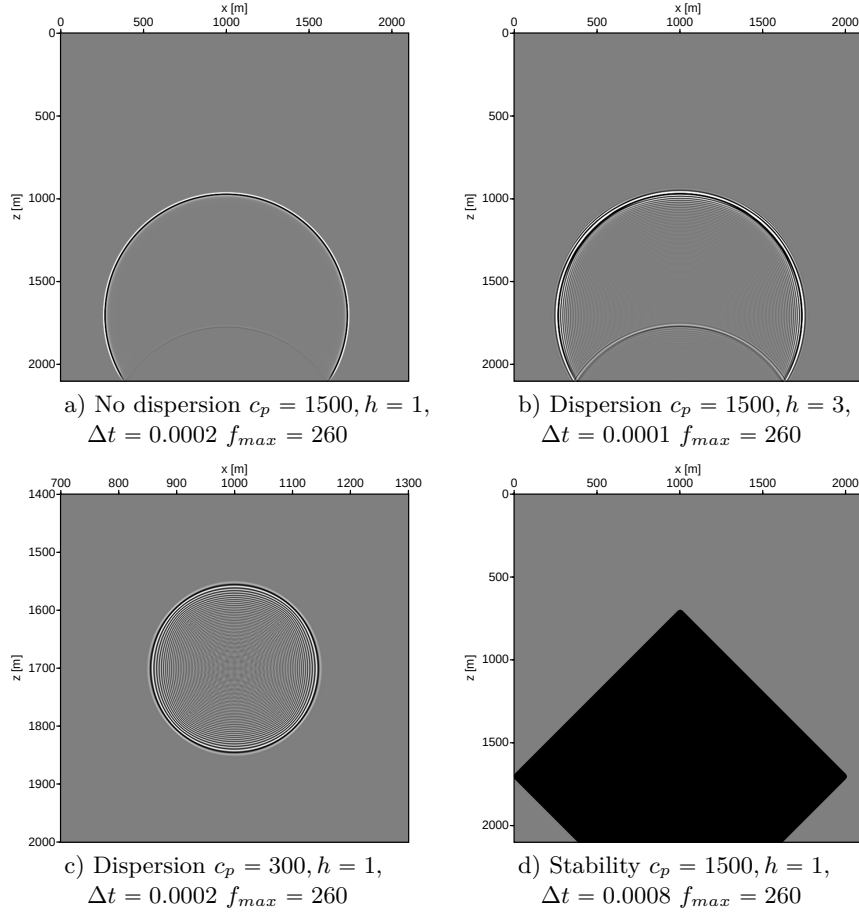


Figure 3: Snapshots of dispersion (b and c) and unstable (d) schemes. The script `fdelmodc_stab.scr` in the demo directory reproduces the pictures.

Figure 3 shows different snapshots with no dispersion (a), dispersion (b and c) and the results for an unstable scheme (d). Note that before starting the calculating the program checks if the stability and dispersion equations 10 and 11 are satisfied. If they are not satisfied the programs stops with an error message and suggestion how to change the discretization interval or maximum frequency, to get a stable scheme. The dispersion check can be overruled by using the `fmax=` parameter smaller than the actual maximum frequency found in the source wavelet. For a more detailed discussion about stability in finite-difference schemes see Sei (1995), Sei and Symes (1995) Bauer et al. (2008).

Unfortunately, the stability and dispersion criteria shown in equation 10 is not valid for visco-elastic media. See the end of section 5 for some guidelines.

2 Acoustic

The linearized equation of motion (Newton's second law) and equation of deformation (Hook's law) are given by:

$$\begin{aligned}\frac{\partial V_x}{\partial t} &= -\frac{1}{\rho} \frac{\partial P}{\partial x}, \\ \frac{\partial V_z}{\partial t} &= -\frac{1}{\rho} \frac{\partial P}{\partial z}, \\ \frac{\partial P}{\partial t} &= -\frac{1}{\kappa} \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\},\end{aligned}\tag{12}$$

where V_x, V_z are the particle velocity components in the x and z -direction, respectively, and P the acoustic pressure. In the staggered-grid implementation, ρ_x and V_x , ρ_z and V_z , and $\rho c_p^2 = \frac{1}{\kappa}$ and P are

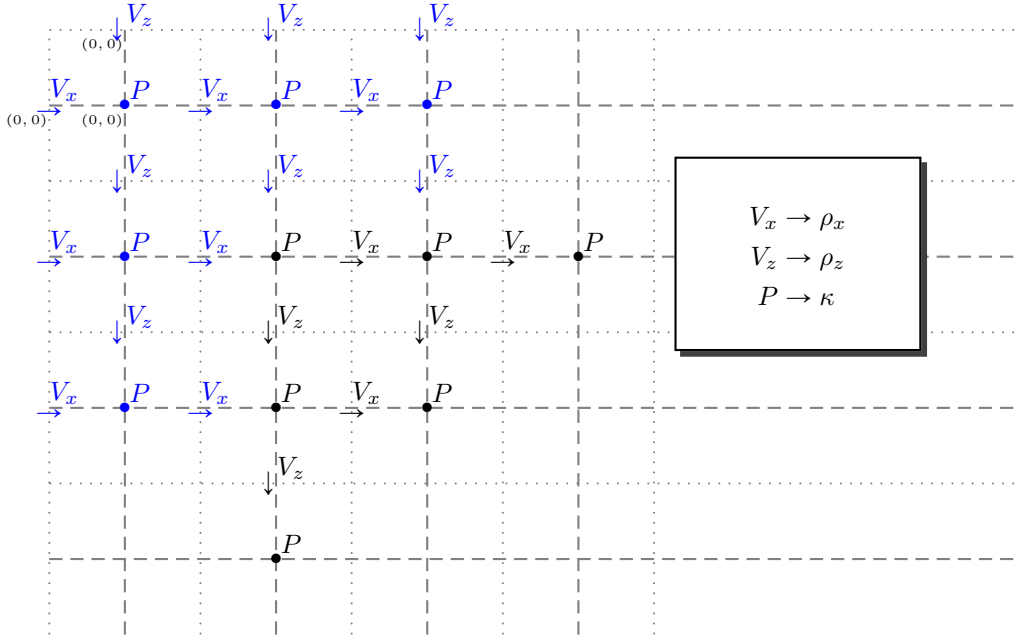


Figure 4: *Acoustic staggered calculation grid for a fourth-order scheme in space.* V_z, V_x represent the particle velocity of the wavefield in the z and x direction, respectively, and P represent the acoustic pressure. The blue fields are auxiliary points used to calculate the black field values. Those blue points are not updated and initialized to zero. On all sides of the model a virtual V_x or V_z layer has been added for proper handling of the edges of the model.

put on the same calculation grid. The computational grid (represented by ρ and c_p) is placed at an offset (one or two grid-points, depending of the field component) for efficient handling of the boundaries.

The pressure/stress wavefields are computed on different time steps than the particle-velocity fields. In the algorithm the first time-step ($it = 0$) computed for the particle-velocity fields uses pressure/stress fields at $it = 0$ resulting in particle-velocity fields at time steps $(it + \frac{1}{2})\Delta t$. The pressure/stress fields are computed at time steps $it\Delta t$ using the particle-velocity fields at time steps $(it + \frac{1}{2})\Delta t$.

For the staggered-grid implementation, shown in Figure 4, every field quantity has a different origin. The origins of the field are chosen in such a way that interpolation of one field grid to another field grid can be done in a straightforward way (see also section 6.6.3). The derivative operators need two points on each sides of their centre to calculate the derivative at the centre. By offsetting the grid, the extra points needed to calculate the derivative at the boundaries of the model are added. These extra layers, needed at the edges of the model, are also taken into account in the choice of the origin. The origins of the medium parameters (and the different fields) are defined according to the following mapping:

$$\begin{aligned}
 [z, x] \\
 \rho_x[1, 2] &\leftarrow 0.5 * (\rho[0, 0] + \rho[0, 1]) \\
 \rho_z[2, 1] &\leftarrow 0.5 * (\rho[0, 0] + \rho[1, 0]) \\
 \kappa[1, 1] &\leftarrow c_p^2[0, 0]\rho[0, 0].
 \end{aligned}$$

Note that the choice for the origin is just a choice for convenience and nothing else.

In the code section below the `io**` variables define the origin-offsets used in the calculations.

2.1 Staggered scheme

```

c1 = 9.0/8.0;
c2 = -1.0/24.0;

/* Vx: rox */
ioXx=2;
ioXz=ioXx-1;

```

```

/* Vz: roz */
ioZz=2;
ioZx=ioZz-1;
/* P, Txx, Tzz: lam, l2m */
ioPx=1;
ioPz=ioPx;
/* Txz: muu */
ioTx=2;
ioTz=ioTx;

rox = 1/rho_x * dt/dx
roz = 1/rho_z * dt/dx
l2m = cp*cp*rho * dt/dx

/* calculate vx for all grid points except on the virtual boundary*/
for (ix=ioXx; ix<nx+1; ix++) {
    for (iz=ioXz; iz<nz+1; iz++) {
        vx[ix*n1+iz] -= rox[ix*n1+iz]*(
            c1*(p[ix*n1+iz] - p[(ix-1)*n1+iz]) +
            c2*(p[(ix+1)*n1+iz] - p[(ix-2)*n1+iz]));
    }
}

/* calculate vz for all grid points except on the virtual boundary */
for (ix=ioZx; ix<nx+1; ix++) {
    for (iz=ioZz; iz<nz+1; iz++) {
        vz[ix*n1+iz] -= roz[ix*n1+iz]*(
            c1*(p[ix*n1+iz] - p[ix*n1+iz-1]) +
            c2*(p[ix*n1+iz+1] - p[ix*n1+iz-2]));
    }
}

/* calculate p/tzz for all grid points except on the virtual boundary */
for (ix=ioPx; ix<nx+1; ix++) {
    for (iz=ioPz; iz<nz+1; iz++) {
        p[ix*n1+iz] -= l2m[ix*n1+iz]*(
            c1*(vx[(ix+1)*n1+iz] - vx[ix*n1+iz]) +
            c2*(vx[(ix+2)*n1+iz] - vx[(ix-1)*n1+iz]) +
            c1*(vz[ix*n1+iz+1] - vz[ix*n1+iz]) +
            c2*(vz[ix*n1+iz+2] - vz[ix*n1+iz-1]));
    }
}

```

2.2 Computing SH-waves with acoustic wave equation

Reference to paper of Kees
 Example from Faezeh.

3 Visco-Acoustic

For a visco-acoustic medium the linearized equation of motion (Newton's second law) and equation of deformation (Hook's law) are :

$$\frac{\partial V_x}{\partial t} = -\frac{1}{\rho} \frac{\partial P}{\partial x} \quad (13)$$

$$\frac{\partial V_z}{\partial t} = -\frac{1}{\rho} \frac{\partial P}{\partial z} \quad (14)$$

$$\frac{\partial P}{\partial t} = -\frac{1}{\kappa} \frac{\tau_\epsilon^p}{\tau_\sigma} \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\} + r_p \quad (15)$$

$$\frac{\partial r_p}{\partial t} = -\frac{1}{\tau_\sigma} \left(r_p + \left(\frac{\tau_\epsilon^p}{\tau_\sigma} - 1 \right) \left(\frac{1}{\kappa} \right) \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\} \right) \quad (16)$$

For the attenuation implementation a leap-frog scheme in time is used and shown in the implementation below. The so-called memory variable r_p (Robertsson et al., 1994) are introduced for the relaxation mechanism.

```

/* calculate p/tzz for all grid points except on the virtual boundary */
for (ix=ioPx; ix<nx+1; ix++) {
    for (iz=ioPz; iz<nz+1; iz++) {
        dxvx[iz] = c1*(vx[(ix+1)*n1+iz] - vx[ix*n1+iz]) +
            c2*(vx[(ix+2)*n1+iz] - vx[(ix-1)*n1+iz]);
    }
    for (iz=ioPz; iz<nz+1; iz++) {
        dzvz[iz] = c1*(vz[ix*n1+iz+1] - vz[ix*n1+iz]) +
            c2*(vz[ix*n1+iz+2] - vz[ix*n1+iz-1]);
    }

    /* help variables to let the compiler vectorize the loops */
    for (iz=ioPz; iz<nz+1; iz++) {
        Tpp = tep[ix*n1+iz]*tss[ix*n1+iz];
        Tlm[iz] = (1.0-Tpp)*tss[ix*n1+iz]*l2m[ix*n1+iz]*0.5;
        Tlp[iz] = l2m[ix*n1+iz]*Tpp;
    }
    for (iz=ioPz; iz<nz+1; iz++) {
        Tt1[iz] = 1.0/(ddt+0.5*tss[ix*n1+iz]);
        Tt2[iz] = ddt-0.5*tss[ix*n1+iz];
    }

    /* the update with the relaxation correction */
    for (iz=ioPz; iz<nz+1; iz++) {
        p[ix*n1+iz] -= Tlp[iz]*(dzvz[iz]+dxvx[iz]) + q[ix*n1+iz];
    }
    for (iz=ioPz; iz<nz+1; iz++) {
        q[ix*n1+iz] = (Tt2[iz]*q[ix*n1+iz] + Tlm[iz]*(dxvx[iz]+dzvz[iz]))*Tt1[iz];
        p[ix*n1+iz] += q[ix*n1+iz];
    }
}

```

The relaxation parameters $\tau_\epsilon^p, \tau_\sigma$ are defined in the program indirectly by defining the Quality factor (Q). This Q-factor can be defined in the program by setting the parameter **Qp=** for a constant-Q medium, or by using a gridded file **file_qp=**, which defines a different Q-factor for every grid point. The Q-factors are transformed inside the program to the relaxation parameters (used in the numerical scheme) by using (Robertsson et al., 1994; Blanch et al., 1996; Bohlen, 2002).

The Zener (1948) or standard linear solid model is the most general linear equation in stress σ , strain ϵ and their first time derivatives (Wang, 2008). In the Fourier transform domain the stress and the strain

are, in the Zener model, then linked by the complex modulus defined as (Ben-Menahem and Singh, 1981):

$$M(\omega) = M_r \left(1 - L + \sum_{l=1}^L \frac{1 + j\omega t_{e,l}}{1 + j\omega t_{s,l}} \right) \quad (17)$$

$$Q = \frac{\Re\{M(\omega)\}}{\Im\{M(\omega)\}} \quad (18)$$

where M_r is a (relaxed) deformation modulus and is obtained from $\omega \rightarrow 0$ Wang (2008). The real part of the complex modulus $M(\omega)$ for $L = 1$, is given by (Blanch et al., 1996; Wang, 2008)

$$\Re\{M(\omega)\} = M_r \frac{(1 + \omega^2 \tau_\sigma \tau_\varepsilon)}{(1 + \omega^2 \tau_\sigma^2)} \quad (19)$$

and the imaginary part by

$$\Im\{M(\omega)\} = M_r \frac{(\tau_\varepsilon - \tau_\sigma) \omega}{(1 + \omega^2 \tau_\sigma^2)} \quad (20)$$

This gives for $L=1$ (see also to Blanch et al. (1996) eq. 2.11),

$$Q = \frac{(1 + \omega^2 \tau_\sigma \tau_\varepsilon)}{(\tau_\varepsilon - \tau_\sigma) \omega} \quad (21)$$

It is found that stress waves whose periods are close to the 'relaxation times' of such a medium are severely attenuated when passing through it (Wang, 2008). Defining a single relaxation time as

$$\tau_c = \sqrt{\tau_\sigma \tau_\varepsilon} \quad (22)$$

and by choosing the single relaxation time

$$\tau_c = \frac{1}{f_\omega}, \quad (23)$$

with f_ω the reference angular frequency (usually chosen to be the central frequency of the source wavelet), this attenuation will be strongest at the central frequency. The stress and strain relaxation times of the Zener Body (Dutta and Schuster (2014) equation 10, Wang (2008) equation 3.60), τ_σ and τ_ε in equation 24 and ??, are related to the quality factor Q as in Robertsson et al. (1994);

$$\tau_\sigma = \frac{\sqrt{1.0 + \frac{1.0}{Q_p^2} - \frac{1.0}{Q_p}}}{f_\omega} \quad (24)$$

$$\tau_\varepsilon^p = \frac{1.0}{f_\omega^2 \tau_\sigma} \quad (25)$$

$$\tau_\varepsilon^s = \frac{1.0 + f_\omega Q_s \tau_\sigma}{f_\omega Q_s - f_\omega^2 \tau_\sigma} \quad (26)$$

where the central frequency (of the used wavelet) is given by parameter **fw** in the program.

Note, $M(\omega)$ in equation 17 is defined in the frequency domain and this model is approximated to work also in the time domain for finite difference. There is no direct translation possible between the $t_{e,l}, t_{s,l}$ and the values in use by the finite difference program. For $l = 1$ the notation of these variables is shortened as $t_{e,1} = \tau_\varepsilon$ and $t_{s,1} = \tau_\sigma$. Equation 22 and 23 show that $\tau_\sigma, \tau_\varepsilon$ define a value τ_c that is in use by the finite difference program. In the finite difference program the input values are **qp**= (textttqs=) and textttfw=. From equation 24 τ_σ is computed and with equation 25 and 26 τ_ε^p and τ_ε^s (these represent τ_c of equation 23 for P and S waves) and these values define the attenuation in Finite difference.

In the code these variables are computed as follows:

```
a = (sqrt(1.0+(1.0/(qp[iz]*qp[iz])))-(1.0/qp[iz]))/mod.fw; ==> equation 24
b = 1.0/(mod.fw*mod.fw*a); ==> equation 25
tss[(i+ioPx)*n1+iz+ioPz] = 1.0/a;
tep[(i+ioPx)*n1+iz+ioPz] = b;
```

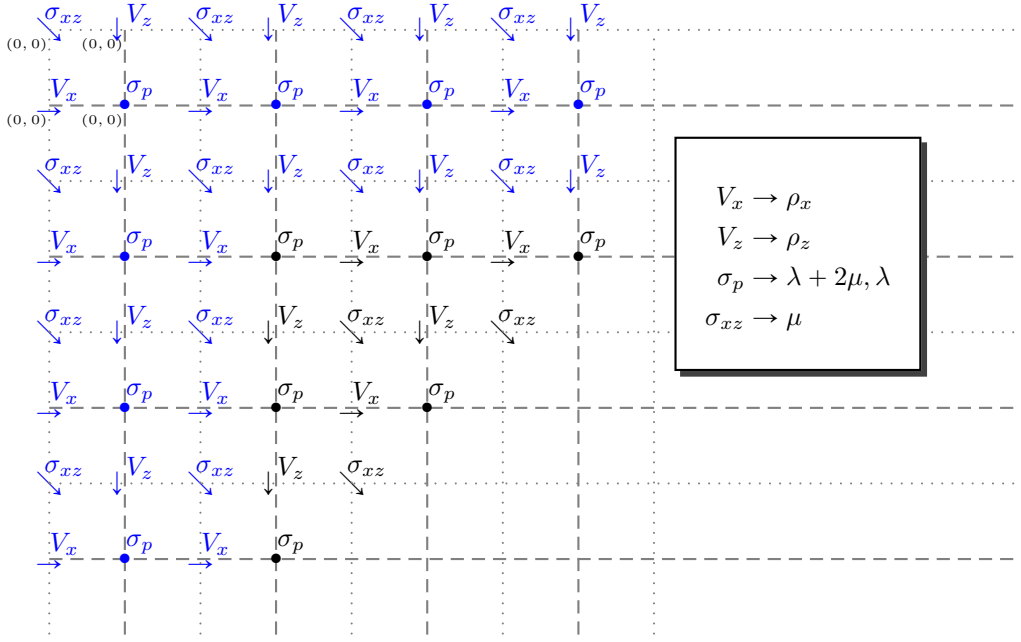


Figure 5: *Elastic staggered calculation grid for a fourth-order scheme in space.* V_z, V_x represent the particle velocity of the wavefield in the z and x direction, respectively, and $\sigma_p(\sigma_{xx}$ or $\sigma_{zz}), \sigma_{xz}$ represent the stress fields. The blue fields are auxiliary points used to calculate the black field values. Those blue points are not updated and initialized to zero. On all sides of the model a virtual V_x, σ_{xz} or V_z, σ_{xz} layer has been added for proper handling of the edges of the model.

and $\mathbf{tss}=1/\tau_\epsilon^p$ and $\mathbf{tep}=\tau_\sigma$ are in use by equation 15 and 16.

In viscoelastic media, wavenumber k becomes a complex function, defined as (Wang, 2008)

$$k(\omega) = \frac{\omega}{c(\omega)} = \frac{\omega}{v(\omega)} + j\alpha(\omega) \quad (27)$$

$$c(\omega) = \sqrt{\frac{M(\omega)}{\rho}} \quad (28)$$

where $c(\omega)$ is the complex velocity and ρ is the density. This shows that $\alpha(\omega)$ is related to the amplitude attenuation, and $\frac{\omega}{v(\omega)}$ is related to the phase change along the propagation distance. For implementation in the frequency domain the expression of $c(\omega)$ is used.

In the program **green** (see for a full description of all variables section 7.6) the analytical Green's functions for volume injection sources is given by :

$$\hat{G}^{p,p}(\mathbf{x}, \mathbf{x}_s) = P^{mon} = \frac{-j\rho}{4} H_0^{(2)}(kr), \quad (29)$$

where

$$r = \sqrt{(x^2 + (z_s - z_r)^2)}, \quad (30)$$

and k defined as in equation (27).

4 Elastic

Linearized equation of motion (Newton's second law) and equation of deformation (Hook's law) are used:

$$\frac{\partial V_x}{\partial t} = -\frac{1}{\rho} \left\{ \frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xz}}{\partial z} \right\} \quad (31)$$

$$\frac{\partial V_z}{\partial t} = -\frac{1}{\rho} \left\{ \frac{\partial \sigma_{xz}}{\partial x} + \frac{\partial \sigma_{zz}}{\partial z} \right\} \quad (32)$$

$$\frac{\partial \sigma_{xx}}{\partial t} = -\left\{ \frac{1}{\kappa} \frac{\partial V_x}{\partial x} + \lambda \frac{\partial V_z}{\partial z} \right\} \quad (33)$$

$$\frac{\partial \sigma_{zz}}{\partial t} = -\left\{ \frac{1}{\kappa} \frac{\partial V_z}{\partial z} + \lambda \frac{\partial V_x}{\partial x} \right\} \quad (34)$$

$$\frac{\partial \sigma_{xz}}{\partial t} = -\mu \left\{ \frac{\partial V_x}{\partial z} + \frac{\partial V_z}{\partial x} \right\} \quad (35)$$

where σ_{ij} denotes the ij th component of the symmetric stress tensor Virieux (1986).

The derivative operators need two points on each side of their centre to calculate the derivative at the centre. By offsetting the grid, the extra points needed to calculate the derivative at the boundaries of the model are added. These extra layers needed at the edges of the model are also taken into account in the choice of the origin. The origins are defined according to the following mapping:

```

[z,x]
rho_x[1,2] ← 0.5 * (rho[0,0] + rho[0,1])
rho_z[2,1] ← 0.5 * (rho[0,0] + rho[1,0])
kappa[1,1] ← c_p^2[0,0]rho[0,0]
mu[2,2] ← c_s^2[0,0]rho[0,0]
lambda[1,1] ← c_p^2[0,0]rho[0,0] - 2c_p^2[0,0]rho[0,0].

/* Vx: rox */
ioXx=mod.iorder/2;
ioXz=ioXx-1;
/* Vz: roz */
ioZz=mod.iorder/2;
ioZx=ioZz-1;
/* P, Txx, Tzz: lam, l2m */
ioPx=mod.iorder/2-1;
ioPz=ioPx;
/* Txz: muu */
ioTx=mod.iorder/2;
ioTz=ioTx;

/* calculate vx for all grid points except on the virtual boundary*/
for (ix=ioXx; ix<nx+1; ix++) {
  for (iz=ioXz; iz<nz+1; iz++) {
    vx[ix*n1+iz] -= rox[ix*n1+iz]*(
      c1*(txx[ix*n1+iz] - txx[(ix-1)*n1+iz] +
        txz[ix*n1+iz+1] - txz[ix*n1+iz]) +
      c2*(txx[(ix+1)*n1+iz] - txx[(ix-2)*n1+iz] +
        txz[ix*n1+iz+2] - txz[ix*n1+iz-1]) );
  }
}

/* calculate vz for all grid points except on the virtual boundary */
for (ix=ioZx; ix<nx+1; ix++) {
  for (iz=ioZz; iz<nz+1; iz++) {

```

```

        vz[ix*n1+iz] -= roz[ix*n1+iz]*(
            c1*(tzz[ix*n1+iz]      - tzz[ix*n1+iz-1] +
                txz[(ix+1)*n1+iz] - txz[ix*n1+iz]) +
            c2*(tzz[ix*n1+iz+1]    - tzz[ix*n1+iz-2] +
                txz[(ix+2)*n1+iz] - txz[(ix-1)*n1+iz]) );
    }
}

/* calculate Txx/tzz for all grid points except on the virtual boundary */
for (ix=ioPx; ix<nx+1; ix++) {
    for (iz=ioPz; iz<nz+1; iz++) {
        dvvx[iz] = c1*(vx[(ix+1)*n1+iz] - vx[ix*n1+iz]) +
            c2*(vx[(ix+2)*n1+iz] - vx[(ix-1)*n1+iz]);
    }
    for (iz=ioPz; iz<nz+1; iz++) {
        dvvz[iz] = c1*(vz[ix*n1+iz+1] - vz[ix*n1+iz]) +
            c2*(vz[ix*n1+iz+2] - vz[ix*n1+iz-1]);
    }
    for (iz=ioPz; iz<nz+1; iz++) {
        txx[ix*n1+iz] -= l2m[ix*n1+iz]*dvvx[iz] + lam[ix*n1+iz]*dvvz[iz];
        tzz[ix*n1+iz] -= l2m[ix*n1+iz]*dvvz[iz] + lam[ix*n1+iz]*dvvx[iz];
    }
}

/* calculate Txz for all grid points except on the virtual boundary */
for (ix=ioTx; ix<nx+1; ix++) {
    for (iz=ioTz; iz<nz+1; iz++) {
        txz[ix*n1+iz] -= mul[ix*n1+iz]*(
            c1*(vx[ix*n1+iz]      - vx[ix*n1+iz-1] +
                vz[ix*n1+iz]      - vz[(ix-1)*n1+iz]) +
            c2*(vx[ix*n1+iz+1]    - vx[ix*n1+iz-2] +
                vz[(ix+1)*n1+iz] - vz[(ix-2)*n1+iz]) );
    }
}

```

To handle a solid fluid interface an extra layer is introduced between the interfaces. This technique is described by van Vossen et al. (2002).

5 Visco-Elastic

Linearized equation of motion (Newton's second law) and equation of deformation (Hook's law) used are:

$$\frac{\partial V_x}{\partial t} = -\frac{1}{\rho} \left\{ \frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xz}}{\partial z} \right\} \quad (36)$$

$$\frac{\partial V_z}{\partial t} = -\frac{1}{\rho} \left\{ \frac{\partial \sigma_{xz}}{\partial x} + \frac{\partial \sigma_{zz}}{\partial z} \right\} \quad (37)$$

$$\frac{\partial \sigma_{xx}}{\partial t} = -\frac{1}{\kappa} \frac{\tau_\varepsilon^p}{\tau_\sigma} \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\} - 2\mu \frac{\tau_\varepsilon^s}{\tau_\sigma} \frac{\partial V_z}{\partial z} + r_{xx} \quad (38)$$

$$\frac{\partial \sigma_{zz}}{\partial t} = -\frac{1}{\kappa} \frac{\tau_\varepsilon^p}{\tau_\sigma} \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\} - 2\mu \frac{\tau_\varepsilon^s}{\tau_\sigma} \frac{\partial V_x}{\partial x} + r_{zz} \quad (39)$$

$$\frac{\partial \sigma_{xz}}{\partial t} = -\mu \frac{\tau_\varepsilon^s}{\tau_\sigma} \left\{ \frac{\partial V_x}{\partial z} + \frac{\partial V_z}{\partial x} \right\} + r_{xz} \quad (40)$$

$$\frac{\partial r_{xx}}{\partial t} = -\frac{1}{\tau_\sigma} \left(r_{xx} + \left(\frac{\tau_\varepsilon^p}{\tau_\sigma} - 1 \right) \left(\frac{1}{\kappa} \right) \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\} - \left(\frac{\tau_\varepsilon^s}{\tau_\sigma} - 1 \right) 2\mu \frac{\partial V_z}{\partial z} \right) \quad (41)$$

$$\frac{\partial r_{zz}}{\partial t} = -\frac{1}{\tau_\sigma} \left(r_{zz} + \left(\frac{\tau_\varepsilon^p}{\tau_\sigma} - 1 \right) \left(\frac{1}{\kappa} \right) \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\} - \left(\frac{\tau_\varepsilon^s}{\tau_\sigma} - 1 \right) 2\mu \frac{\partial V_x}{\partial x} \right) \quad (42)$$

$$\frac{\partial r_{xz}}{\partial t} = -\frac{1}{\tau_\sigma} \left(r_{xz} + \left(\frac{\tau_\varepsilon^s}{\tau_\sigma} - 1 \right) \mu \left\{ \frac{\partial V_x}{\partial z} + \frac{\partial V_z}{\partial x} \right\} \right) \quad (43)$$

More details about visco-elastic FD modelling can be found in Robertsson et al. (1994), Saenger and Bohlen (2004), and Bohlen (2002).

The relaxation parameters $\tau_\varepsilon^p, \tau_\varepsilon^s, \tau_\sigma$ are defined in the program indirectly by defining the Quality factor (Q). This Q-factor can be defined in the program by setting the parameter **Qp= Qs=** for a constant-Q medium, or by using a gridded file **file_qp= file_qs=** which defines a different Q-factor for every grid point. The Q-factors are in the program transformed into the relaxation parameters (used in the numerical scheme) by using Robertsson et al. (1994):

$$Q = \frac{t_s (1 + w^2 t_s t_e)}{(t_e - t_s) w t_s} \quad (44)$$

$$\tau_\sigma = \frac{\sqrt{1.0 + \frac{1.0}{Q_p^2}} - \frac{1.0}{Q_p}}{f_w} \quad (45)$$

$$\tau_\varepsilon^p = \frac{1.0}{f_w^2 \tau_\sigma} \quad (46)$$

$$\tau_\varepsilon^s = \frac{1.0 + f_w Q_s \tau_\sigma}{f_w Q_s - f_w^2 \tau_\sigma} \quad (47)$$

where f_w is the central frequency (of the used wavelet) given by parameter **fw=**.

The relaxation parameters are defined by the following damping model:

$$M(\omega) = M_r \left(1 - L \sum_{l=1}^L \frac{1 + j\omega t_{e,l}}{1 + j\omega t_{s,l}} \right) \quad (48)$$

$$Q = \frac{\Re\{M(\omega)\}}{\Im\{M(\omega)\}} \quad (49)$$

Unfortunately, the stability calculations within the program are not always valid for visco-elastic media. There is no general rule of thumb for visco-elastic media to calculate a stability criterion. If in a modeling experiment, with a chosen Qp and Qs, the program is not stable, the advise is to increase the smallest Q-factor (for example from 20 to 30), or use **ischeme=3**, and see if that gives a stable modeling result. If you get a stable result by increasing Q (or **ischeme=3**) and you want to use the lower Q, value you have to make the dx (and possibly dt) smaller to get stable answers for that Q value as well.

6 Parameters in program fdelmodc

The self-doc of the program is shown by typing `fdelmodc` on the command line without any arguments. You will then see the following exhaustive list of parameters:

`fdelmodc` - elastic acoustic finite difference wavefield modeling

IO PARAMETERS:

`file_cp=` P (cp) velocity file
`file_cs=` S (cs) velocity file
`file_den=` density (ro) file
`file_src=` file with source signature
`file_rcv=rcv.su` .. base name for receiver files
`file_snap=snap.su` . base name for snapshot files
`file_beam=beam.su` . base name for beam fields
`dx=` read from model file: if `dx==0` then `dx=` can be used to set it
`dz=` read from model file: if `dz==0` then `dz=` can be used to set it
`dt=` read from `file_src`: if `dt` is set it will interpolate `file_src` to `dt` sampling

OPTIONAL PARAMETERS:

`ischeme=3` 1=acoustic, 2=visco-acoustic 3=elastic, 4=visco-elastic, 5=double-couple
`tmod=(nt-1)*dt` total modeling time (nt from `file_src`)
`ntaper=0` length of taper in points at edges of model
`npml=35` length of PML layer in points at edges of model
`R=1e-4` the theoretical reflection coefficient at PML boundary
`m=2.0` scaling order of the PML sigma function
`tapfact=0.30` taper strength: larger value gets stronger taper
For the 4 boundaries the options are: 1=free 2=pml 3=rigid 4=taper
`top=1` type of boundary on top edge of model
`left=4` type of boundary on left edge of model
`right=4` type of boundary on right edge of model
`bottom=4` type of boundary on bottom edge of model
`grid_dir=0` direction of time modeling (1=reverse time)
`Qp=15` global Q-value for P-waves in visco-elastic (ischeme=2,4)
`file_qp=` model file Qp values as function of depth
`Qs=Qp` global Q-value for S-waves in visco-elastic (ischeme=4)
`file_qs=` model file Qs values as function of depth
`fw=0.5*fmax` central frequency for which the Q's are used
`sinkdepth=0` receiver grid points below topography (defined bij `cp=0.0`)
`sinkdepth_src=0` ... source grid points below topography (defined bij `cp=0.0`)
`sinkvel=0` use velocity of first receiver to sink through to next layer
`beam=0` calculate energy beam of wavefield in model
`disable_check=0` ... disable stability and dispersion check and continue modeling
`verbose=0` silent mode; =1: display info

SHOT AND GENERAL SOURCE DEFINITION:

`src_type=1` 1=P 2=Txx 3=Tzz 4=Txx 5=S-pot 6=Fx 7=Fz 8=P-pot 9=double-couple 10=Fz by P 11=m
`src_orient=1` orientation of the source
 - 1=monopole
 - 2=dipole +/- vertical oriented
 - 3=dipole - + horizontal oriented
 - 4=dipole +/-
 - 5=dipole + -
`dip=0.0` dip for double-couple source
`strike=90.0` strike for double-couple source
`rake=90.0` rake for double-couple source
`Mxx=1.0` xx component for the moment tensor source

Mzz=1.0 zz component for the moment tensor source
Mxz=1.0 xz and zx component for the moment tensor source
xsrc=middle x-position of (first) shot
zsrc=zmin z-position of (first) shot
nshot=1 number of shots to model
dxshot=dx if nshot > 1: x-shift in shot locations
dzshot=0 if nshot > 1: z-shift in shot locations
xsrca= defines source array x-positions
zsrca= defines source array z-positions
src_txt=..... text file with source coordinates. Col 1: x, Col. 2: z
wav_random=1 1 generates (band limited by fmax) noise signatures
fmax=from_src maximum frequency in wavelet
src_multiwav=0 use traces in file_src as areal source
src_at_rcv=1 inject wavefield at receiver coordinates (1), inject at source (0)
src_injectionrate=0 set to 1 to use injection rate source

PLANE WAVE SOURCE DEFINITION:

plane_wave=0 model plane wave with nsrc= sources
nsrc=1 number of sources per (plane-wave) shot
src_angle=0 angle of plane source array
src_velo=1500 velocity to use in src_angle definition
src_window=0 length of taper at edges of source array

RANDOM SOURCE DEFINITION FOR SEISMIC INTERFEROMETRY:

src_random=0 1 enables nsrc random sources positions in one modeling
nsrc=1 number of sources to use for one shot
xsrc1=0 left bound for x-position of sources
xsrc2=0 right bound for x-position of sources
zsrc1=0 left bound for z-position of sources
zsrc2=0 right bound for z-position of sources
tsrc1=0.0 begin time interval for random sources being triggered
tsrc2=tmod end time interval for random sources being triggered
tactive=tsrc2 end time for random sources being active
tlength=tsrc2-tsrc1 average duration of random source signal
length_random=1 ... duration of source is rand*tlength
amplitude=0 distribution of source amplitudes
distribution=0 random function for amplitude and tlength 0=flat 1=Gaussian
seed=10 seed for start of random sequence

SNAP SHOT SELECTION:

tsnap1=0.1 first snapshot time (s)
tsnap2=0.0 last snapshot time (s)
dtsnap=0.1 snapshot time interval (s)
dxsnap=dx sampling in snapshot in x-direction
xsnap1=0 first x-position for snapshots area
xsnap2=0 last x-position for snapshot area
dzsnap=dz sampling in snapshot in z-direction
zsnap1=0 first z-position for snapshots area
zsnap2=0 last z-position for snapshot area
snapwithbnd=0 write snapshots with absorbing boundaries
sna_type_p=1 p registration _sp
sna_type_vz=1 Vz registration _svz
sna_type_vx=0 Vx registration _svx
sna_type_txx=0 Txx registration _stxx
sna_type_tzz=0 Tzz registration _stzz
sna_type_txz=0 Txz registration _stxz
sna_type_pp=0 P (divergence) registration _sP

```

sna_type_ss=0 ..... S (curl) registration _sS
sna_vxvztime=0 .... registration of vx/vz times
                        The fd scheme is also staggered in time.
                        Time at which vx/vz snapshots are written:
                        - 0=previous vx/vz relative to txx/tzz/txz at time t
                        - 1=next      vx/vz relative to txx/tzz/txz at time t

```

RECEIVER SELECTION:

```

xrcv1=xmin ..... first x-position of linear receiver array(s)
xrcv2=xmax ..... last x-position of linear receiver array(s)
dxrcv=dx ..... x-position increment of receivers in linear array(s)
zrcv1=zmin ..... first z-position of linear receiver array(s)
zrcv2=zrcv1 ..... last z-position of linear receiver array(s)
dzrcv=0.0 ..... z-position increment of receivers in linear array(s)
dtrcv=.004 ..... desired sampling in receiver data (seconds)
xrcva= ..... defines receiver array x-positions
zrcva= ..... defines receiver array z-positions
rrcv= ..... radius for receivers on a circle
arcv= ..... vertical arc-length for receivers on a ellipse (rrcv=horizontal)
oxrcv=0.0 ..... x-center position of circle
ozrcv=0.0 ..... z-center position of circle
dphi=2 ..... angle between receivers on circle
rcv_txt=..... text file with receiver coordinates. Col 1: x, Col. 2: z
rec_ntsam=nt ..... maximum number of time samples in file_rcv files
rec_delay=0 ..... time in seconds to start recording: recorded time = tmod - rec_delay
rec_type_p=1 ..... p registration _rp
rec_type_vz=1 ..... Vz registration _rvz
rec_type_vx=0 ..... Vx registration _rvx
rec_type_txx=0 .... Txx registration _rtxx
rec_type_tzz=0 .... Tzz registration _rtzz
rec_type_txz=0 .... Txz registration _rtxz
rec_type_dzvz=0 ... dzVz registration _rdzvz
rec_type_dxvx=0 ... dxVx registration _rdxvx
rec_type_pp=0 ..... P (divergence) registration _rP
rec_type_ss=0 ..... S (curl) registration _rS
rec_type_ud=0 ..... 1:pressure normalized decomposition in up and downgoing waves _ru, _rd
..... 2:particle velocity normalized decomposition in up and downgoing waves _ru, _rd
..... 3:flux normalized decomposition in up and downgoing waves _flup, _flip
kangle= ..... maximum wavenumber angle for decomposition
rec_int_vx=0 ..... interpolation of Vx receivers
                        - 0=Vx->Vx (no interpolation)
                        - 1=Vx->Vz
                        - 2=Vx->Txx/Tzz(P)
                        - 3=Vx->receiver position
rec_int_vz=0 ..... interpolation of Vz receivers
                        - 0=Vz->Vz (no interpolation)
                        - 1=Vz->Vx
                        - 2=Vz->Txx/Tzz(P)
                        - 3=Vz->receiver position
rec_int_p=0 ..... interpolation of P/Tzz receivers
                        - 0=P->P (no interpolation)
                        - 1=P->Vz
                        - 2=P->Vx
                        - 3=P->receiver position

```

NOTES: For viscoelastic media dispersion and stability are not always guaranteed by the calculated criteria, especially for Q values smaller than 13

Jan Thorbecke 2011
TU Delft
E-mail: janth@xs4all.nl
2015 Contributions from Max Holicki

If you are not considering doing special things, the default values are most of the times sufficient and only a few parameters have to be changed from their default values. For all types of FD modeling experiments, the medium parameters must be given. The medium parameters describe the discretized medium through which the modelling is carried out. The source wavelet must also be given. Besides that no other parameters are needed and the program will start modelling with a source positioned at the top middle of the model (z), with receivers placed at the top with a distance equal to the grid distance. This minimum parameter set is:

```
fdelmodc file_cp=filecp.su file_cs=filecs.su file_den=filero.su \
file_src=wavelet.su
```

In the next subsections all the parameters will be described in more detail and guidelines will be given how to use them.

6.1 Modelling parameters

The `ischeme=` selects the kind of finite-difference scheme to be used. Currently there are four options:

1. acoustic, see section 2
2. visco-acoustic, see section 3
3. elastic, see section 4
4. visco-elastic, see section 5

For visco-acoustic (elastic) media extra options are: `Qp=` (and `Qs=`) for selecting an overall Q factor for all layers. This Q value is defined for a frequency at `fw=`, other frequencies will have slightly different Q values. It is also possible to define a Q value for every grid point in the medium. These arrays must be stored in SU files, have the same dimensions as the files of the gridded medium parameters. The program parameters `file_qp=` (and `file_qs=` for elastic media) will read these Q defined values. You can use the program `makemod` (see section D) to define a model and use one of medium parameters as Q value. For example you can use the `cs` layer parameter to define a different Q -factor in each layer. You can make your own gridded Q definition in for example Matlab; make sure that this gridded file has same size as the `file_cp/file_ro` grids.

6.2 Medium parameters

The parameters `file_cp`, `file_cs`, `file_den` represent the filenames of the gridded model files in SU format. The fastest dimension (n_1 , number of samples per trace, z) in the file represents depth and the second dimension (n_2 , number of traces, x) represents the lateral position. The distance between the grid points has to be the same in the z and x position. The grid distance is read from the headers `d1,d2` in the model files. The origin of the model is read from the headers `f1,f2`, where `f1` is the depth of the first sample and `f2` is the lateral position of the first trace. If those headers are not set then the user can define the sampling distance by using the parameters `dx= dz=`. The output files (receivers and snapshots) will also contain the lateral coordinates in the `gx` headers. The gridded model files can be generated by the, also provided, program `makemod` in the `utils` directory.

Together with the minimum and maximum velocities in the model files, the spatial- and time-sampling the stability of the solution can be calculated (see section 1.2).

The units of the velocity in the medium is $[m/s]$ and for the density $[kg/m^3]$. The unit for the length direction can also be feet (or any other length measurement), as long as the same length unit is used on the distance and the velocity of the medium.

If you want to make a region where no waves propagate; define the (P and or S) velocity to zero, but not the density. Otherwise `fdelmodc` will give an error message:

Warning in fdelmodc: Zero density for trace=0 sample=10;
 Error in fdelmodc: ERROR zero density is not a valid value, program exit

In the algorithms the reciprocal value of the density is used. To avoid checking zero densities for each loop (and therefore make the code perform slower) zero densities are not allowed.

6.3 Boundaries

There are four boundary types used in the FD schemes. The boundary type is selected with the parameters `left= right= top= bottom=` and are identified with a number. The default values of these parameters are; free surface for the top (1) and tapered (4) for the other three boundaries. The different boundary types are:

- 'Absorbing' tapered boundaries (4)

One of the most important boundary type is the absorbing boundary. This type of boundary is used to avoid reflections from the sides of the model. The boundaries in a numerical model are in most cases not physical boundaries, but artificial boundaries introduced to limit the size of the model. Reflections coming from these boundaries are artificial and must therefore be suppressed. There are many possible implementations to absorb these artificial reflections. In the program, the most simple absorbing boundary condition is implemented: a taper on the V_x and V_z fields. It is difficult to give guidelines how many grid points the taper length should be to suppress the side reflections. The wave field is gradually tapered over a specified range of grid points (window length `ntaper`). The default setting for the taper length is: $4 * ((cp_{max}/f_{max})/dx)$, 4 wavelengths. Depending on the size of the grid a window length of 40,80 grid points might be sufficient. You may alter these, if you like, in order to increase or decrease the amount of tapering. The larger the window length, the better the absorption, but the longer the modeling will take.

Using the parameters and `ntaper=n` enables the tapered boundaries with a taper length of n points. Besides those parameters specific boundaries must be put 'on' for tapering by using `left=4 right=4 top=4 bottom=4`. All enabled boundaries are using the same taper length and it is not possible to use different taper lengths for different boundaries. The number of taper points should be chosen such 1-2 times the main wavelength in the modeled data. The program calculates the number of taper points to be five times the wavelength $5\lambda_{tap} = 5 \frac{max(c_p)}{f_{max}}$;

$$taper[ix] = \exp - (0.30 * \frac{ix}{ntaper})^2, \quad (50)$$

where ix is an integer ranging from 0 to $ntaper - 1$ and 0.30 is the taper factor. This taper factor can be changed with the parameter `tapfact=`. A larger taper factor will make the taper go steeper to 0.0. In Figure 7 different `tapfact=` choices are shown. Note that if the taper is chosen too steep (larger than 0.5) the wavelet will already start reflecting from the beginning of the tapered boundary.

Figure 6 shows the effects of the taper in a homogenous medium. It effectively suppresses the reflections from the sides of the model. The chosen taper length in this example is 200 points long. This is a large number of points and used for illustration purposes. Using a taper is not a very efficient way of suppressing side reflections and there are plans to implement a better absorbing boundary method such as the Perfectly Matched Layer (PML) approach.

- Free surface (1)

The free surface implementation for the acoustic scheme is straightforward and just sets the pressure field to zero on the free surface. For the elastic scheme the implementation is more involved and the free surface conditions (for the upper boundary) are :

1. $\sigma_{zz} = 0$ is set
2. $\sigma_{xz} = 0$ is implemented by defining an odd symmetry $\sigma_{xz}[iz] = -\sigma_{xz}[iz + 1]$.
3. σ_{xx} : removed term with $\frac{\partial V_z}{\partial z}$, and add extra term with $\frac{\partial V_x}{\partial x}$, corresponding to free-surface condition for σ_{xx} . Other boundaries (left, right and bottom) are treated in a similar way.

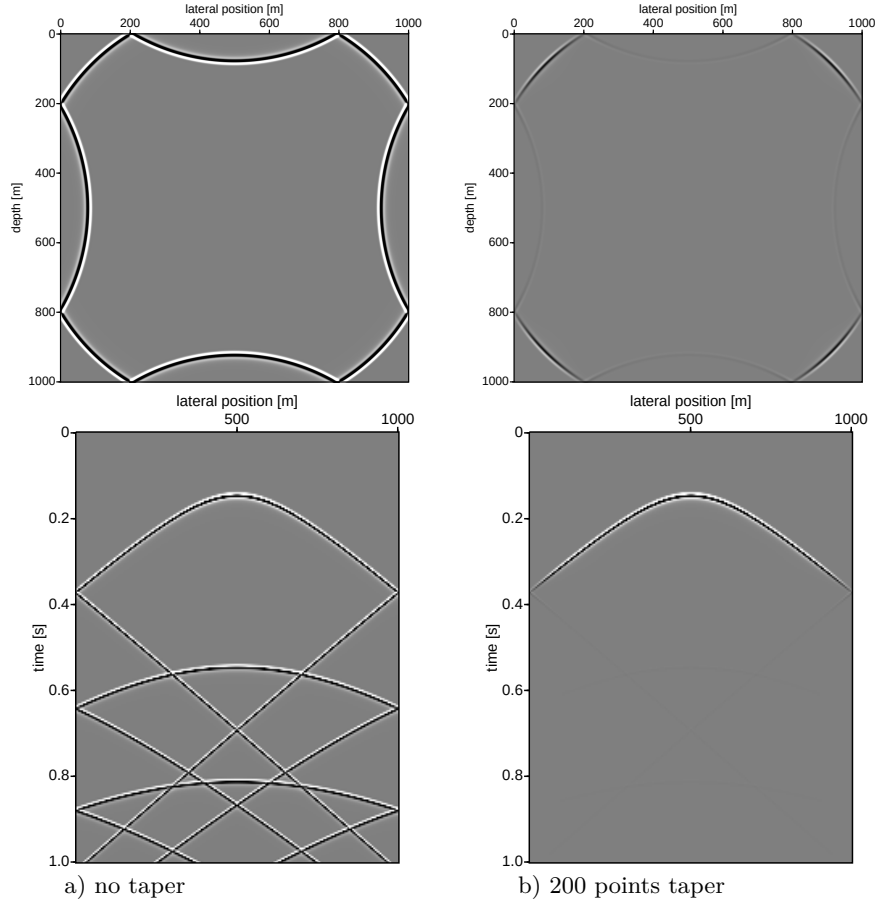


Figure 6: Snapshots and receiver recording in homogeneous medium with and without taper. The receivers are placed at 300 m depth and the source is positioned in the middle of the model at (500,500). The grid distance is 1 meter. The script `fdelmodc_taper.scr` in the demo directory reproduces the pictures.

The linearized equation of motion (Newton's second law) and equation of deformation (Hook's law) for the free surface become:

$$\sigma_{zz} = 0 = \frac{1}{\kappa} \frac{\partial V_z}{\partial z} + \lambda \frac{\partial V_x}{\partial x} \quad (51)$$

$$\sigma_{xz} = 0 = \mu \left\{ \frac{\partial V_x}{\partial z} + \frac{\partial V_z}{\partial x} \right\} \quad (52)$$

In the FD code σ_{zz} is set to 0 at the free surface position $z = 0$. σ_{xz} is constructed in such a way that the difference around the free surface ends up to be zero:

$$\begin{aligned} \sigma_{xz}(0 - \tfrac{1}{2}\Delta z) &= -\sigma_{xz}(0 + \tfrac{1}{2}\Delta z) \\ \sigma_{xz}(0 - 1\tfrac{1}{2}\Delta z) &= -\sigma_{xz}(0 + 1\tfrac{1}{2}\Delta z) \end{aligned}$$

Note that the location in the equations above are with respect to the grid for σ_{zz} . The trick for σ_{xz} is only needed for staggered grids to make the free surface of σ_{xz} on the same level as σ_{zz} .

For an expression of σ_{xx} on the free surface:

$$\sigma_{xx} = \frac{1}{\kappa} \frac{\partial V_x}{\partial x} + \lambda \frac{\partial V_z}{\partial z} \quad (53)$$

we substitute equation (51) $\frac{\partial V_z}{\partial z} = -\lambda\kappa \frac{\partial V_x}{\partial x}$ into and gives:

$$\sigma_{xx} = \frac{1}{\kappa} \frac{\partial V_x}{\partial x} - \lambda^2 \kappa \frac{\partial V_x}{\partial x} \quad (54)$$

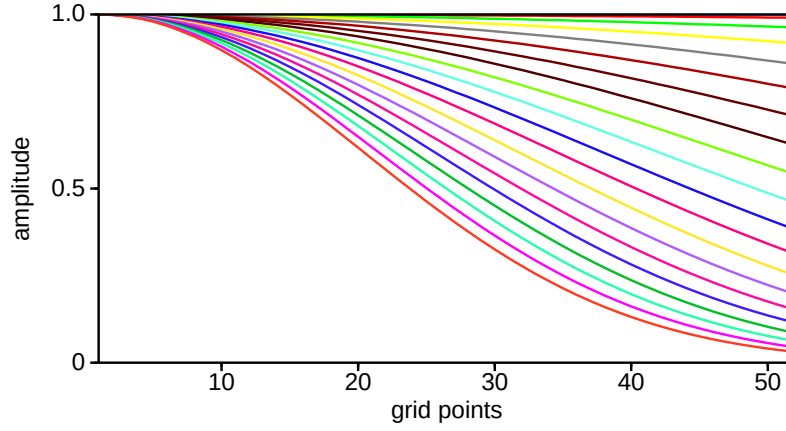


Figure 7: The boundary taper as function of the *tapfact*= parameter is shown. The red line with the highest amplitude has *tapfact*=0.1, each line with a lower amplitude has a tap fact 0.1 larger (e.g. the green line has 0.2, the yellow line 0.3).

Using the parameters *left*=1 *right*=1 *top*=1 *bottom*=1 enables a free surface boundary for all 4 sides.

```

if (bnd.top==1) { /* free surface at top */
    izp = bnd.surface[ixo];
    for (ix=ixo; ix<ixe; ix++) {
        iz = bnd.surface[ix-1];
        if ( izp==iz ) {
            /* clear normal pressure */
            tzz[ix*n1+iz] = 0.0;
        }
        izp=iz;
    }

    izp = bnd.surface[ixo];
    for (ix=ixo+1; ix<ixe+1; ix++) {
        iz = bnd.surface[ix-1];
        if ( izp==iz ) {
            /* assure that txz=0 on boundary by filling virtual boundary */
            txz[ix*n1+iz] = -txz[ix*n1+iz+1];
            /* extra line of txz has to be copied */
            txz[ix*n1+iz-1] = -txz[ix*n1+iz+2];
        }
        izp=iz;
    }

    /* calculate txx on top stress-free boundary */
    izp = bnd.surface[ixo];
    for (ix=ixo; ix<ixe; ix++) {
        iz = bnd.surface[ix-1];
        if ( izp==iz ) {
            dp = l2m[ix*n1+iz]-lam[ix*n1+iz]*lam[ix*n1+iz]/l2m[ix*n1+iz];
            dvx = c1*(vx[(ix+1)*n1+iz] - vx[(ix)*n1+iz]) +
                  c2*(vx[(ix+2)*n1+iz] - vx[(ix-1)*n1+iz]);
            txx[ix*n1+iz] = -dvx*dp;
        }
        izp=iz;
    }
}

```


}

Placing a pressure source exactly on the free surface will not eject any energy into the medium and the resulting wavefield will contain only zero's. To overcome that you can use a F_z source (`src_type=7`) or place the pressure source one grid-point below the free-surface.

Note that you will always get a reflection from the free-surface. To summarize the effects:

- a P-source on a free surface can not put energy into the medium, and gives gathers with all zero s
- a F_z source (`src_type=7`) on the free-surface can put energy into the medium and is a good alternative for a P-source
- placing a P-source one-grid point below the surface, will simulate a dipole source. The – part of the dipole coming from the reflection from the free surface.
- placing receivers, just one grid-point below the free surface, also gives a dipole receiver response.
- V_z receivers on a free surface measure an wavefield, P-receivers will not measure anything on a free surface.

To correct for the ghost of the source it is possible to de-ghost the measured response. This can be done with the program `basop option=ghost`. There is also a lot of literature about "source de-ghosting" (google search). The basop implementation is the most simple one.

- Rigid surface (3)
The rigid boundary condition sets the velocities on the boundaries to zero. For the top surface these conditions are met by setting:

- $V_x[iz] = 0.0$
- $V_z[iz] = -V_z[iz + 1]$

Setting the boundary parameters `left= right= top= bottom=` to 3 enables a rigid surface boundary for the selected boundary.

- PML (2)
Only implemented for acoustic.
- Topography
On the top of the model an irregular topography can be used. The density and velocity model must have zero-values above the defined topography. To place a source or receiver on the topography it is sufficient to place it at the correct lateral position above the topography. In the code the depth is searched for the first non-zero medium parameter and at that depth the source or receiver is placed. The parameter `sinkdepth=n` places the receiver position `n` grid-points below the found depth point on the topography. For the source position the parameter `sinkdepth_src=n` places the source position `n` grid-points below the found depth point on the topography. When the parameter `sinkvel=1` is used the receiver (not the source position) can also sink through a layer with a non-zero velocity. The velocity of the first receiver is used as the velocity to sink through to the next layer.

For the elastic scheme the topography is implemented as described in Robertsson (1996) and Pérez-Ruiz et al. (2005). In those schemes the points at the topography layer are treated differently (depending on which side of the topography the free-surface is), such that the free surface is taken into account in the best possible way.

6.3.1 Recursive Integration PML: acoustic

The linearized equation of motion (Newton's second law) and equation of deformation (Hook's law) are given by:

$$\frac{\partial V_x}{\partial t} = -\frac{1}{\rho} \frac{\partial P}{\partial x}, \quad (55)$$

$$\frac{\partial V_z}{\partial t} = -\frac{1}{\rho} \frac{\partial P}{\partial z}, \quad (56)$$

$$\frac{\partial P}{\partial t} = -\frac{1}{\kappa} \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\}. \quad (57)$$

On PML introduces stretched-coordinate space (in the frequency domain) following Drossaert and Giannopoulos (2007) and Chew and Liu (1996):

$$\epsilon_x = 1 + \frac{\sigma_x}{j\omega} \quad (58)$$

$$\epsilon_z = 1 + \frac{\sigma_z}{j\omega} \quad (59)$$

$$\frac{\partial V_x}{\partial t} = -\frac{1}{\rho} \frac{1}{\epsilon_x} \frac{\partial P}{\partial x}, \quad (60)$$

$$\frac{\partial V_z}{\partial t} = -\frac{1}{\rho} \frac{1}{\epsilon_z} \frac{\partial P}{\partial z}, \quad (61)$$

$$\frac{\partial P}{\partial t} = -\frac{1}{\kappa} \left\{ \frac{1}{\epsilon_x} \frac{\partial V_x}{\partial x} + \frac{1}{\epsilon_z} \frac{\partial V_z}{\partial z} \right\}, \quad (62)$$

The manner with which coordinate stretching causes losses in a medium can be easily understood as follows: Given a plane wave propagating in a lossless medium described by $\exp(jkx)$, a coordinate stretching is a change $x = x'(a + j\alpha)$. Then $\exp(jkx)$ becomes $\exp(jkax' - k\alpha x') = \exp(jkax') \exp(-k\alpha x')$. In other words, a wave becomes attenuative in the stretched x' coordinate if the stretching variable is complex (Chew and Liu, 1996).

Introducing auxiliary variables

$$S_x = \frac{1}{\epsilon_x} \frac{\partial P}{\partial x} \rightarrow \frac{\partial V_x}{\partial t} = -\frac{1}{\rho} S_x, \quad (63)$$

$$S_z = \frac{1}{\epsilon_z} \frac{\partial P}{\partial z} \rightarrow \frac{\partial V_z}{\partial t} = -\frac{1}{\rho} S_z, \quad (64)$$

$$E_x = \frac{1}{\epsilon_x} \frac{\partial V_x}{\partial x} \quad E_z = \frac{1}{\epsilon_z} \frac{\partial V_z}{\partial z} \rightarrow \frac{\partial P}{\partial t} = -\frac{1}{\kappa} \{E_x + E_z\} \quad (65)$$

Substituting the stretch variables ϵ_x and ϵ_z into these auxiliary variables gives:

$$S_x + \frac{\sigma_x}{j\omega} S_x = \frac{\partial P}{\partial x}, \quad (66)$$

$$S_z + \frac{\sigma_z}{j\omega} S_z = \frac{\partial P}{\partial z}, \quad (67)$$

$$E_x + \frac{\sigma_x}{j\omega} E_x = \frac{\partial V_x}{\partial x}, \quad (68)$$

$$E_z + \frac{\sigma_z}{j\omega} E_z = \frac{\partial V_z}{\partial z} \quad (69)$$

To compute S_x , the other components can be done completely analogous, the following steps are carried out. The integration over time of S_x is approximated by using trapezoidal integration.

$$\frac{\sigma_x}{j\omega} S_x = \frac{\partial P}{\partial x} - S_x \quad (70)$$

$$\int_0^t \sigma_x S_x dt = \frac{\partial P}{\partial x} - S_x \quad (71)$$

$$\int_0^{n\Delta t} \sigma_x S_x dt \approx \frac{1}{2} \Delta t \sigma_x S_x^0 + \Delta t \sum_{t=1}^{n-1} \sigma_x S_x^t + \frac{1}{2} \Delta t \sigma_x S_x^n \quad (72)$$

$$(73)$$

assuming $S_x^0 = 0$ results at time step n

$$S_x^n (1 + \frac{1}{2} \Delta t \sigma_x) + \Delta t \sum_{t=1}^{n-1} \sigma_x S_x^t = \frac{\partial P^n}{\partial x} \quad (74)$$

$$S_x^n = RA \left(\frac{\partial P^n}{\partial x} - \Delta t \Omega_x^{n-1} \right) \quad (75)$$

$$RA = \frac{1}{1 + \frac{1}{2} \Delta t \sigma_x} \quad (76)$$

$$\Omega_x^n = \Omega_x^{n-1} + \sigma_x S_x^n \quad (77)$$

The first few time steps in the PML regions are then computed as follows:

$$n = 0 : S_x^0 = RA \left(\frac{\partial P^0}{\partial x} \right) \quad (78)$$

$$\Omega_x^0 = \sigma_x S_x^0 \quad (79)$$

$$n = 1 : S_x^1 = RA \left(\frac{\partial P^1}{\partial x} - \Delta t \Omega_x^0 \right) \quad (80)$$

$$\Omega_x^1 = \Omega_x^0 + \sigma_x S_x^1 \quad (81)$$

$$n = 2 : S_x^2 = RA \left(\frac{\partial P^2}{\partial x} - \Delta t \Omega_x^1 \right) \quad (82)$$

$$\Omega_x^2 = \Omega_x^1 + \sigma_x S_x^2 \quad (83)$$

With these steps the auxiliary variables can now be computed and substituted into equations 63 for the PML regions.

6.3.2 Complex frequency shifted RIPML: acoustic

(This PML is not yet implemented.) Stretched-coordinate space (in the frequency domain) following Drossaert and Giannopoulos (2007):

$$\epsilon_x = \kappa_x + \frac{\sigma_x}{\alpha_x + j\omega} \quad (84)$$

$$\epsilon_z = \kappa_z + \frac{\sigma_z}{\alpha_z + j\omega} \quad (85)$$

$$(86)$$

Following the same steps as before: Substituting these stretch variables ϵ_x and ϵ_z into the auxiliary variables 63 gives:

$$(\kappa_x \alpha_x + \kappa_x j\omega + \sigma_x) S_x = (\alpha_x + j\omega) \frac{\partial P}{\partial x}, \quad (87)$$

$$(\kappa_z \alpha_z + \kappa_z j\omega + \sigma_z) S_z = (\alpha_z + j\omega) \frac{\partial P}{\partial z}, \quad (88)$$

$$(\kappa_x \alpha_x + \kappa_x j\omega + \sigma_x) E_x = (\alpha_x + j\omega) \frac{\partial V_x}{\partial x}, \quad (89)$$

$$(\kappa_z \alpha_z + \kappa_z j\omega + \sigma_z) E_z = (\alpha_z + j\omega) \frac{\partial V_z}{\partial z} \quad (90)$$

To compute S_x , the other components can be done completely analogous, the following steps are carried out: divide by $j\omega$ results in

$$(\kappa_x \alpha_x + \kappa_x j\omega + \sigma_x) S_x = (\alpha_x + j\omega) \frac{\partial P}{\partial x}, \quad (91)$$

$$\frac{\kappa_x \alpha_x + \sigma_x}{j\omega} S_x + \kappa_x S_x = \frac{\alpha_x}{j\omega} \frac{\partial P}{\partial x} + \frac{\partial P}{\partial x}, \quad (92)$$

$$\int_0^t (\kappa_x \alpha_x + \sigma_x) S_x dt + \kappa_x S_x = \int_0^t \alpha_x \frac{\partial P}{\partial x} dt + \frac{\partial P}{\partial x} \quad (93)$$

$$(94)$$

The integration over time of S_x and $\frac{\partial P}{\partial x}$ is approximated by using trapezoidal integration.

$$\int_0^{n\Delta t} a S_x dt \approx \frac{1}{2} \Delta t a S_x^0 + \Delta t \sum_{t=1}^{n-1} a S_x^t + \frac{1}{2} \Delta t a S_x^n \quad (95)$$

assuming $S_x^0 = 0$ and $\frac{\partial P^0}{\partial x} = 0$ results at time step n in

$$\Delta t \sum_{t=1}^{n-1} (\kappa_x \alpha_x + \sigma_x) S_x^t + \frac{1}{2} \Delta t (\kappa_x \alpha_x + \sigma_x) S_x^n + \kappa_x S_x^n = \Delta t \sum_{t=1}^{n-1} \alpha_x \frac{\partial P^t}{\partial x} + \frac{1}{2} \Delta t \alpha_x \frac{\partial P^n}{\partial x} + \frac{\partial P^n}{\partial x} \quad (96)$$

$$\Delta t \sum_{t=1}^{n-1} (\kappa_x \alpha_x + \sigma_x) S_x^t + \left(\frac{1}{2} \Delta t (\kappa_x \alpha_x + \sigma_x) + \kappa_x \right) S_x^n = \Delta t \sum_{t=1}^{n-1} \alpha_x \frac{\partial P^t}{\partial x} + \left(\frac{1}{2} \Delta t \alpha_x + 1 \right) \frac{\partial P^n}{\partial x} \quad (97)$$

$$S_x^n = RA \frac{\partial P^n}{\partial x} - RB \Omega_x^{n-1} \quad (98)$$

$$RA = \frac{(\frac{1}{2} \Delta t \alpha_x + 1)}{\frac{1}{2} \Delta t (\kappa_x \alpha_x + \sigma_x) + \kappa_x} \quad (99)$$

$$RB = \frac{\Delta t}{\frac{1}{2} \Delta t (\kappa_x \alpha_x + \sigma_x) + \kappa_x} \quad (100)$$

$$\Omega_x^n = \Omega_x^{n-1} + (\kappa_x \alpha_x + \sigma_x) S_x^n - \alpha_x \frac{\partial P^n}{\partial x} \quad (101)$$

The first few time steps in the complex frequency shifted PML regions are then computed as follows:

$$n = 0 : S_x^0 = RA \frac{\partial P^0}{\partial x} \quad (102)$$

$$\Omega_x^0 = (\kappa_x \alpha_x + \sigma_x) S_x^0 - \alpha_x \frac{\partial P^0}{\partial x} \quad (103)$$

$$n = 1 : S_x^1 = RA \frac{\partial P^1}{\partial x} - RB \Omega_x^0 \quad (104)$$

$$\Omega_x^1 = \Omega_x^0 + (\kappa_x \alpha_x + \sigma_x) S_x^1 - \alpha_x \frac{\partial P^1}{\partial x} \quad (105)$$

$$n = 2 : S_x^2 = RA \frac{\partial P^2}{\partial x} - RB \Omega_x^1 \quad (106)$$

$$\Omega_x^2 = \Omega_x^1 + (\kappa_x \alpha_x + \sigma_x) S_x^2 - \alpha_x \frac{\partial P^2}{\partial x} \quad (107)$$

6.4 Source signature parameters

The parameter `file_src=` is used to define the filename of a SU file containing the source signature. This source signature should not have a large value at time $t = 0$, since this will represent a spike at $t = 0$,

introduce high frequencies, and can make the modelling dispersive. If `file_src=` is not given the source sampling can be defined by setting `dt=`. To avoid numerical dispersion the maximum frequency content of the source wavelet must be limited (see section 1.2). The program tries to estimate the maximum amplitude in the frequency domain of the source signal, and based on this maximum determines if the modelling will be stable. The maximum frequency can also be given by the user with `fmax=` and will overrule the estimated maximum frequency. This overruling of the maximum frequency can be useful when the amplitude of the source signal is complex and the program can not make a good estimate of the maximum frequency. The parameter `fmax=` can also be used to overrule the programs error message (which stops the program) for dispersive modeling. By setting `fmax=` lower than the actual maximum frequency in the wavelet (estimated by the program) dispersion can be introduced. In some cases dispersion can be allowed, or dispersion is not relevant for the purpose of the modelling.

For example; a wavelet is created by :

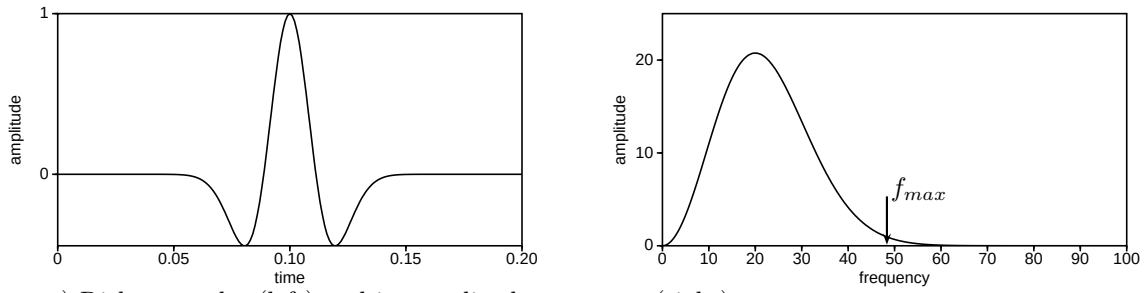
```
makewave w=g2 fmax=45 t0=0.10 dt=0.001 nt=4096 db=-40 file_out=G2.su
verbose=1
```

This wavelet has a spectrum and time signal shown in Figure 8a and the maximum frequency estimated by the program is 48.8 Hz. To calculate the maximum frequency the program searches the peak frequency in the amplitude spectrum of the wavelet and then looks for the first frequency, larger than the peak frequency, where the amplitude is smaller than 0.0025 times the peak amplitude.

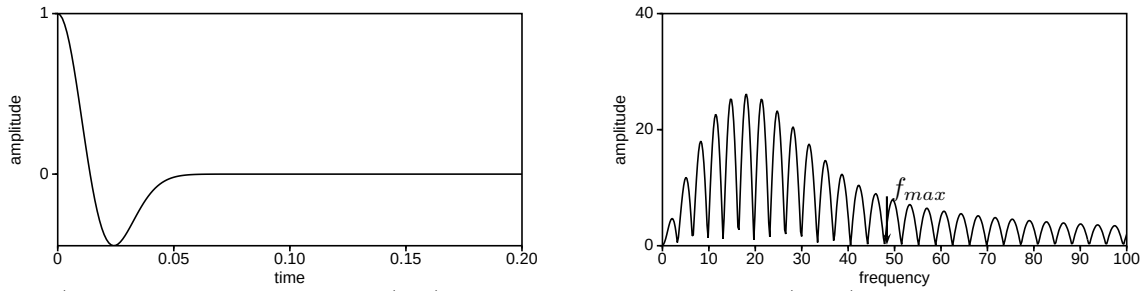
Note that the peak in the time domain of the wavelet in Figure 8a is time shifted with 0.1 seconds. Starting from $t = 0$ the amplitude of the wavelet is increased smoothly to its peak value and in this way avoids high frequencies. A spike at $t=0$ (or any other time), or a truncated wavelet as shown in Figure 8b introduces high frequencies and will cause dispersion in the solution. So it is always recommended to check the wavelet on spikes and truncation before the modelling is started.

If the modelling is finished, and one has for example modeled a shot record with reflections, and would like to pick travel times on the peak of the wavelet, the 0.1 s, time shift of the wavelet should be taken into account! Good practice after modelling is to shift the peak of the wavelet back to $t = 0$ by using the parameter `rec_delay=0.1`. After the modeling one can also use other program, like `basop choice=shift shift=-0.1`, to shift the peak back to $t = 0$. When `rec_delay=` is set to a positive non-zero value the modelling time recorded in the receiver array will be the total modeling time decreased by the time of the `rec_delay`.

Noise signals are created (`wav_random=1`) by setting random values to the amplitude and phase of the source signal up to the given maximum frequency (`fmax=`). This signal is transformed back to the time domain and truncated in time to the desired source duration. Figure 9 shows 20 random signals in the time domain with varying source duration (average duration of 2.5 s `tlength=2.5`). Without any tapering this truncation in the time domain will introduce high frequencies. To suppress these high frequencies, the beginning and the end of the source signal are smoothly extrapolated (using cubic splines) to an amplitude value of 0.0. The bottom pictures in Figure 10 show a noise signal and its amplitude spectrum. This signal was constructed with a maximum frequency of 30 Hz. The start and beginning of the noise signal are smoothly starting and ending at amplitude zero. The red circles and lines shows how this signal is constructed. Despite the smooth start and ending of the signal the spectrum of the noise signal does continue after 30 Hz, but the amplitude after 30 Hz is so low that it does not give rise to severe dispersion in the modelling. The calculated noise source signatures are written to an SU file if the parameter `verbose>3` (see also Table 1). Note that if `file_src` is defined then `wav_random=0` is default set off. However, if `src_random=1` is used `wav_random=1` is default set on. The length (duration) of the random signal is chosen to be a random number (between 0.0 and 1.0 multiplied) by `tlength`. By setting `length_random=0` all random signals will have the same length given by `tlength`.



a) Ricker wavelet (left) and its amplitude spectrum (right).



b) Truncated Ricker wavelet (left) and its amplitude spectrum (right).

Figure 8: *Wavelet and its amplitude spectrum. The maximum frequency in the wavelet is found by searching from the maximum amplitude to the first frequency amplitude $\leq 0.0025 * A_{max}$ (indicated with an arrow). Note that due to the truncation at $t = 0$ in b) high frequencies are introduced and can cause dispersion. The script `fdelmodc_rand.scr` in the demo directory calculates the data and `eps_for_manual.scr` reproduces the pictures.*

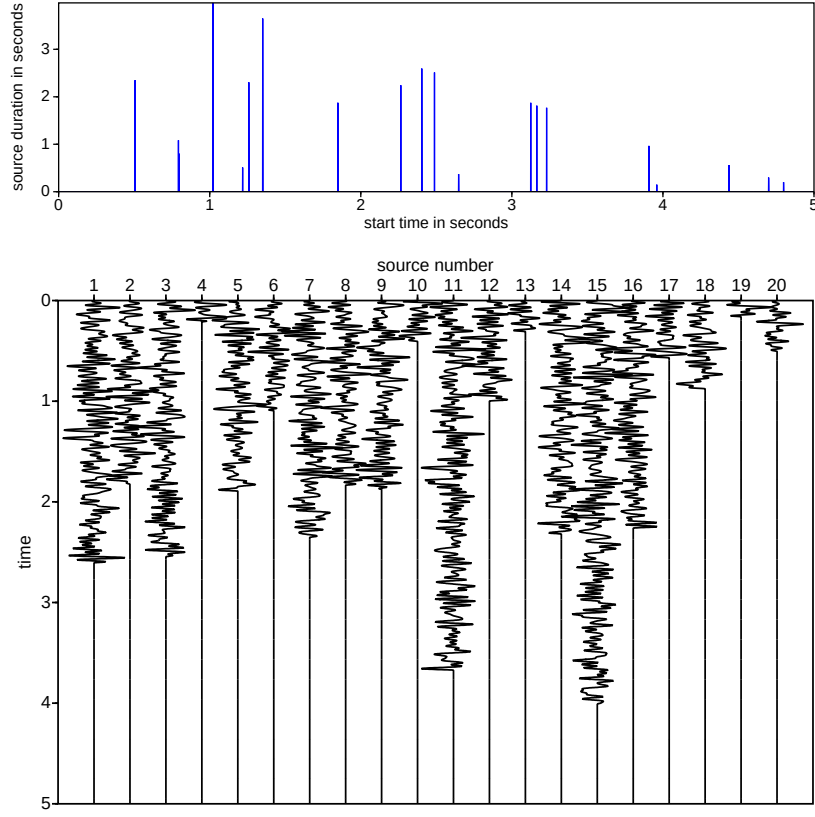


Figure 9: *Random source signatures with varying source duration (top picture). Note that the sources start at random times in the interval $t_{src1} = : t_{src2}$. The script `fdelmodc_rand.scr` in the demo directory calculates the data and `eps_for_manual.scr` reproduces the pictures.*

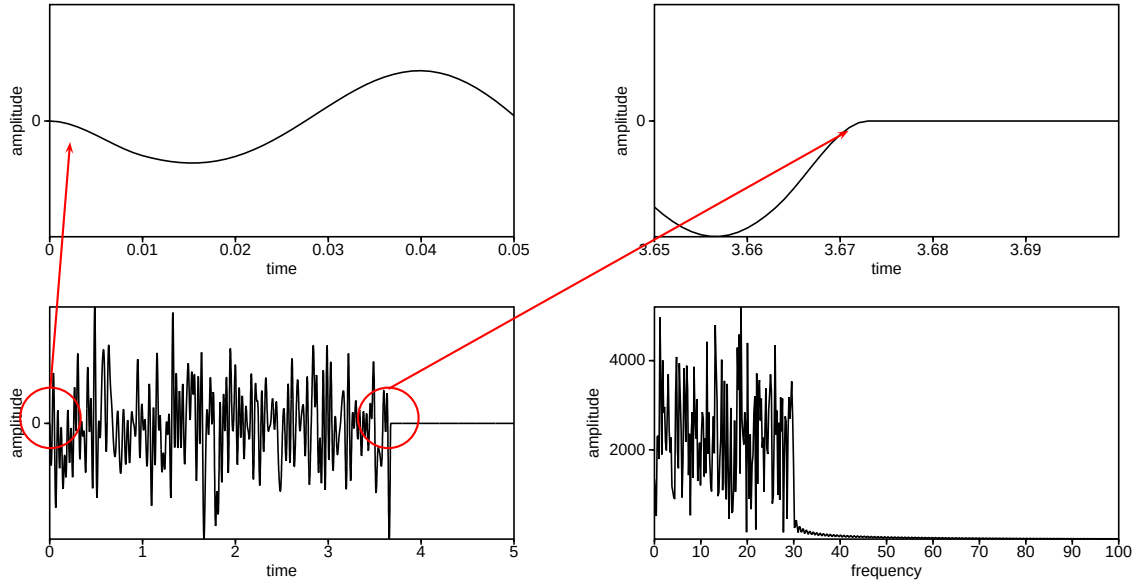


Figure 10: *Random source signature and its amplitude spectrum. The start and beginning of the source signature are smoothly (cubic spline) starting from and ending at amplitude 0. Despite this smooth transition the frequency spectrum of the signature still contains some energy after the defined maximum frequency. The script `fdelmodc_rand.scr` in the demo directory calculates the data and `eps_for_manual.scr` reproduces the pictures.*

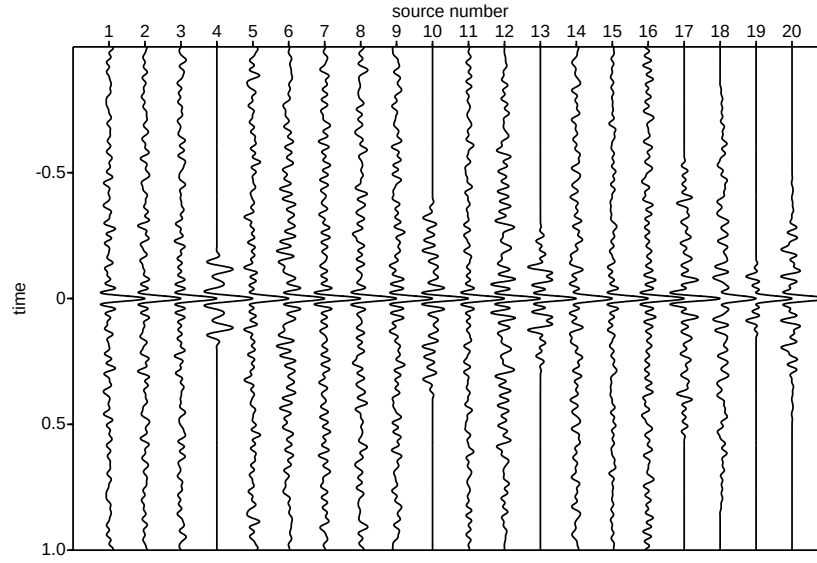


Figure 11: *Auto-correlated random source signatures with varying source duration normalized to the maximum amplitude per trace. The longest signals (source numbers 11 and 15) have the highest auto-correlation peak and best S/N ratio. The script **Figure17_19AppendixA.scr** in the *FiguresPaper* directory reproduces the pictures.*

The autocorrelation of the source signal gives an indication of the contribution of the individual sources to a Seismic Interferometry result. In Figure 11 the normalized auto-correlation of the signals in Figure 9 is shown. The longest signals will give a contribution, and has the highest signal (peak at $t = 0$) to noise ratio. The longer the source is active the more energy it will bring into the medium and the better the S/N ratio will be.

The cross-correlation of the source signals with each other gives how the different source signatures interfere with each other. Ideally the signatures should not interfere. Figure 12 show 100 source signatures and the cross-correlation. The diagonal is the auto correlation, and dominates the cross-correlation picture, indicating that the source are not correlated to each other.

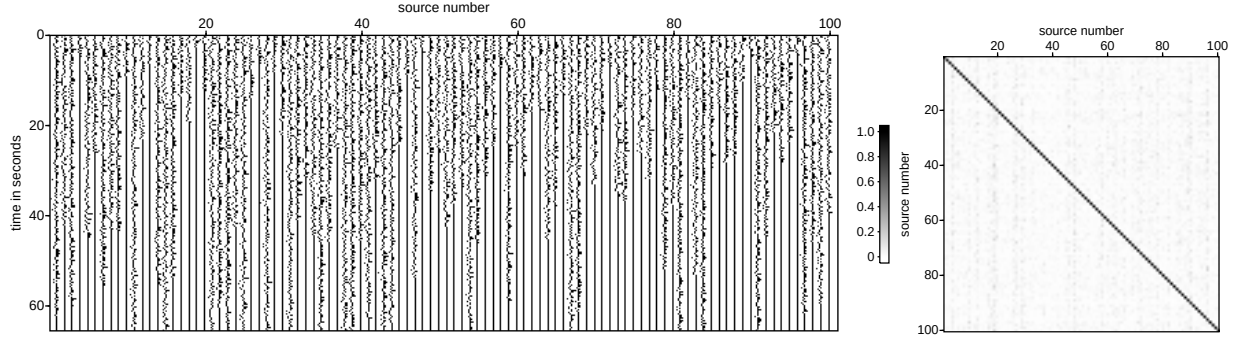


Figure 12: The 100 source signatures on the left side have been cross-correlated with each other and the result is shown on the right side. The script `cross.scr` in the `FiguresPaper` directory reproduces these pictures.

6.5 Source type and position parameters

The source amplitudes are added directly on the grid at the source position(s). The FD scheme solves the first order equations in(1) and the source amplitudes are added into these first order equations on the P , V_x or V_z grid. For example for a pressure source in an acoustic scheme (in a homogenous medium, to make the equations easier to read) the amplitudes are added at the P field only:

$$\frac{\partial P(x, z, t)}{\partial t} = -\frac{1}{\kappa} \left\{ \frac{\partial V_x(x, z, t)}{\partial x} + \frac{\partial V_z(x, z, t)}{\partial z} \right\} + \frac{1}{\kappa} \delta(x - x', z - z') S(t), \quad (108)$$

$$\frac{\partial V_x(x, z, t)}{\partial t} = -\frac{1}{\rho} \frac{\partial P(x, z, t)}{\partial x}, \quad (109)$$

$$\frac{\partial V_z(x, z, t)}{\partial t} = -\frac{1}{\rho} \frac{\partial P(x, z, t)}{\partial z}, \quad (110)$$

where $S(t)$ represents the source signature injected at position x' . Substituting Hooke's Law in Newton's law gives the second order wave equation. Adding an amplitude on a grid point (representing a delta pulse $\delta(x - x', z - z')$) in the first order equations, results in a source term of $\partial_t \delta(x - x', z - z')$ in the right-hand side in the (second order) wave equation

$$\frac{\partial^2 P(x, z, t)}{\partial t^2} - c_p^2 \left\{ \frac{\partial^2 P(x, z, t)}{\partial x^2} + \frac{\partial^2 P(x, z, t)}{\partial z^2} \right\} = \delta(x - x', z - z') \frac{\partial S(t)}{\partial t}. \quad (111)$$

To end up with a source injection in the wave equation, and not injection rates as in equation 111, the source signature is adjusted (in the frequency domain by $\frac{1}{-j\omega}$: integrating over time) before it is applied to the grid. The input parameter `src_injectionrate` can be set to choose between injection rate (set to 1) or injection (set to 0, which is the default).

For example when `src_injectionrate=0` a measured P response of a P source will measure the same source signature. The output file `src_nwav.su` (this file is written when `verbose=4`) contains the wavelet as it is being added to the grids in the FD code. Meaning that with `src_injectionrate=0` (default) it will be a time integrated version of the wavelet given by `file_src=`.

TODO. The general source function can be described as Wapenaar 1989 page 13:

$$S(t) = \frac{\partial \delta(t)}{\partial t} - \frac{\partial F_z}{\partial z} \quad (112)$$

Monopole source ($\frac{\partial \delta(t)}{\partial t}$) has same wavelet shape as Force source (dipole $\frac{\partial F_z}{\partial z}$)

6.5.1 Source type

The type of source and orientation is chosen with the parameters **src_type=** and **src_orient=**. The source type options are:

- 1=P for acoustic scheme placed on P grid and for elastic scheme on σ_{zz} and σ_{xx} grid with amplitude s
- 2=Txz for elastic scheme placed on σ_{xz} grid with amplitude s
- 3=Tzz for elastic scheme placed on σ_{zz} grid with amplitude s
- 4=Txx for elastic scheme placed on σ_{xx} grid with amplitude s
- 5=S-potential for elastic scheme placed on V_x and V_z grid with amplitude s (experimental)
- 6=Fx placed on V_x grid with amplitude $\frac{s\Delta x}{\rho}$
- 7=Fz placed on V_z grid with amplitude $\frac{s\Delta z}{\rho}$
- 8=P-potential for elastic scheme placed on V_x and V_z grid with amplitude s (experimental)
- 9=double-couple source, define direction with **dip=0.0**, **strike=90.0** and **rake=90.0**
- 10=Fz
- 11=moment tensor, define with **Mxx=1.0**, **Mzz=1.0** and **Mxz=1.0**

where s represents the amplitude of the source signal (from **file_src=** or a generated random signature). The source orientation can be changed from monopole to dipole, where the orientation of the dipole can also be changed. The implementation of the source orientation options are shown in Figure 13. The source orientation is only implemented for three types of sources: compressional (**src_type=1**), Txz (**src_type=2**), and pure shear source (**src_type=5**). For the other source types the **src_orient=** parameter has no effects and a monopole source is always used.

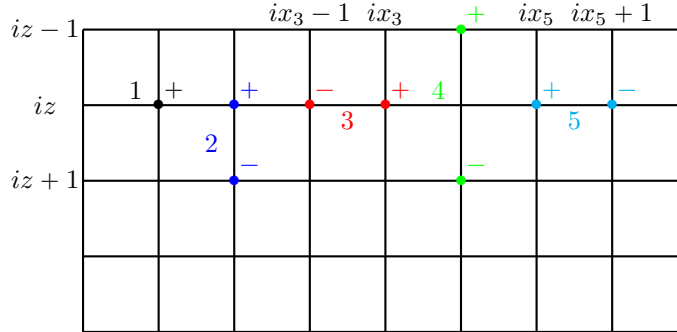


Figure 13: Implementation of different source orientations on the grid. The different numbers (colors) show the options of parameter **src_orient=**. The + and - symbol represent the sign of the source amplitude added to the computational grid.

Note, to check source-receiver reciprocity the dipole/monopole character of the source and receiver has to be taken into account as well. For example a dipole source with a V_z receiver (rvz) will be reciprocal with each other. The same for a monopole source and P receiver (rp). For a monopole source and V_z receiver this is reciprocal with a dipole source and and P receiver. For example run the code with: **src_orient=1**

and select V_z receivers and after changing source and receiver x,z positions, set **src_orient=2** and use P receivers. Those measurements are source-receiver reciprocal with each other.

Pressure source

$$\frac{\partial P(x, z, t)}{\partial t} = -\frac{1}{\kappa} \left\{ \frac{\partial V_x(x, z, t)}{\partial x} + \frac{\partial V_z(x, z, t)}{\partial z} \right\} + \delta(x - x', z - z') \left(\frac{1}{\kappa} S(t) \right), \quad (113)$$

$$\frac{\partial V_x(x, z, t)}{\partial t} = -\frac{1}{\rho} \frac{\partial P(x, z, t)}{\partial x}, \quad (114)$$

$$\frac{\partial V_z(x, z, t)}{\partial t} = -\frac{1}{\rho} \frac{\partial P(x, z, t)}{\partial z}, \quad (115)$$

Force source

$$\frac{\partial P(x, z, t)}{\partial t} = -\frac{1}{\kappa} \frac{\partial V_x(x, z, t)}{\partial x}, \quad (116)$$

$$\frac{\partial V_x(x, z, t)}{\partial t} = -\frac{1}{\rho} \frac{\partial P(x, z, t)}{\partial x} + \delta(x - x', z - z') \left(\frac{1}{\rho} S(t) \right), \quad (117)$$

$$\frac{\partial V_z(x, z, t)}{\partial t} = -\frac{1}{\rho} \frac{\partial P(x, z, t)}{\partial z}, \quad (118)$$

Potential S source (stype=5)

$$S_s(x, z, t) = \frac{\partial V_z(x, z, t)}{\partial x} - \frac{\partial V_x(x, z, t)}{\partial z}, \quad (119)$$

$$(120)$$

```
vx[ix*n1+iz] += src_ampl*sdx;
vx[ix*n1+iz-1] -= src_ampl*sdx;
vz[ix*n1+iz] -= src_ampl*sdx;
vz[(ix-1)*n1+iz] += src_ampl*sdx;
```

Potential P source (stype=8)

$$P_s(x, z, t) = \frac{\partial V_z(x, z, t)}{\partial z} + \frac{\partial V_x(x, z, t)}{\partial x}, \quad (121)$$

$$(122)$$

```
vx[(ix+1)*n1+iz] += src_ampl*sdx;
vx[ix*n1+iz] -= src_ampl*sdx;
vz[ix*n1+iz+1] += src_ampl*sdx;
vz[ix*n1+iz] -= src_ampl*sdx;
```

Double-couple source (stype=9)

The direction of the double-couple source is defined with **dip=0.0**(δ), **strike=90.0**(Φ) and **rake=90.0**(λ) and relates to Moments:

$$M_{xx} = -(\sin(\delta) * \cos(\lambda) * \sin(2\Phi) + \sin(2\delta) * \sin(\lambda) * \sin(\Phi)^2), \quad (123)$$

$$M_{xz} = -(\cos(\delta) * \cos(\lambda) * \cos(\Phi) + \cos(2\delta) * \sin(\lambda) * \sin(\Phi)), \quad (124)$$

$$M_{zz} = \sin(2\delta) * \sin(rake). \quad (125)$$

and is implemented by

```
txx[ix*n1+iz] -= src.Mxx*src_ampl;
tzz[ix*n1+iz] -= src.Mzz*src_ampl;
txz[ix*n1+iz] -= src.Mxz*src_ampl;
```

F_z source (stype=10)

```
tzz[ix*n1+iz-1] -= src_ampl*roz[ix*n1+iz]/(2.0*mod.dt);
tzz[ix*n1+iz+1] += src_ampl*roz[ix*n1+iz]/(2.0*mod.dt);
```

Moment-tensor (stype=11)

The Moment-tensor is defined by $M_{xx}=1.0$, $M_{zz}=1.0$ and $M_{xz}=1.0$ and implemented the same as the double-couple source.

To have correct amplitudes between the different source types and independence of the discretization in the finite difference code $(\Delta t, \Delta x)$, scaling factors are applied to the input source wavelet amplitudes.

src-type	amplitude
1,2,3,4,9,11	$\frac{\Delta t}{\Delta x^2} C_p^2 \rho$
5,6,7,8	$\frac{\Delta t}{\Delta x^2} \frac{1.6}{\rho}$
10	$\frac{\Delta t}{2\Delta x^3} C_p^2$

TODO: the relative amplitudes of the sources in the elastic scheme are correct, but still have to check the (absolute) source amplitude factors in the elastic scheme (compare them with analytical Green's functions in homogenous medium).

The parameter `grid_dir` reverses the time of the source wavelet and can be used to back-propagate a recorded source field into the medium.

6.5.2 Source positions

Sources can be defined in three different ways in the program. A source can be placed on single grid point, can be placed on many grid points which are starting at the same time creating an areal source field (or plane wave), or sources can be placed at many random positions on the grid starting at random positions in time.

Single or regular shot distribution The position of a single source is set by the parameters `xsrc=` `zsrc=`. The parameters `nshot=` together with `dxshot=` and `dzshot=` determine how many shots are modeled and the position for each shot. For every new shot of the `nshots` to model the `xsrc` and `zsrc` positions are adjusted with `dxshot` and `dzshot`. The loop over the number of shots to model is

```
for (is=0; is<shot.n; is++) {  
    shot->x[is] = xsrc+is*dxshot;  
    shot->z[is] = zsrc+is*dzshot;  
}
```

For example `xsrc=50 zsrc=0 dxshot=15 dzshot=0 nshot=5` will successively model 5 shots at the positions (50,0) (65,0) (80,0) (95,0) and (110, 0). In Figure 14a the black dots represent the source positions which have been defined using a regular shot position. This means that for every black dot in Figure 14a a shot is calculated.

Source arrays An array of sources (or an areal source) is defined with the parameters `xsrca=` `zsrca=`. The parameters define the position of the sources. For example `xsrca=50,55,67,40,12 zsrca=0,10,7,8,10` defines an areal source consisting of 5 shot positions. The shots at these positions are all fired simultaneously. In Figure 14b the black dots represent the source positions, which have been defined using a source array. This means that all black dots in b) will be fired simultaneously and only one shot is calculated. You can also use multiple sources simultaneously by placing multiple source wavelets (with different amplitude and frequency) into the `file_src` file and set the parameter `src_multiwav=1`. The position of these sources are read from the Seismic Unix header values; `gx` for the x-position, and `gelev` for the z-position of the source. Setting parameter `src_at_rcv=0` will use the `sx` for the x-position and `selev` for the z-position. Note that these setting of the `scalco` (`gx`) and `scacl` (`gelev`) factor in the Seismic Unix file. To fire multiple sources (more than 2) after each other, and construct one receiver gather that contains the wavefields generated by these sources, can be achieved by adding a time delay to the traces of `file_src` and set `src_multiwav=1`. All traces will start at the same time in the modeling program. In this case some trace will only have a non-zero amplitude in the beginning and become active (non-zero) after a certain timing.

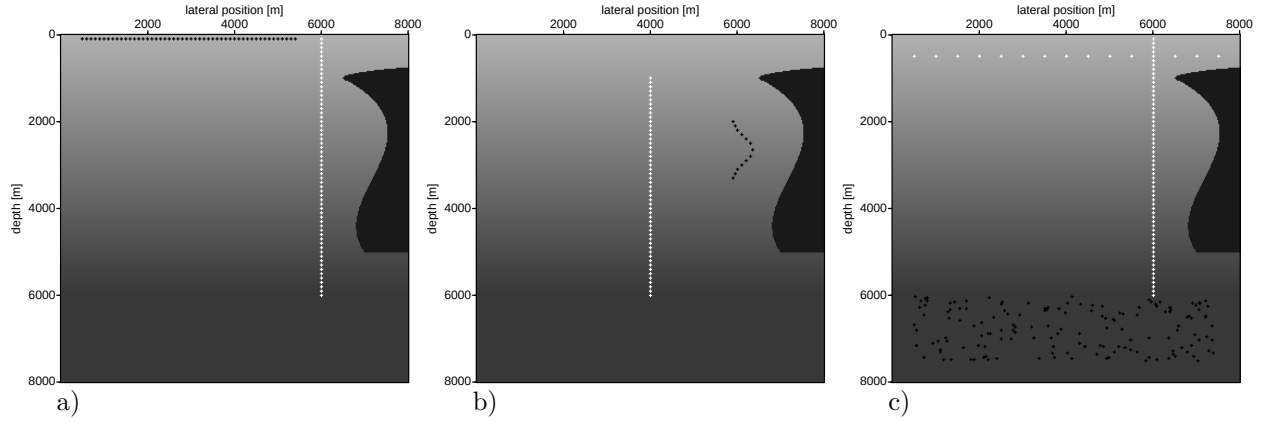


Figure 14: Source (black dots) and receiver (white dots) positions for different choices of the positions parameters. In

- a) `xrcv1=6000 xrcv2=6000 dxrcv=0 zrcv1=100 zrcv2=6000 dzrcv=100 xsrc=500 zsrc=100 nshot=50 dxshot=100 dzshot=0` has been used, for
- b) `xsrc=5900,5950,6000,6100,6200,6300,6350,6300,6200,6100,6000,5950,5900 zsrc=2000,2100,2200,2300,2400,2500,2650,2800,2900,3000,3100,3200,3300 xrcv1=4000 zrcv1=1000 xrcv2=4000 zrcv2=6000 dzrcv=100 dxrcv=0` and for
- c) `xrcv1=6000,500 xrcv2=6000,7500 dxrcv=0,500 zrcv1=100,500 zrcv2=6000,500 dzrcv=100,0 src_random=1 nsrc=150 xsrc1=500 xsrc2=7500 zsrc1=6000 zsrc2=7500`. The script `fdelmodc_srcrec.scr` in the demo directory reproduces the pictures.

Sources arrays from and ASCII text file Reading many source positions with `xsrc= zsrc=` can become cumbersome when many sources are needed. An option has been build in to read source locations from a text file.

With the argument `src_txt=` one can specify an ASCII test file from which source coordinates should be loaded. The file must have the following format:

```
25 10
50 15
75 20
```

each line in this file contains a source location (x,z). Loading this file would place 3 sources at (25,10), (50,15), and (75,20). The shots at these positions are all fired simultaneously like an areal shot record. An example of using `src_txt=` is `fdelmodc_plane_txt.scr`.

Plane wave A plane wave can be defined by using the parameters: `plane_wave= nsrc= src_angle= src_velo= nsrc= src_window=`. The plane wave is implemented by placing a horizontal array of `nsrc=` sources placed on every grid position symmetric around `xsrc=` at the horizontal depth given by `zsrc=`. The angle (ray-parameter) of the plane wave is defined by `src_angle= src_velo=` and is implemented by adding time delays to the shot positions on the horizontal array. Figure 17b shows the plane wave source positions as blue dots in the model.

Source random positions `src_random= nsrc= xsrc1= xsrc2= zsrc1= zsrc2= tsrc1= tsrc2= tactive= tlength= amplitude= distribution= seed=`

When the parameter `src_random=1` is used, `nsrc=` random source positions will be created with positions between `xsrc1= : xsrc2=` and `zsrc1= : zsrc2=`. During the model time `tmod=`, sources will start and at random times in the interval `tsrc1= : tsrc2=` and contribute to the calculated wavefield. The maximum signal length is defined by `tlength=` and the resulting average length = `tlength/2`. Note that if `tlength=` is larger than `tsrc2-tsrc1` then for some sources the time the source is being active (`tlength`) will be truncated to `MIN(tsrc1+tlength, tactive)`. The parameter `tactive=` gives the maximum modeling time the sources are being active. After `tactive=` no sources are being active anymore. The source time settings allows modeling of many random source positions being active by running just one modeling.

The amplitude distribution of the sources is default set to 0, meaning that all sources have the same amplitude. Defining for example `amplitude=10` will introduce an amplitude distribution for the different source with a maximum amplitude of 10. This distribution can be made flat `distribution=0` or Gaussian `distribution=1`. The `seed=` parameter can be used to generate different random sequences, if the same `seed` value is used modeling results can be reproduced. In Figure 14c random source positions are used for the lower part in the model and visible as small black dots in the Figure.

6.6 Receiver, Snapshot and Beam parameters

6.6.1 Receiver, Snapshot and Beam type

The type of wavefield component of the receiver, snapshot and beam recordings which will be written to file can be selected by using `sna_type_*` for snapshots and beam fields and `rec_type_*` for the receivers, where `*` is one of the following characters:

- `p`: P registration with file extension `_rp` only for acoustic scheme
- `vz`: V_z registration with file extension `_rvz`
- `vx`: V_x registration with file extension `_rvx`
- `txx`: T_{xx} registration with file extension `_rtxx`
- `tzz`: T_{zz} registration with file extension `_rtzz`
- `txz`: T_{xz} registration with file extension `_rtxz`
- `dzvz`: $\frac{\partial V_z}{\partial z}$ registration with file extension `_rdzvz`
- `dxvx`: $\frac{\partial V_x}{\partial x}$ registration with file extension `_rdxvx`
- `pp`: P potential registration with file extension `_rP`
- `ss`: S potential registration with file extension `_rS`
- `ud`: up- and down-going decomposition with file extension `_ru`, `_rd` (only for receiver arrays)

In the acoustic scheme only `rec_type_p=`, `rec_type_vz=` and `rec_type_vx=` can be used. In elastic media there is no pressure component, so you can not record it. However, if you set-up an elastic model and put a water layer on top of that elastic model you can place a receiver in that water layer and the T_{zz} component of the elastic modeling scheme will contain the pressure field in the water. We have made a comparison between the T_{zz} component of the elastic field in a water layer with the P field in an acoustic layer and they are identical.

The potential wavefields P and S for the elastic scheme are based on divergence and curl (rotation), respectively, given by:

$$\begin{aligned} \text{rec_pp}[\text{iirec}*\text{rec.nt}+\text{isam}] &= (\text{vx}[\text{ix2}*n1+\text{iz}]-\text{vx}[\text{ix}*n1+\text{iz}] + \\ &\quad \text{vz}[\text{ix}*n1+\text{iz2}]-\text{vz}[\text{ix}*n1+\text{iz}])/\text{mod.dx}; \\ \text{rec_ss}[\text{iirec}*\text{rec.nt}+\text{isam}] &= (\text{vx}[\text{ix2}*n1+\text{iz2}]-\text{vx}[\text{ix2}*n1+\text{iz}] - \\ &\quad (\text{vz}[\text{ix2}*n1+\text{iz2}]-\text{vz}[\text{ix}*n1+\text{iz2}]))/\text{mod.dx}; \end{aligned}$$

The up- and down-going wave fields make use of acoustic decompositions operators Wapenaar (1998). The P and V_z fields needed to do this decomposition are automatically enabled when `rec_type_ud=1` is chosen. In the program all grid points in x , for the chosen receiver-depth level, are stored into memory for the P and V_z fields. The decomposition is carried out, in the wavenumber-frequency domain, on these finely sampled fields. To avoid artefacts from the edges of the model an angle filter is applied in the wavenumber-frequency domain. The cut-off angle in this filter is estimated from the data at the receiver level. The maximum angle present in the receiver data is estimated by summing along $k_x = k \sin(\alpha)$ lines in the wavenumber-frequency amplitude spectrum. The maximum angle in the data is then selected as the angle at the average amplitude (this is just a simple method and can be improved). Figure 15 shows the energy in the angles in the wavenumber domain for the P -field (based on the `decomposition.scr` in

the demo directory). In this example the average energy is 200 and leads to a maximum angle of 67° . The calculated angle file is written to output file `anglerp.su` when the `verbose` option is set to 4 or larger. The use of the calculated angle can be overruled by setting the parameter `kangle=`.

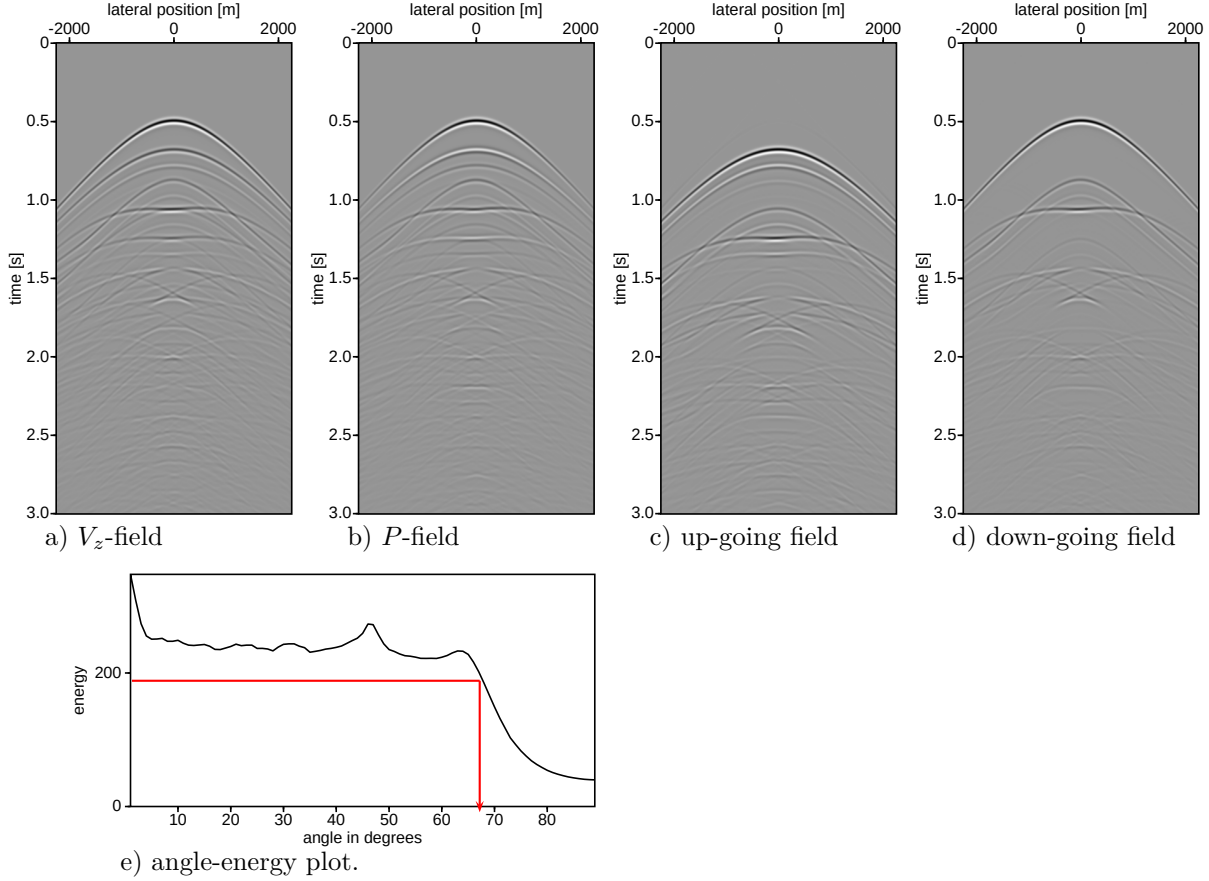


Figure 15: The P (a) and V_z (b) fields and the decomposed up (c) and downgoing (d) fields from example `demo/decomposition.scr`. Selection of maximum angle (e) to filter the decomposition operator in $k_x - \omega$ domain.

The receiver time sampling rate and number of time samples are defined by `dtrcv=` and `rec_ntsam=`. Usually `dtrcv=` is chosen larger than the `dt` of the modeling. Note that `dtrcv=` must be chosen as an integer multiple of the time step `dt` of the modeling. The recording can start later than the modeling, for example to compensate for a time-delay in the source wavelet (see section 6.4) by using `rec_delay=`. Seismic unix (and SEG-Y) uses an unsigned shot integer (16 bytes) in the header section of a trace to store the number of samples in a trace. The maximum value of this short integer is 65536, so more samples can not be stored in a single trace. In passive seismic modeling the recording times can be very long and one easily exceeds 65536 samples in a trace. For long recordings multiple files are created each with `rec_ntsam=` samples per trace in a file. The names of the output files are numbered starting with 001 and this number is placed in the filename just before the name of the receiver type (for example `shotA_001_rp.su`, `shotA_002_rp.su`, `shotA_003_rp.su`, ...).

6.6.2 Receiver positions

Receiver linear array(s) As the name already tells, linear receiver arrays are receivers defined on lines. Using the parameters `xrcv1=`, `xrcv2=` `zrcv1=` and `zrcv2=` multiple lines can be defined. Using `xrcv1=100`, `xrcv2=500` defines one receiver line starting at x-position 100 until 500 m and uses the default z-position which is the top of the model. The distance between the receiver in the line is defined by `dxrcv=` or `dzrcv=`.

Defining multiple lines, for example on multiple depth levels, is accomplished by using `xrcv1=100,100` `xrcv2=500,500` `zrcv1=100,200` `zrcv2=100,200` and `dxrcv=10,10` defines two receiver lines from 100 to

500 m in the lateral direction; one on a depth level of 100 m and another one at 200 m depth. Note that in defining multiple linear arrays the number of arguments in the parameters must be the same. The distance between the receivers can also be different for the different lines by using `dxrcv=10,5` or `dzrcv`. If only one value is used for the distance parameters (one from `dxrcv= dzrcv=`) then all lines will use this distance. A receiver line is defined by the end points: start (`xrcv1,zrcv1`) and end (`xrcv2,zrcv2`) and a distance between the receivers. This distance can be given by `dxrcv=` or `dzrcv`, if both are given then `dxrcv=` is used and `dzrcv` is calculated in the program. However, if `dxrcv=0` and `dzrcv` is also given (and not zero) then the given `dzrcv` is used. Multiple receiver arrays will be placed into the same receiver file in the same order you have defined them. Another example to place a receiver array at the surface and two vertical arrays at $x = 500$ and $x = 1500$ you can use the following parameters:

```
xrcv1=-2500,500,1500 xrcv2=2500,500,1500 zrcv1=0,0,0 zrcv2=0,1000,1000 dxrcv=20,0,0 dzrcv=0,10,10
```

Receiver array An array of receivers at specific points in the medium is defined with the parameters `xrcva= zrcva=`. The parameters define the position of the receivers. For example `xrcva=50,55,67,40,12 zrcva=0,10,7,8,10` defines a receiver array of 5 positions.

Receivers on circle or ellipse To put the receivers on a circle the following parameters can be used: `rrcv=` for the radiation of the circle, `oxrcv,ozrcv` sets the origin (in meter, not grid points) of the circle and `dphi` the distance between the receivers in angle. To make an ellipse use the additional parameter `arcv=`. This sets the size of the vertical arc-length and `rrcv=` represents then the horizontal arc-length. The distance between the receivers on the ellipse are (numerically) made equidistant (the `dphi` parameter is in the ellipse option only used to compute the number of receivers ($=360/\text{dphi}$)). Note that due to the grid of the wavefields the receivers are not placed on exactly a circle/ellipse, but to the closest grid point. The option `rec_int_p` or `rec_int_vz rec_int_vx` set to 3 will interpolate (bi-linear) the fields to the exact position of the receivers on the circle. When `rrcv` is used the parameters `rec_int_p` and `rec_int_vx, rec_int_vz` are automatically set to 3.

Receivers from and ASCII text file *implemented by Max Holicki*

Reading many receiver positions with `xrcva= zrcva=` can become cumbersome when many receivers are needed. An option has been build in to read receiver locations from a text file.

With the argument `rcv_txt=` one can specify an ASCII test file from which receiver coordinates should be loaded. The file must have the following format:

```
25 10
50 15
75 20
```

each line in this file contains a receiver location (x,z). Loading this file would place 3 receivers at (25,10), (50,15), and (75,20).

A demo script to demonstrate the functionality is called `RcvTextInput.scr`.

6.6.3 Interpolation of receiver positions

The parameters `rec_int_vx` and `rec_int_vz` have the options

```
rec_int_vx=0 ..... interpolation of Vx receivers",
                - 0=Vx->Vx (no interpolation)",
                - 1=Vx->Vz",
                - 2=Vx->Txx/Tzz (P)",
                - 3=Vx->receiver position
rec_int_vz=0 ..... interpolation of Vz receivers",
                - 0=Vz->Vz (no interpolation)",
                - 1=Vz->Vx",
                - 2=Vz->Txx/Tzz (P)",
                - 3=Vz->receiver position
```

for reading the wavefield at the receiver positions from the staggered grid positions.

- **rec_int_vz=1** interpolates V_z to the V_x position and makes use of the 4 surrounding V_z points (blue colored items in Figure 16)
- **rec_int_vz=2** interpolates V_z to the σ_p position and uses two 2 (top and down) V_z points (cyan colored items in Figure 16)
interpolates σ_{xz} to the σ_p position and makes use of the 4 surrounding σ_{xz} points (black colored items in Figure 16)
- **rec_int_vx=1** interpolates V_x to the V_z position and makes use of the 4 surrounding V_x points (green colored items in Figure 16)
- **rec_int_vx=2** interpolates V_x to the σ_p position and uses two 2 (left and right) V_x points (magenta colored items in Figure 16)
interpolates σ_{xz} to the σ_p position and makes use of the 4 surrounding σ_{xz} points (black colored items in Figure 16) Using **rec_int_vz=2** and **rec_int_vx=2** puts V_z , V_x and σ_{xz} to the σ_p position.
- **rec_int_vx=3** or **rec_int_vz=3** interpolates all fields to the exact receiver positions given by the user using bi-linear interpolation. In this case the receiver positions do not have to lie on a grid position (yellow colored items in Figure 16). This option is turned on by default when receivers on a circle are defined.

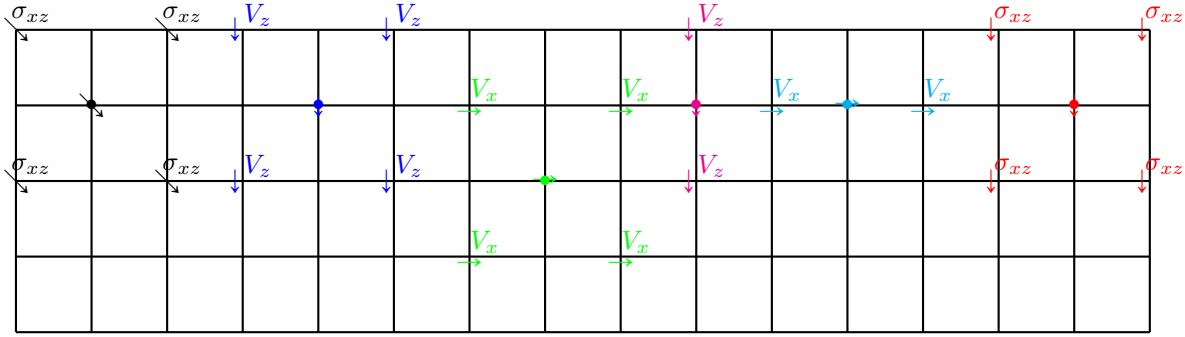


Figure 16: *Receiver interpolation options in the elastic staggered calculation grid. V_z, V_x represent the particle velocity of the wavefield in the z and x direction respectively and $\sigma_p(\sigma_{xx}$ or $\sigma_{zz}), \sigma_{xz}$ represent the stress fields.*

6.6.4 Snapshots and Beams

During modeling time snapshots of complete wavefields can be written to a file **file_snap=snap.su** on disk and, for example, used in a movie showing wave propagation through a model. To define time snapshots the start-time of the first snapshot **tsnap1=**, the time of the last snapshot **tsnap2=**, and the time interval between the snapshots **dt snap1=** have to be defined. Using the default parameters the output size of one snapshot is as large as the gridded model file. To reduce the space of the snapshot, since it might not be needed to have the snapshot on the same fine grid as the modeling, **dxsnap=** and **dzsnap=** can be used to increase the grid distance in the x and z direction. If one is only interested in a certain area the parameters **xsnap1=**, **xsnap2=**, **zsnap1=**, and **zsnap2=** can be used to define that area. The finite-difference scheme is also staggered in time and the parameter **sna_vxvztime** defines which registration of vx/vz times is used in the snapshots.

- 0=previous vx/vz relative to $txx/tzz/txz$ at time t
- 1=next vx/vz relative to $txx/tzz/txz$ at time t

Beams represent the energy of the wavefield during the complete model time. Beams are calculated as the square root of the quadratic field quantities and can be used to investigate how energy propagates through a model. During every time step the 'energy' is calculated and added to the beam array. Beams

are enabled by setting `beam=1` and the same parameters as for the snapshots are used to define grid distance `dxsnap=` and `dzensap=`, area of interest, and type of wavefield component. Note that the beam calculation is done for every time-step and is an expensive computational operation, the total compute time can easily increase by 50% or more.

6.7 Verbose

The parameter `verbose` prints messages and produces additional files during the running of the program. Table 1 shows the kind of messages and the extra files printed using different values for `verbose`. Those messages and files contain extra information for the user. The output files produced by different setting of the verbose parameter are:

- `src_nwav.su`: file which contains the source signatures. For the noise sources each trace in this file contains the noise signal used in the modeling. Note that this file can become large for long wavelengths
- `SrcRecPositions.su`: SU file with the same size as the input model files. At every source position a square (of 5x5 grid points) with value 1 is positioned and at every receiver position a square with value -1 is placed. This file can be used to overlay (after scaling) the source and receiver positions on a velocity model. For example the following command:

```
suop2 SrcRecPositions.su vel2_edge_cp.su w1=6000 w2=1.0 op=sum | suximage bclip=6000
wclip=0
```

adds source and receiver positions to the P-wave velocity file, where the weight factor `w1` is set larger than the maximum velocity in the P-wave velocity file.

- `srcTimeLengthN=ns.bin`: 32 bits floating point binary file with `ns` samples where each sample value gives the start time of the source.
- `SrcPositionsNSRC.txt`: ASCII file which contains the source positions where NSRC is the number of sources.
- `RcvPositionsNRCV.txt`: ASCII file which contains the receiver positions where NRCV is the number of receivers.
- `src_ampl.su`: when the `amplitude` parameter is used this file contains the amplitude distribution of the sources.

setting	messages printed to stdout	files written to disk
0	no messages only warnings	no files
1	model, source, receiver, snapshot and stability info	no files
2	same as 1	no files
3	+ file-info from model and wavelet files	src_ampl.su
4	+ receiver grid positions, source positions	+ srcTimeLengthN=ns.bin, SrcRecPositions.su, src_nwav.su
>4	+ source grid points, amplitude and start time, topography surface points	same as 4

Table 1: The files and messages produced by different values of the `verbose` parameter. The file `src_ampl.su` contains the amplitude variation of the (random) source signals when the parameter `amplitude=` is set to a non-zero value. `srcTimeLengthN=ns.bin` is a binary file with `ns` samples which give the start time of the sources. `SrcRecPositions.su` is a SU file with the same size as the input model files and at every source position a cross with value 1 is positioned. The other positions in the file are set to zero. File `src_nwav.su` contains the generated random source signals. Note that this file can become large for long wavelengths (`length=`)

7 Examples to run the code

The demo directory contains scripts which demonstrate the different possibilities of the modeling program. In the subsections below most demo script are explained and results are shown. To reproduce the Figures shown in the GEOPHYSICS manuscript "Finite-difference modeling experiments for seismic interferometry" the scripts in FiguresPaper directory can be used. Please read the README in the FiguresPaper directory for more instructions and guidelines.

7.1 Example for plane waves: `fdelmodc_plane.scr`

Modeling plane waves can easily be done by setting the plane wave option `plane_wave=1`. The demo script `fdelmodc_plane.scr` is set-up for an elastic medium. The number of sources used to make the plane wave is controlled with `nsrc=`. The centre of the plane wave will be positioned at the source position defined by the parameters `xsrc= zsrc=`. Left and right of this central position $(nsrc-1)/2$ sources are placed on every grid point in the lateral (x) direction. In the example below the plane wave has been given an angle of 5° using `src_velo=1800 src_angle=5`. The script file `demo/fdelmodc_plane.scr` contains all the commands used to make the plane waves based figures shown in this subsection. The runtime of the script is approximately 120 seconds.

The gridded model is shown in Figure 17 and contains one flat smooth interface, a vertical velocity gradient, and a curved interface.

```
../fdelmodc \
  file_cp=$filecp file_cs=$filecs file_den=$filero \
  ischeme=3 \
  file_src=wavelet.su verbose=1 \
  file_rcv=rec.su \
  file_snap=snap.su \
  xrcv1=0 xrcv2=2000 dxrcv=15 \
  zrcv1=400 \
  rec_type_vx=1 rec_type_pp=1 rec_type_ss=1 rec_int_vx=1 \
  dtrcv=0.004 \
  xsrc=1000 zsrc=1700 nshot=1 plane_wave=1 nsrc=301 \
  src_type=1 tmod=3.0 src_velo=1800 src_angle=5 \
  ntaper=120 \
  left=4 right=4 bottom=4 top=4 \
  tsnap1=0.1 tsnap2=3.0 dtsnap=0.1 \
  sna_type_ss=1 sna_type_pp=1 verbose=4
```

The receivers are placed at $z = 400$ m. The measured particle velocity field V_z and the calculated P- and S-wave potentials are shown in Figure 18a,b, and c respectively. Snapshots, where three modeling times are added together, are shown in Figure 18d,e, and f. In the S-potential snapshot (Figure 18f) it is observed that the S-waves are firstly occurring when the P-wave hits the curved reflector. Note also the better resolution of the S-potential. In the run script the verbose option is set to 4, which means that auxiliary files are also written to disk. These files contain information about the used source and receiver positions and can be used to plot the source and receiver positions in the model.

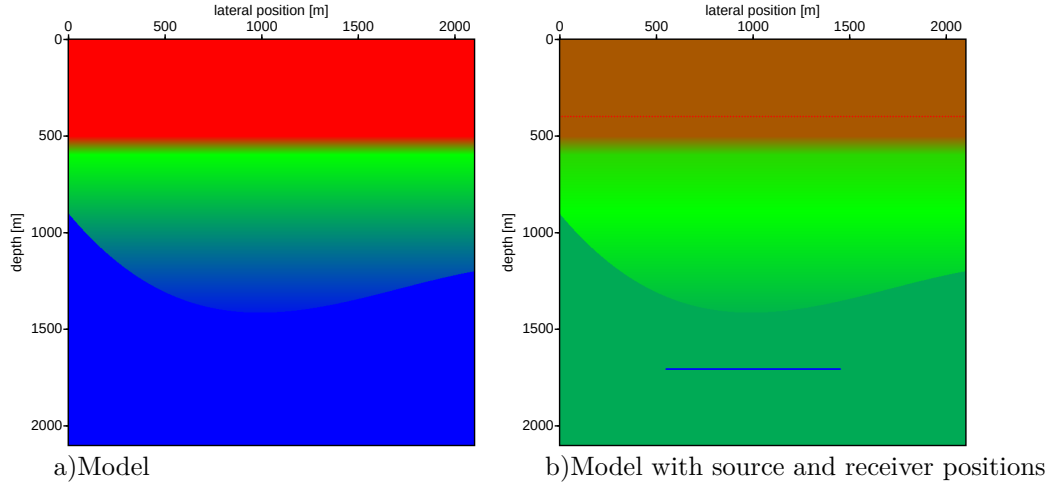


Figure 17: Gridded model used to model a plane wave at $x = 1000, z = 1700$. The receivers are placed at depth level $z = 400$ m, just outside the taper area at the top, which ends at $z = 360$ m. Source (blue dots) and receiver (red dots) positions are shown in b).

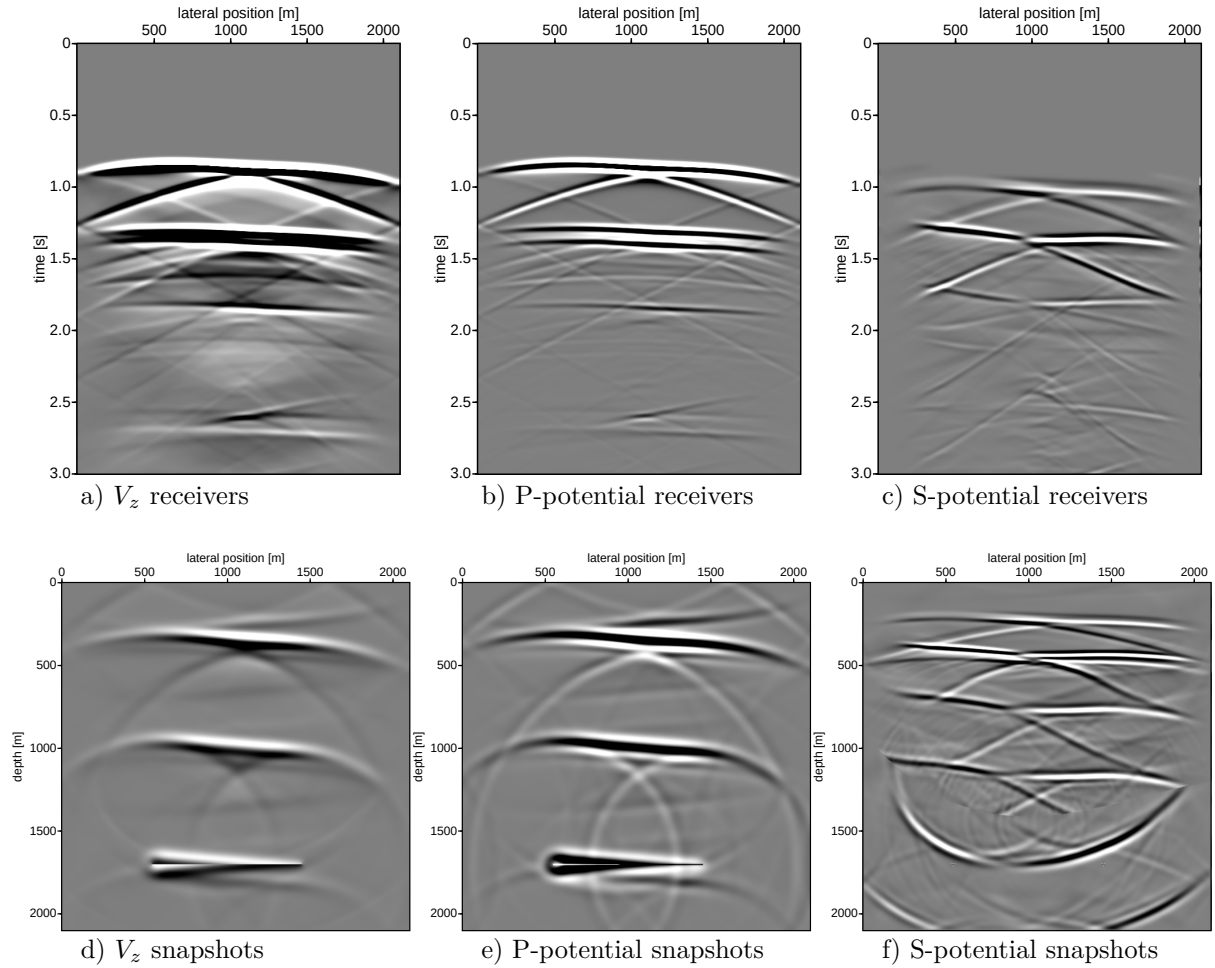


Figure 18: Panel a, b, and c show three different types of receiver measurements at depth $z = 400$ m. Panel d, e, and f show three snapshots at $t = 0.5, 0.9, 1.3$ for different types of wavefield components.

7.2 Example for viscoelastic media: `fdelmodc_visco.scr`

Visco-elastic modeling is enabled by choosing the parameter `ischeme=4`. In the demo script `fdelmodc_visco.scr` a model is created with `makemod` for the density and the P and S wave velocities. Besides the model which describes the medium parameters another gridded model is created with `makemod` for the Q-values (Q_p and Q_s) of the medium. The parameters of `makemod` are set-up to be used to create `cp,cs` and `rho` gridded model files. However, it can be (ab-)used for building any gridded file, where the values of `cp`, `cs` and `rho` just get another meaning. To create the Q-values with `makemod` the `cp` velocity is used as Q_p and the `cs` value is used as Q_s , the density value is not used. In this way the Q-model is created and used in `fdelmodc`. The results of running the demo script is shown in Figure 19.

Another trick has been used to subtract the direct field of the measured recordings. This is done by running `fdelmodc` a second time, and this time in a homogeneous medium with the velocity of the layer where the receivers are placed. Then the recorded fields of the direct field is subtracted from the completed field resulting in only the reflected field. Running the `fdelmodc_visco.scr` takes about 10 minutes.

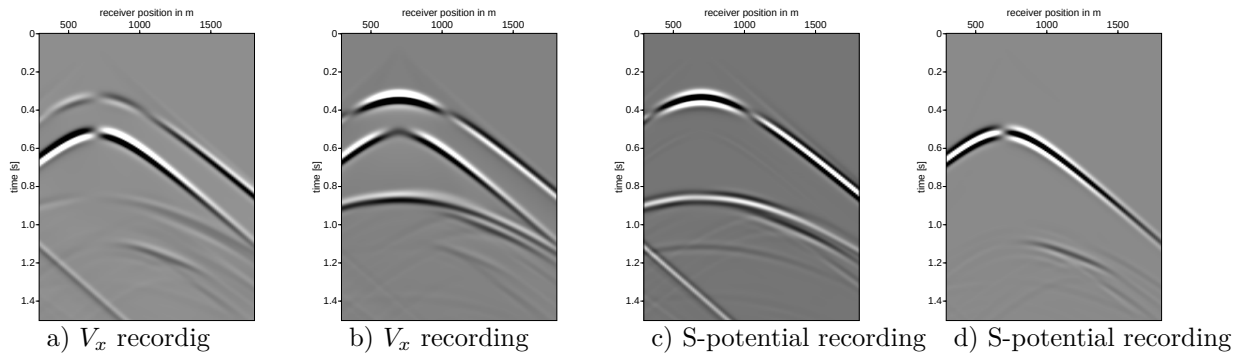


Figure 19: Panel a, b, c and d show different types of receiver measurements in a visco-elastic medium. Note that the direct field has been subtracted from the recordings.

7.3 Example for different source distributions: `fdelmodc_sourcepos.scr`

Figure 20 shows recorded data for different kind of source distributions and different source signatures. The used source distributions are random source positions between $500 \leq z \leq 4100$, random positions between $2700 \leq z \leq 4100$, and a plane layer at $z = 2700$ m. For the source positions 800 sources are used, for the sources positioned on the vertical plane (20c and f) on every grid point a source is placed. Two types of source signatures are used; a Ricker wavelet with a frequency peak at 10 Hz, and random source signals with a maximum frequency of 30 Hz.

The script file `demo/fdelmodc_sourcepos.scr` show how these 6 modeled shots can easily be modeled. The script illustrates how to define different source distributions within the program and how a random source signature is defined. These kind of experiments can be used to investigate the sensitivity of seismic interferometry on different source distributions (Thorbecke and Draganov, 2011). Each modeling takes about 100 seconds making the total runtime of the script about 10 minutes.

7.4 Example with receivers on a circle: `fdelmodc_circ.scr`

Receivers can be placed on a circle using the parameters `rrcv=` for the radiation of the circle, `oxrcv, ozrcv` sets the origin (in meter, not grid points) of the circle and `dphi` the distance between the receivers in angle. Note that due to the grid of the wavefields the receivers are not placed on exactly a circle, but to the closest grid point. The option `rec_int_vx` or `rec_int_vz` set to 3 will interpolate (bi-linear) the fields to the exact position of the receivers on the circle. When `rrcv` is used the parameters `rec_int_vx` and `rec_int_vz` are automatically set to 3. The result of using this option, in a homogeneous background medium with a contrast placed in the middle of the model, is shown in Figure 21. The runtime of this script is 40 seconds.

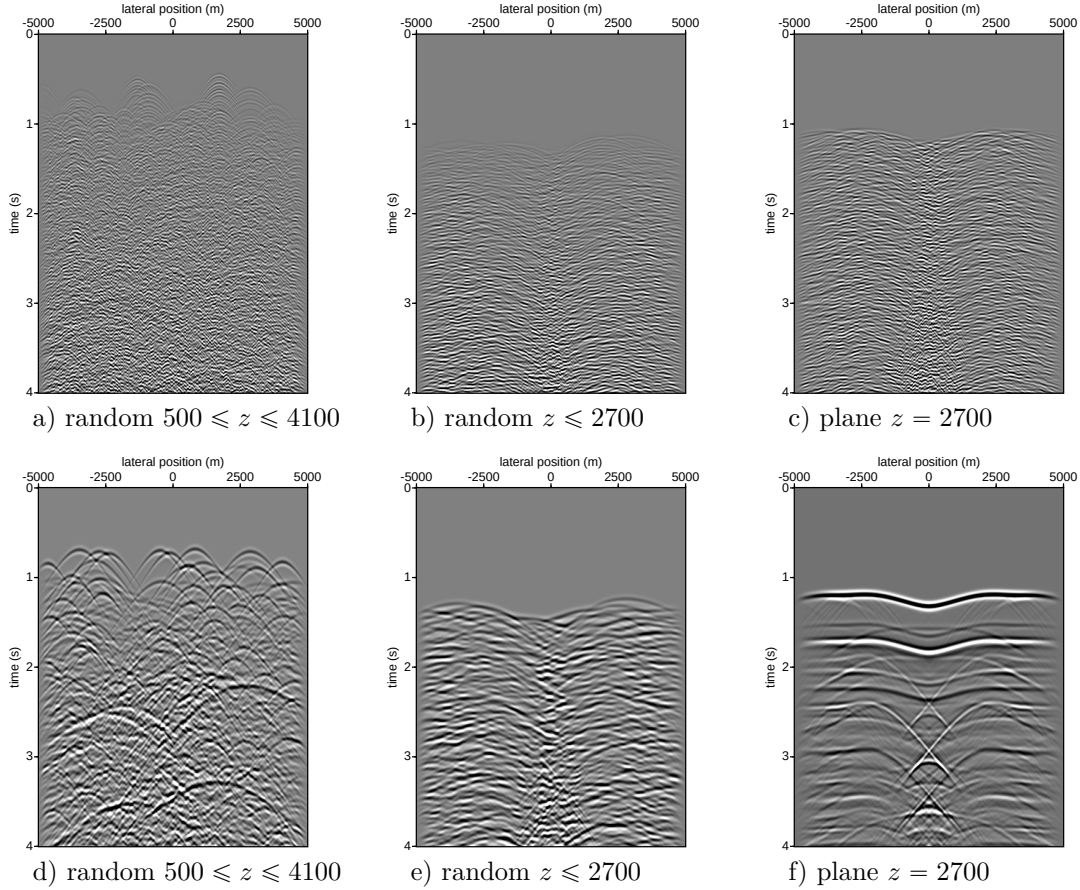


Figure 20: To investigate the type of noise present in the reconstructed reflection from seismic interferometry different experiments have been carried out. The first 4 seconds of different types of source signatures and source distribution are shown in a to f. Noise signatures are used in a, b and c and a Ricker wavelet is used on d, e and f. The sources are distributed in the area indicated in the caption of the pictures: for x -positions in the indicated z -range.

7.5 Example with topography: `fdelmodc_topography.scr`

To make a more complicated a topography can be included. The receivers are placed on this topography. A directly modeled result is generated with an active source in the middle of the model and is shown in Figure 22. To include topography in a model the c_p velocity must be chosen zero in the layer above the topography. Note that the density must not be set to zero. The source and receiver positions are then given on a horizontal surface above the defined topography. The code then searches for the first non-zero c_p value below the horizontal surface. In this way the source and receiver positions are placed at the first non-zero c_p position. The parameter `sinkdepth=` can be used to place the source and receiver `sinkdepth` grid positions deeper than the first non-zero velocity. The command how to make the model and how the shot record is computed is shown below and can be found in the script file `demo/fdelmodc_topography.scr`. The runtime of this script is 500 seconds.

Note that the receiver (not the source) positions can also be sunk through a layer, with a non-zero velocity, to the next interface when the parameter `sinkvel=` is defined. The receivers must then be placed somewhere in that layer and only works for homogeneous layers. An example of usage is to model OBC data where the receivers have to be placed on the topography of the sea-bottom and the sources are close to the free surface. Using `sinkvel=1` will place the receivers at the bottom of the sea layer. The script `fdelmodc_topography.scr` can also be used to generate OBC data. In the script you then have to change in `makemod cp0=1500` and in `fdelmodc` add the parameter `sinkvel=1`.

```
makemod sizex=10000 sizez=4100 dx=5 dz=5 cp0=0 ro0=1000 file_base=real2.su \
  orig=0,-800 gradunit=0 \
  intt=def poly=2 cp=2450 ro=1000 gradcp=14 grad=0 \
```

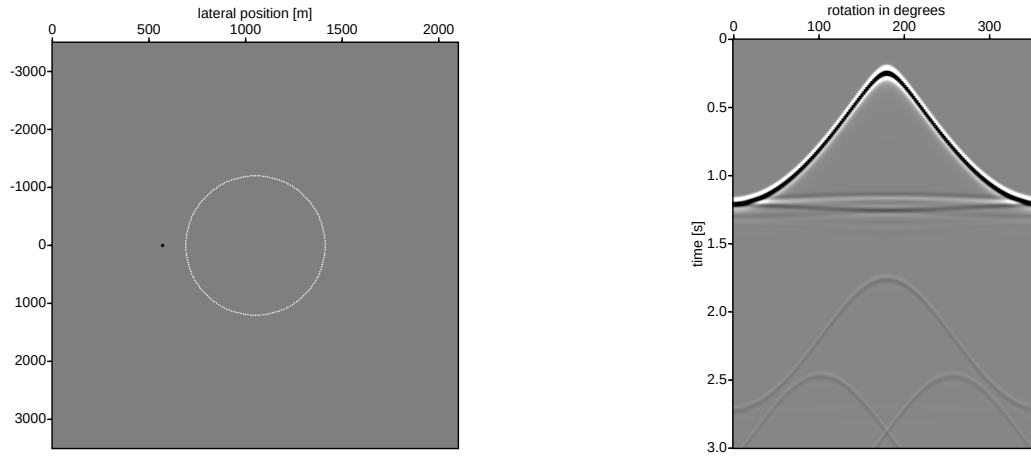


Figure 21: The middle of the model contains a circular contrast and the receiver are placed on a circle around this contrast. The left picture show the source (black dot) and receiver (white dots) positions. Right is the recorded wavefield at the receiver positions.

```

x=0,1000,1700,1800,2000,3000,4000,4500,6000,6800,7000,7500,8100,8800,10000 \
z=-100,-200,-250,-200,-200,-120,-300,-600,-650,-500,-350,-200,-200,-150,-200 \
intt=rough var=200,3.2,1 poly=2 x=0,3000,8000,10000 \
      z=400,250,300,500 cp=4500,4200,4800,4500 ro=1400 gradcp=5 grad=0 \
intt=def poly=2 x=0,2000,3000,5000,7000,8000,10000 \
      z=1100,1100,1100,1600,1100,1100,1100 cp=4000 ro=2000 gradcp=8 grad=0 \
intt=def poly=0 x=0,10000 z=1750,2050 cp=4500,5100 ro=1500 gradcp=13 grad=0 \
intt=def poly=0 x=0,10000 z=1850,2150 cp=6000,4200 ro=1500 gradcp=14 grad=0 \
intt=def poly=0 x=0,10000 z=1950,2250 cp=4800,4800 ro=1500 gradcp=5 grad=0 \
intt=def poly=0 x=0,10000 z=2000,2300 cp=6100,5000 ro=1500 gradcp=13 grad=0 \
intt=def poly=0 x=0,10000 z=2100,2400 cp=3800,5000 ro=1500 gradcp=20 grad=0 \
intt=def poly=0 x=0,10000 z=2150,2450 cp=5000 ro=1500 gradcp=14 grad=0 \
intt=def poly=0 x=0,10000 z=2350,2650 cp=5800 ro=1500 gradcp=5 grad=0 \
intt=def poly=0 x=0,10000 z=2600,2600 cp=5500 ro=2200 gradcp=5 grad=0

fdelmodc \
  file_cp=vel2_edge_cp.su ischeme=1 \
  file_den=vel2_edge_ro.su \
  file_rcv=shot_real2_x5000_topo.su \
  file_src=G2.su \
  dtrcv=0.004 \
  verbose=4 \
  tmod=3.004 \
  dxrcv=20.0 \
  zrcv1=-800 \
  xrcv1=0 \
  xrcv2=10000 \
  sinkdepth=1 \
  src_random=0 \
  wav_random=0 \
  xsrc1=5000 \
  xsrc2=5000 \
  zsrc1=-800 \
  tsrc1=0.0 \
  dipsrc=1 \
  ntaper=$ntap \
  left=4 right=4 bottom=4 top=1

```

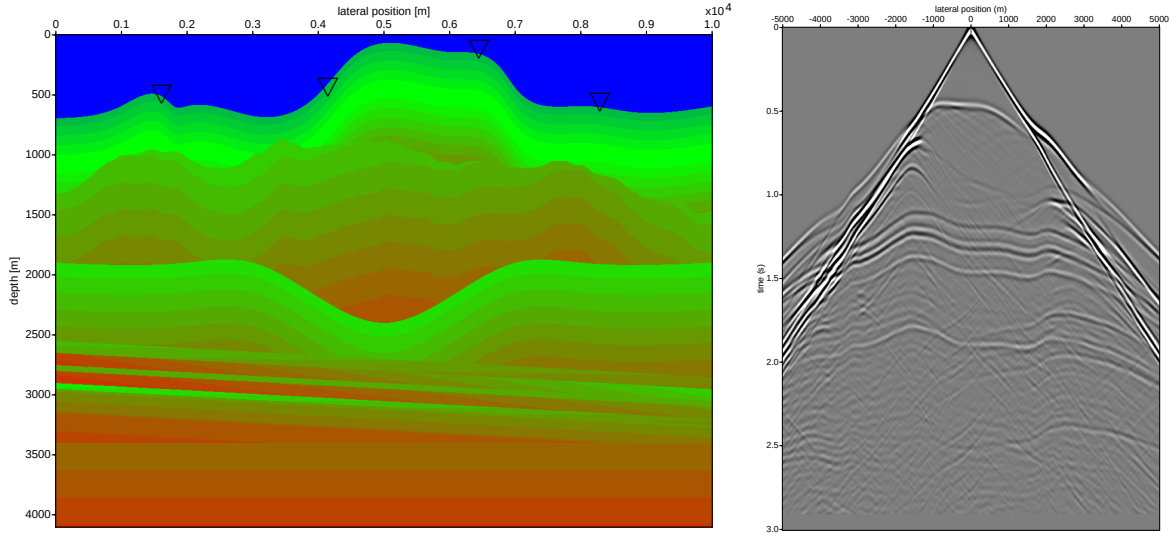


Figure 22: The receivers are placed from -5000 m to 5000 m at every 20 m on the topography. The modeled reflection response, for a source at 5000 m placed on the the topography, is shown in the right picture.

7.6 Example verification with analytical results: FigureGreenDxAppendixA.scr

7.6.1 Acoustic

To verify the accuracy and the correctness of the FD program we have compared the finite-difference calculation of a Green's function in a homogenous acoustic medium with the analytical Green's function. Four analytical Green's functions have been used for verification using:

- monopole source and pressure (P) receivers,
- monopole source and vertical particle-velocity (V_z) receivers,
- dipole source and P receivers,
- dipole source and V_z receivers.

The wave equation solved by the FD program implements the following source term for volume injection sources (parameter `src_injectionrate=0`, which is the default setting):

$$\begin{aligned} \rho \delta(\mathbf{x} - \mathbf{x}_s) \delta(t) \\ \rho \delta(\mathbf{x} - \mathbf{x}_s) \end{aligned} \quad (126)$$

and for volume injection rate sources (parameter `src_injectionrate=1`):

$$\begin{aligned} \rho \delta(\mathbf{x} - \mathbf{x}_s) \frac{\partial \delta(t)}{\partial t} \\ j\omega \rho \delta(\mathbf{x} - \mathbf{x}_s) \end{aligned} \quad (127)$$

The analytical Green's functions for volume injection sources are given by (Berkhout, 1987, page 141-146):

$$\hat{G}^{p,p}(\mathbf{x}, \mathbf{x}_s) = P^{mon} = \frac{-j\rho}{4} H_0^{(2)}(kr), \quad (128)$$

$$\hat{G}^{v,p}(\mathbf{x}, \mathbf{x}_s) = V_z^{mon} = \frac{\cos(\phi)}{4c} H_1^{(2)}(kr), \quad (129)$$

$$\hat{G}^{p,v}(\mathbf{x}, \mathbf{x}_s) = P^{dip} = j\omega \frac{\cos(\phi)}{4c} H_1^{(2)}(kr), \quad (130)$$

$$\hat{G}^{v,v}(\mathbf{x}, \mathbf{x}_s) = V_z^{dip} = \frac{-k\cos^2(\phi)}{4\rho c} H_0^{(2)}(kr) - \frac{k(1 - 2\cos^2(\phi))}{4\rho ckr} H_1^{(2)}(kr), \quad (131)$$

where

$$H_0^{(2)}(kr) = J_0(kr) - jY_0(kr), \quad (132)$$

$$H_1^{(2)}(kr) = J_1(kr) - jY_1(kr), \quad (133)$$

$$r = \sqrt{(x^2 + (z_s - z_r)^2)}, \quad (134)$$

$$\cos(\phi) = \frac{|z_s - z_r|}{r}, \quad (135)$$

x represents the lateral distance and z_s and z_r are the depth positions of the source and receiver, respectively. J_0 and J_1 are the Bessel functions of the first kind of orders 0 and 1, respectively. Y_0 and Y_1 are the Bessel functions of the second kind of orders 0 and 1, respectively. The wavenumber $k = \omega/c$, where c is the velocity of the medium. The analytical responses are generated by the program 'green', included in the source code distribution in the `utils` directory. In the beginning of section 6.5 it is explained that an injection source is implemented (and not injection rate). Sometimes in literature equations 128-131 have an extra factor $j\omega$ when injection rates are used.

Note that in 2D media the far field expression of the Hankel functions in equations (128)-(131) contain a 45 degree phase-shift. This phase shift is of course also present in the computed FD results. For example if you have a Ricker wavelet as input source (e.g. modelling in a homogeneous medium), then the recorded wave will have a 45 degree phase shift compared to the input Ricker wavelet. Another way to see this is, is to realise that a point source in 2D is represented by a line source in 3D. For a more detailed explanation see also (Berkhout, 1987, page 141-146).

In the staggered-grid implementation, the P - and V_z -fields are positioned at different spatial grids and the V_z fields have been interpolated to the P -field grid position to be able to compare them with the analytical solution positioned at the P -field position. The FD scheme is also staggered in time and the modeled P -field is shifted half a time step compared to the V_z -field. For the implementation of a dipole source, two grid positions are used and this gives an extra time delay of $\frac{0.5\Delta z}{c}$, where c is the velocity at the source position and Δz the discretization step in the z -direction. In the comparison with the analytical Green's functions these discretization effects have all been taken into account.

The reference medium has a velocity of 2000 m/s and a density of 1000 kg/m³. The source is positioned 500 m below the receiver. For the finite-difference code a spatial grid of 2.5 m and a time step of 0.5 ms has been used to compute the results.

The comparison between the analytical Green's functions and the finite-difference computed results are shown in Figure 23 for the four different configurations mentioned. The curves are perfectly overlapping and only after zooming in at the peak of the wavelet it is possible to observe differences. The difference between the analytical Green's function and the FD results is less then 1% of the peak of the analytical Green's function.

The script to reproduce the verification figures can be found in FiguresPaper directory (`FigureGreenAppendixA.scr`).

The difference between the analytical Greens function and the finite-difference result is less then 1% and shown in Figure 24 for the same Green's functions shown in Figure 23. The small peak at 0.4115 seconds is caused by the program to compute the shift of -0.1 seconds and caused by the the limited number of time samples. This shift puts the peak of the input wavelet at $t = 0$. To have the wavefields on the same grid the V_z grid has been interpolated to the P grid and also an extra time shift of a half Δt has been used to compensate for the staggering in time.

The difference between the analytical Green's function and the finite-difference result using a smaller grid spacing will give smaller errors and larger grid spacing will give larger errors. This is shown in Figure 25. Note that for $\Delta = 5$ dispersion is starting to become visible in the difference plot, but the error is still smaller than 1.2%. The script to reproduce the pictures of Figure 25 can be found in the `fdelmodc/FiguresPaper` directory (`FigureGreenDxAppendixA.scr`). The FD program takes 60 seconds to model the result using a grid spacing of $\Delta = 5$, 200 seconds with $\Delta = 2.5$ and 800 seconds with $\Delta = 1.0$.

7.6.2 Elastic

This section is made together with Karel van Dalen, who has derived the Green's function in a homogenous elastic medium.

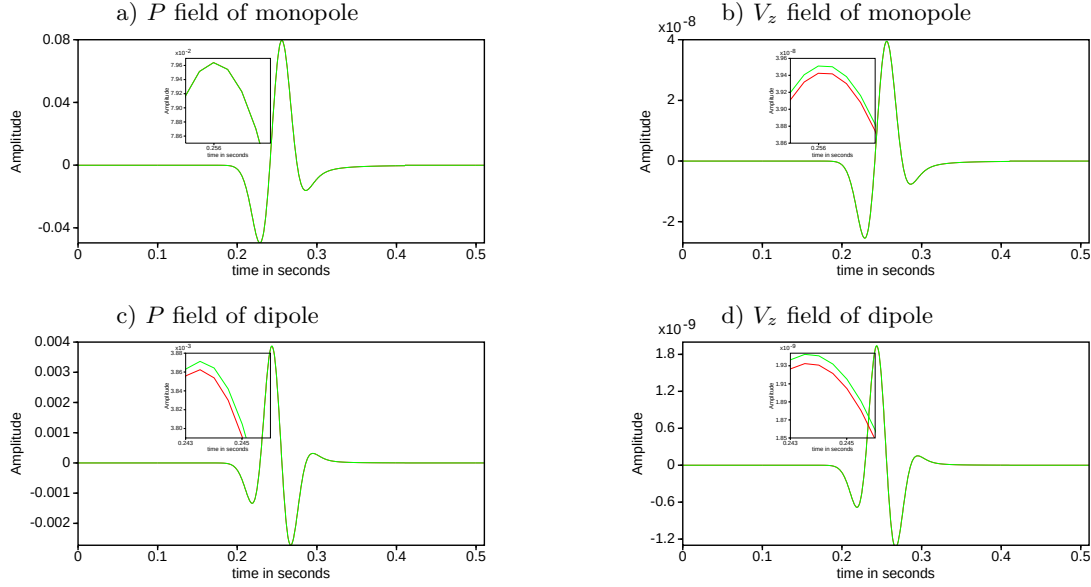


Figure 23: Comparison of Green's functions in an acoustic homogeneous medium for monopole (top) and dipole sources (bottom) with pressure (P) and particle velocity (V_z), left and right, respectively, recorded wavefields. The onsets show the differences for the positive peak of the wavelet, the lower line represents the finite-difference result. The script `FigureGreenAppendixA.scr` in the `FiguresPaper` directory calculates the data and reproduces the pictures.

To verify the accuracy and the correctness of the FD program we have compared the finite-difference calculation of a Green's function in a homogenous elastic medium with the analytical Green's function. Four analytical Green's functions have been used for verification :

- force source in x -direction source and vertical particle-velocity (V_z) receivers ($\mathbf{F}\mathbf{x}=1$),
- force source in z -direction source and vertical particle-velocity (V_z) receivers ($\mathbf{F}\mathbf{z}=1$),
- deformation source in x -direction source and vertical particle-velocity (V_z) receivers ($\mathbf{T}\mathbf{x}=1$),
- deformation source in z -direction source and vertical particle-velocity (V_z) receivers ($\mathbf{T}\mathbf{z}=1$).

General Green's functions:

$$\hat{G}_{i,j}^{v,f}(\mathbf{x}, \mathbf{x}_s) = \frac{j\omega}{\rho} \left\{ \frac{1}{c_s^2} \delta_{ij} \hat{G}_s(\mathbf{x}, \mathbf{x}_s) - \frac{1}{\omega^2} \partial_i \partial_j (\hat{G}_p(\mathbf{x}, \mathbf{x}_s) - \hat{G}_s(\mathbf{x}, \mathbf{x}_s)) \right\} \quad (136)$$

$$\hat{G}_p(\mathbf{x}, \mathbf{x}_s) = \frac{-j}{4} H_0^{(2)}(k_p r), \text{ with } k_p = \frac{\omega}{c_p} \quad (137)$$

$$\hat{G}_s(\mathbf{x}, \mathbf{x}_s) = \frac{-j}{4} H_0^{(2)}(k_s r), \text{ with } k_s = \frac{\omega}{c_s} \quad (138)$$

$$\partial_i \partial_j \hat{G}(\mathbf{x}, \mathbf{x}_s) = \frac{-j}{4} \left\{ -\frac{\bar{x}_i \bar{x}_j}{r^2} k^2 H_0^{(2)}(kr) + k \left(2 \frac{\bar{x}_i \bar{x}_j}{r^3} - \frac{\delta_{ij}}{r} \right) H_1^{(2)}(kr) \right\} \quad (139)$$

$$\partial_z \partial_z \hat{G}(\mathbf{x}, \mathbf{x}_s) = \frac{-j}{4} \left\{ -\frac{\bar{z}^2}{r^2} k^2 H_0^{(2)}(kr) + k \left(2 \frac{\bar{z}^2}{r^3} - \frac{1}{r} \right) H_1^{(2)}(kr) \right\} \quad (140)$$

$$\partial_z \partial_x \hat{G}(\mathbf{x}, \mathbf{x}_s) = \frac{j \cos(\phi) \bar{x}}{4r} \left\{ k^2 H_0^{(2)}(kr) - \frac{2k}{r} H_1^{(2)}(kr) \right\} \quad (141)$$

$$(142)$$

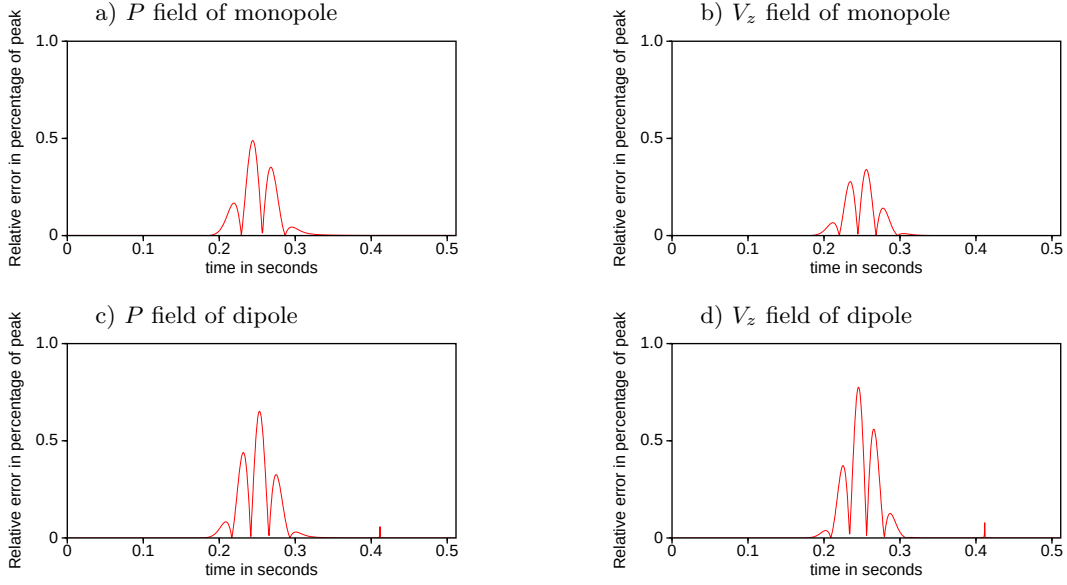


Figure 24: Difference between the analytical Green's function and the finite-difference result in an acoustic medium for monopole and dipole sources for P and V_z recorded fields. The difference is shown as percentage of the maximum peak in the analytical Green's function. A grid spacing of $\Delta = 2.5$ m is used. The script `FigureGreenDxAppendixA.scr` in the `FiguresPaper` directory calculates the data and reproduces the pictures.

The corresponding analytical Green's functions are given by:

$$\hat{G}^{v,fx}(\mathbf{x}, \mathbf{x}_s) = -\frac{j}{\omega\rho} \{\partial_z \partial_x (\hat{G}_p(\mathbf{x}, \mathbf{x}_s) - \hat{G}_s(\mathbf{x}, \mathbf{x}_s))\}, \quad (143)$$

$$= \frac{\bar{z}\bar{x}}{4\omega\rho r^2} \{ (k_p^2 H_0^{(2)}(k_p r) - \frac{2k_p}{r} H_1^{(2)}(k_p r)) - (k_s^2 H_0^{(2)}(k_s r) - \frac{2k_s}{r} H_1^{(2)}(k_s r)) \} \quad (144)$$

$$\hat{G}^{v,fz}(\mathbf{x}, \mathbf{x}_s) = \frac{j\omega}{c_s^2\rho} \hat{G}_s(\mathbf{x}, \mathbf{x}_s) - \frac{j}{\omega\rho} \{\partial_z \partial_z (\hat{G}_p(\mathbf{x}, \mathbf{x}_s) - \hat{G}_s(\mathbf{x}, \mathbf{x}_s))\}, \quad (145)$$

$$= \frac{\omega}{4c_s^2\rho} H_0^{(2)}(k_s r) \quad (146)$$

$$- \frac{k_p}{4\omega\rho r^2} \{ -\bar{z}^2 k_p H_0^{(2)}(k_p r) + (\frac{\bar{z}^2 - \bar{x}^2}{r}) H_1^{(2)}(k_p r) \} \quad (147)$$

$$+ \frac{k_s}{4\omega\rho r^2} \{ -\bar{z}^2 k_s H_0^{(2)}(k_s r) + (\frac{\bar{z}^2 - \bar{x}^2}{r}) H_1^{(2)}(k_s r) \} \quad (148)$$

$$(149)$$

$$\hat{G}^{v,\tau_{xz}}(\mathbf{x}, \mathbf{x}_s) = \quad (150)$$

$$(151)$$

$$\hat{G}^{v,\tau_{zz}}(\mathbf{x}, \mathbf{x}_s) = \quad (152)$$

$$(153)$$

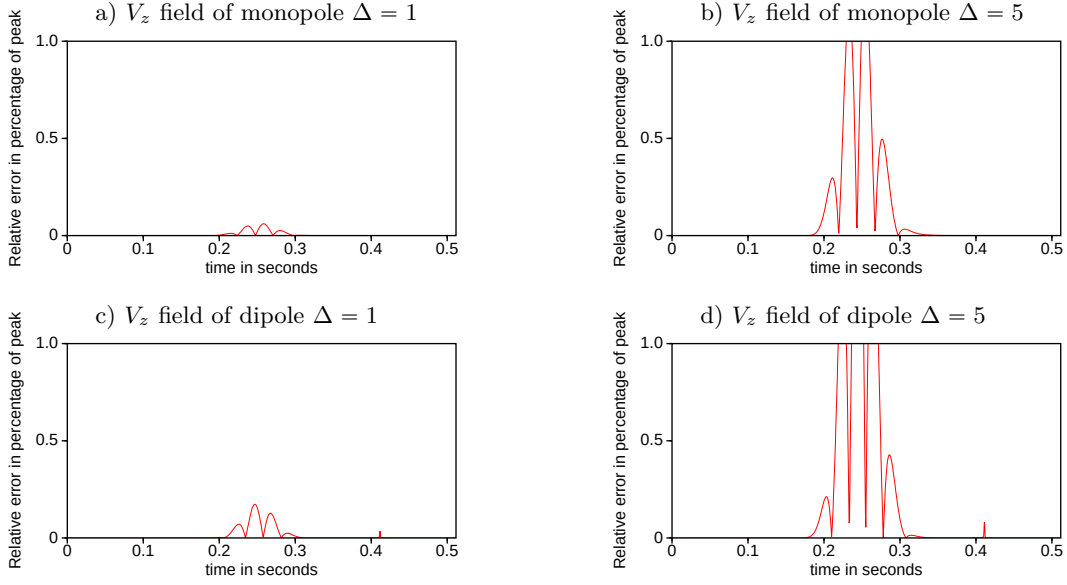


Figure 25: Difference between the analytical Green's function and the finite-difference result in an acoustic medium for monopole and dipole sources for V_z recorded fields using a grid spacing of 1 and 5 meter. The difference is shown as percentage of the maximum peak in the analytical Green's function. The larger errors for the $\Delta = 5$ model results indicates that dispersion starts to develop. The script `FigureGreenDxAppendixA.scr` in the `FiguresPaper` directory calculates the data and reproduces the pictures.

where

$$H_0^{(2)}(kr) = J_0(kr) - jY_0(kr), \quad (154)$$

$$H_1^{(2)}(kr) = J_1(kr) - jY_1(kr), \quad (155)$$

$$\bar{x} = x - x_s, \quad (156)$$

$$\bar{z} = z - z_s, \quad (157)$$

$$r = \sqrt{(x - x_s)^2 + (z - z_s)^2}, \quad (158)$$

$$r = \sqrt{\bar{x}^2 + \bar{z}^2}, \quad (159)$$

$$\cos(\phi) = \frac{|z - z_s|}{r}, \quad (160)$$

x represents the lateral distance and z_s and z are the depth positions of the source and receiver, respectively. J_0 and J_1 are the Bessel functions of the first kind of orders 0 and 1, respectively. Y_0 and Y_1 are the Bessel functions of the second kind of orders 0 and 1, respectively. The wavenumber $k = \omega/c$, where c is the velocity of the medium. The analytical responses are generated by the program 'green', included in the source code distribution in the `utils` directory.

7.7 Verification with Scattered field of a cylinder

The directory `demo/matlab` contains 3 Matlab scripts that compute the scattered field of a cylinder due to a point source. The scripts compare the analytically computed scattered field of a cylinder with the finite difference solution for one selected frequency. The cylinder has a radius of 40 meter and its center is the origin of the coordinate system. The source is placed at position $z = -100, x = 0$ [m] and the receiver-array is placed at level $z = -60, x = -100 : 100$ [m]. The analytical solution of the cylinder problem is explained in detail in van den Berg (2017) and the Matlab scripts have kindly been provided by Peter van den Berg.

The results in Figure 26 show that the finite difference solution of the scattered field has a maximum error of 5%. This error will become smaller when a smaller discretisation step is chosen.

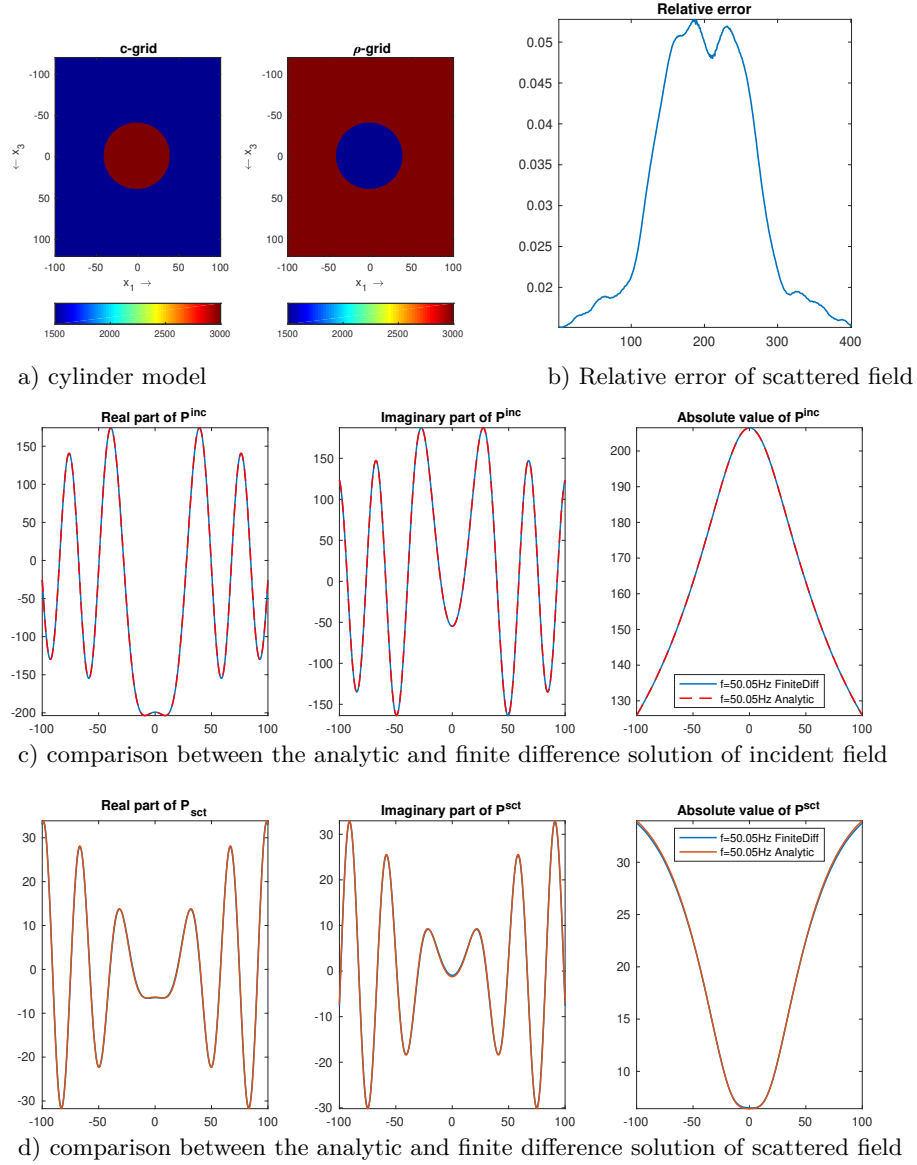


Figure 26: Comparison for a frequency of 50.0488 Hz between an analytical computed field and the finite-difference solution in an acoustic medium with a circular contrast. The finite difference code is modeled with a gridsize of 0.5 m.

The following explanation and theoretical derivation is an excerpt from the book van den Berg (2017).

In order to have a necessary check on the correctness of the coding of the domain integral equations at hand, we present the analytical expressions for some simple configurations. We first start with the one-dimensional scattering by a homogenous slab. Secondly, we discuss the two-dimensional scattering by a homogeneous circular cylinder, where we consider the special case of no contrast in wave speed.

The theoretical expressions are derived in the following subsections. Note that

$$\hat{\gamma}_0 = s/c_0, \quad \hat{\gamma}_{sct} = s/\hat{c}_{sct} \quad (161)$$

where $s = 10^{-16} - i2\pi f$, where f denotes the frequency. Note that $\hat{Q} = \hat{Q}(s)$ is the wavelet in the frequency domain.

7.7.1 1D scattering by a slab

In order to generate synthetic data for an one-dimensional configuration, we consider the scattering problem by a homogeneous slab (Fig. 27). The slab has a thickness of $2a$. The medium inside the slab is characterized by the constant mass density $\hat{\rho}_{sct}$ and constant compressibility $\hat{\kappa}_{sct}$. Then, the wave speed $\hat{c}_{sct}$ is constant as well. The center of the slab is at $x_1 = 0$. The pressure of the acoustic wave field is generated by a planar source at x_1^S and is given by

$$\hat{p}^{inc}(x_1|x_1^S) = \frac{\hat{Q}}{2} \exp[-\hat{\gamma}_0|x_1 - x_1^S|] . \quad (162)$$

The amplitude of the pressure of the incident field at $x_1 = -a$ is given by

$$\hat{p}^{inc}(-a|x_1^S) = \frac{\hat{Q}}{2} \exp[\hat{\gamma}_0(x_1^S + a)] . \quad (163)$$

This is a common factor in all wave field quantities.

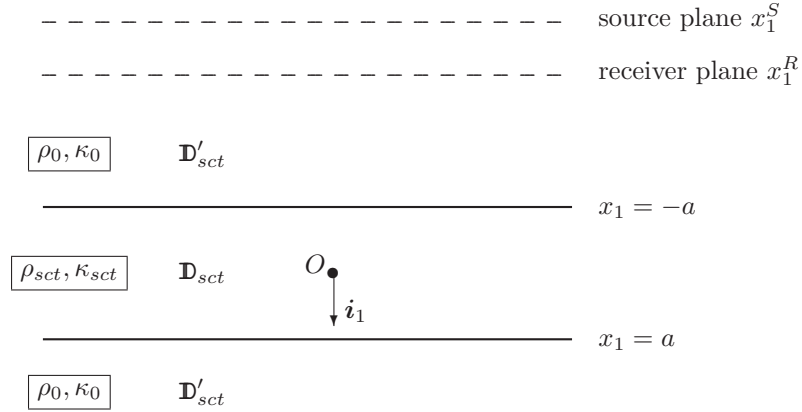


Figure 27: The 1D configuration with homogeneous slab.

The different wave constituents are considered to be generated at the interfaces. Taking into account the causality condition for the reflected field, we write the reflected field in the negative x_1 -direction as

$$\hat{p}^{rfl}(x_1|x_1^S) = \frac{\hat{Q}}{2} \exp[\hat{\gamma}_0(x_1^S + a)] \hat{R} \exp[\hat{\gamma}_0(x_1 + a)], \quad x_1 < -a . \quad (164)$$

The wave field inside the the slab consists of two waves propagating in opposite directions, hence, we write the pressure of this interior wave field as

$$\hat{p}^{int}(x_1|x_1^S) = \frac{\hat{Q}}{2} \exp[\hat{\gamma}_0(x_1^S + a)] \left\{ \hat{A} \exp[-\hat{\gamma}_{sct}(x_1 + a)] + \hat{B} \exp[\hat{\gamma}_{sct}(x_1 - a)] \right\} , \quad (165)$$

when $-a < x_1 < a$. For $x_1 > a$, the field transmitted in the positive x_1 -direction is given by

$$\hat{p}^{trm}(x_1|x_1^S) = \frac{\hat{Q}}{2} \exp[\hat{\gamma}_0(x_1^S + a)] \hat{T} \exp[-\hat{\gamma}_0(x_1 + a)] . \quad (166)$$

The unknown factors $\hat{R}$, $\hat{A}$, $\hat{B}$ and $\hat{T}$ follow interface conditions at $r = -a$ and $r = a$, where the pressure is continuous and the inverse of the mass density times the normal derivative of

the pressure is continuous. This leads to

$$\lim_{x_1 \uparrow -a} [\hat{p}^{inc}(x_1|x_1^S) + \hat{p}^{rfl}(x_1|x_1^S)] = \lim_{x_1 \downarrow -a} \hat{p}^{int}(x_1|x_1^S), \quad (167)$$

$$\lim_{x_1 \uparrow -a} \frac{1}{s\rho_0} \partial_1 [\hat{p}^{inc}(x_1|x_1^S) + \hat{p}_k^{rfl}(x_1|x_1^S)] = \lim_{x_1 \downarrow -a} \frac{1}{s\hat{\rho}_{sct}} \partial_1 \hat{p}^{int}(x_1|x_1^S), \quad (168)$$

$$\lim_{x_1 \uparrow a} \hat{p}^{int}(x_1|x_1^S) = \lim_{x_1 \downarrow a} \hat{p}^{trm}(x_1|x_1^S), \quad (169)$$

$$\lim_{x_1 \uparrow a} \frac{1}{s\hat{\rho}_{sct}} \partial_1 \hat{p}^{int}(x_1|x_1^S) = \lim_{x_1 \downarrow a} \frac{1}{s\rho_0} \partial_1 \hat{p}^{trm}(x_1|x_1^S). \quad (170)$$

Substitution of the wave-field expressions into these interface conditions leads to the following set of equations:

$$1 + \hat{R} = \hat{A} + \exp(-2\hat{\gamma}_{sct}a) \hat{B}, \quad (171)$$

$$-\frac{1}{Z_0} + \frac{1}{Z_0} R = -\frac{1}{\hat{Z}_{sct}} \hat{A} + \frac{1}{\hat{Z}_{sct}} \exp(-2\hat{\gamma}_{sct}a) \hat{B}, \quad (172)$$

$$\exp(-2\hat{\gamma}_{sct}a) \hat{A} + \hat{B} = \hat{T} \exp(-2\hat{\gamma}_0a), \quad (173)$$

$$-\frac{1}{\hat{Z}_{sct}} \exp(-2\hat{\gamma}_{sct}a) \hat{A} + \frac{1}{\hat{Z}_{sct}} \hat{B} = -\frac{1}{Z_0} \hat{T} \exp(-2\hat{\gamma}_0a), \quad (174)$$

where

$$Z_0 = \rho_0 c_0, \quad \text{and} \quad \hat{Z}_{sct} = \hat{\rho}_{sct} \hat{c}_{sct}. \quad (175)$$

Elimination of $\hat{R}$ from the first and the second equation and $\hat{T}$ from the third and the fourth equation leads to two equations for $\hat{A}$ and $\hat{B}$, with solution

$$\hat{A} = \frac{\hat{\tau}}{1 - \hat{\rho}^2 \exp(-4\hat{\gamma}_{sct}a)}, \quad \hat{B} = \frac{-\hat{\tau} \hat{\rho} \exp(-2\hat{\gamma}_{sct}a)}{1 - \hat{\rho}^2 \exp(-4\hat{\gamma}_{sct}a)}, \quad (176)$$

where

$$\hat{\rho} = \frac{Z_0^{-1} - \hat{Z}_{sct}^{-1}}{Z_0^{-1} + \hat{Z}_{sct}^{-1}}, \quad \hat{\tau} = \frac{2Z_0^{-1}}{Z_0^{-1} + \hat{Z}_{sct}^{-1}}, \quad (177)$$

are the local reflection and transmission factors. Subsequently, the global reflection and transmission factors are obtained as

$$\hat{R} = \hat{A} + \exp(-2\hat{\gamma}_{sct}a) \hat{B} - 1, \quad (178)$$

$$\hat{T} = \left[\exp(-2\hat{\gamma}_{sct}a) \hat{A} + \hat{B} \right] \exp(2\hat{\gamma}_0a). \quad (179)$$

Note that, in case the impedance of the medium in the vertical directions does not change, i.e., $\hat{Z}_{sct} = Z_0$, we have $\hat{\rho} = 0$, $\hat{\tau} = 1$, $\hat{A} = 1$, $\hat{B} = 0$ and $\hat{R} = 0$. Then, the slab is reflection free and is invisible for acoustic waves.

As far as the scattered field is concerned, we distinguish between a point of observation above the slab and below the slab. When we have a receiver at the point x_1^R above the slab, the scattered wave is denoted as the reflected wave and is given by

$$\hat{p}^{rfl}(x_1^R|x_1^S) = \frac{\hat{Q}}{2} \hat{R} \exp[\hat{\gamma}_0(x_1^R + x_1^S + 2a)], \quad x_1^R < -a. \quad (180)$$

To arrive at the scattered field below the slab, we have to subtract the incident field from the transmitted field. We obtain

$$\hat{p}^{sct}(x_1^R|x_1^S) - \hat{u}^{inc}(x_1^R|x_1^S) = \frac{\hat{Q}}{2} (\hat{T} - 1) \exp[-\hat{\gamma}_0(x_1^R - x_1^S)], \quad x_1^R > a. \quad (181)$$

7.7.2 2D scattering by a circular cylinder

In order to generate synthetic data for a two-dimensional configuration, we consider the scattering problem by a homogeneous, circular cylinder (Fig. 28). We define the spatial position by $\mathbf{x}_T = (x_1, x_2)$. The cylinder has a radius a . The medium in the interior of the cylinder is characterized by the constant wave speed $\hat{c}_{sct}$ and constant mass density $\hat{\rho}_{sct}$. The center of the cylinder is at $x_1 = 0, x_2 = 0$. The pressure of the incident wave field is generated by a monopole line source at $\mathbf{x}_T^S = (x_1^S, x_2^S)$ and is given by

$$\hat{p}^{inc}(\mathbf{x}_T | \mathbf{x}_T^S) = \hat{\gamma}_0 \frac{\hat{Q}}{2\pi} K_0(\hat{\gamma}_0 |\mathbf{x}_T - \mathbf{x}_T^S|). \quad (182)$$

In order to solve our scattering problem at hand, we introduce polar coordinates adapted to the geometry of the circular cylinder,

$$x_1 = r \cos(\phi), \quad x_2 = r \sin(\phi), \quad 0 \leq \phi < 2\pi. \quad (183)$$

Similarly for the source and receiver coordinates, we introduce

$$x_1^S = r^S \cos(\phi^S), \quad x_2^S = r^S \sin(\phi^S), \quad 0 \leq \phi^S < 2\pi, \quad (184)$$

$$x_1^R = r^R \cos(\phi^R), \quad x_2^R = r^R \sin(\phi^R), \quad 0 \leq \phi^R < 2\pi. \quad (185)$$

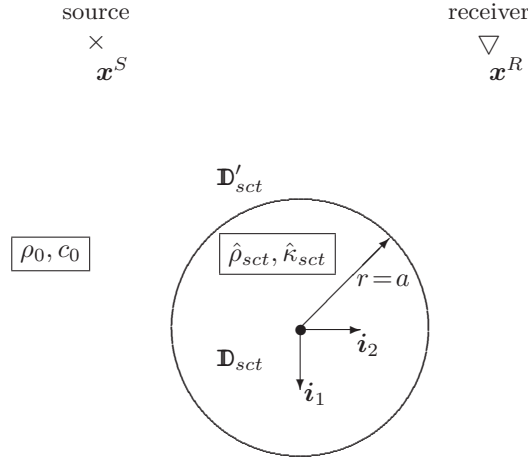


Figure 28: The 2D configuration with homogeneous circular cylinder.

7.7.3 3D scattering by a sphere

In order to generate synthetic data for a 3D scatterer, we consider the scattering problem by a homogeneous, sphere (Fig. 29). The sphere has a radius a . The medium in the interior of the sphere is characterized by the constant wave speed $\hat{c}_{sct}$ and the constant mass density $\hat{\rho}_{sct}$. The center of the sphere is at $\mathbf{x} = (0, 0, 0)$. The incident wave field is generated by a monopole source at $\mathbf{x}^S = (x_1^S, x_2^S, x_3^S)$ and is given by

$$\hat{p}^{inc}(\mathbf{x} | \mathbf{x}^S) = \hat{\gamma}_0 \hat{Q} \frac{\exp(-\hat{\gamma}_0 |\mathbf{x} - \mathbf{x}^S|)}{4\pi |\mathbf{x} - \mathbf{x}^S|}. \quad (186)$$

In order to solve our scattering problem at hand, we introduce spherical coordinates adapted to the geometry of the sphere,

$$x_1 = r \sin(\theta) \cos(\phi), \quad x_2 = r \sin(\theta) \sin(\phi), \quad x_3 = r \cos(\theta), \quad (187)$$

with $0 \leq \phi < 2\pi$ and $0 \leq \theta < \pi$. Similarly for the source and receiver coordinates, we introduce

$$x_1^S = r^S \sin(\theta^S) \cos(\phi^S), \quad x_2^S = r^S \sin(\theta^S) \sin(\phi^S), \quad x_3^S = r^S \cos(\theta^S), \quad (188)$$

$$x_1^R = r^R \sin(\theta^R) \cos(\phi^R), \quad x_2^R = r^R \sin(\theta^R) \sin(\phi^R), \quad x_3^R = r^R \cos(\theta^R). \quad (189)$$

Similarly as in Chapter 1, for $r \leq r^S$, the incident acoustic pressure is represented as

$$\hat{p}^{inc}(\mathbf{x}|\mathbf{x}^S) = \frac{\hat{\gamma}_0^2 \hat{Q}}{2\pi^2} \sum_{n=0}^{\infty} (2n+1) i_n^{(1)}(\hat{\gamma}_0 r) k_n(\hat{\gamma}_0 r^S) P_n[\cos(\mathbf{x}, \mathbf{x}^S)], \quad (190)$$

Here, $i_n^{(1)}(\cdot)$ and $k_n(\cdot)$ are the n^{th} order modified spherical Bessel functions of the first kind and second kind, respectively.

Taking into account the causality condition for the reflected wave field, we write the reflected wave field outside the scattering sphere as

$$\hat{p}^{rfl}(\mathbf{x}|\mathbf{x}^S) = \frac{\hat{\gamma}_0^2 \hat{Q}}{2\pi^2} \sum_{n=0}^{\infty} (2n+1) \hat{A}_n k_n(\hat{\gamma}_0 r) k_n(\hat{\gamma}_0 r^S) P_n[\cos(\mathbf{x}, \mathbf{x}^S)]. \quad (191)$$

The wave field inside the sphere has to be bounded at $r = 0$, hence, we write this interior wave field as

$$\hat{p}^{int}(\mathbf{x}|\mathbf{x}^S) = \frac{\hat{\gamma}_0^2 \hat{Q}}{2\pi^2} \sum_{n=0}^{\infty} (2n+1) \hat{B}_n i_n^{(1)}(\hat{\gamma}_{sct} r) k_n(\hat{\gamma}_0 r^S) P_n[\cos(\mathbf{x}, \mathbf{x}^S)]. \quad (192)$$

The unknown expansion factors $\hat{A}_n$ and $\hat{B}_n$ follow from the interface conditions at $r = a$, where the wave field and its radial derivative are continuous. This leads to

$$\lim_{r \downarrow a} [\hat{p}^{inc}(\mathbf{x}|\mathbf{x}^S) + \hat{p}^{rfl}(\mathbf{x}|\mathbf{x}^S)] = \lim_{r \uparrow a} \hat{p}^{int}(\mathbf{x}|\mathbf{x}^S), \quad (193)$$

$$\lim_{r \downarrow a} \partial_r [\hat{p}^{inc}(\mathbf{x}|\mathbf{x}^S) + \hat{p}^{rfl}(\mathbf{x}|\mathbf{x}^S)] = \lim_{r \uparrow a} \partial_r \hat{p}^{int}(\mathbf{x}|\mathbf{x}^S). \quad (194)$$

Substitution of the wave-field expressions into these interface conditions leads, for each n , to the following set of equations:

$$i_n^{(1)}(\hat{\gamma}_0 a) + \hat{A}_n k_n(\hat{\gamma}_0 a) = \hat{B}_n i_n^{(1)}(\hat{\gamma}_{sct} a), \quad (195)$$

$$\frac{1}{s\rho_0} \left[\partial_r i_n^{(1)}(\hat{\gamma}_0 r) + \hat{A}_n \partial_r k_n(\hat{\gamma}_0 r) \right]_{r=a} = \hat{B}_n \frac{1}{s\hat{\rho}_{sct}} \partial_r i_n^{(1)}(\hat{\gamma}_{sct} r) \Big|_{r=a}. \quad (196)$$

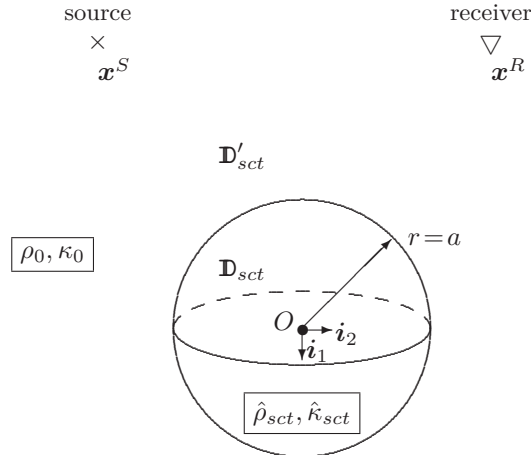


Figure 29: The 3D configuration with homogeneous sphere.

Solving these equations for the coefficients A_n and B_n leads to

$$\hat{A}_n = -\frac{\hat{Z}_{sct}^{-1} \partial i_n^{(1)}(\hat{\gamma}_{sct} a) i_n^{(1)}(\hat{\gamma}_0 a) - Z_0^{-1} \partial i_n^{(1)}(\hat{\gamma}_0 a) i_n^{(1)}(\hat{\gamma}_{sct} a)}{\hat{Z}_{sct}^{-1} \partial i_n^{(1)}(\hat{\gamma}_{sct} a) k_n(\hat{\gamma}_0 a) - Z_0^{-1} \partial k_n(\hat{\gamma}_0 a) i_n^{(1)}(\hat{\gamma}_{sct} a)} \quad (197)$$

and

$$\hat{B}_n = \frac{i_n^{(1)}(\hat{\gamma}_0 a) + \hat{A}_n k_n(\hat{\gamma}_0 a)}{i_n^{(1)}(\hat{\gamma}_{sct} a)}, \quad (198)$$

where $\partial i_n^{(1)}(\cdot)$ and $\partial k_n(\cdot)$ denote the derivatives of the functions $i_n^{(1)}(\cdot)$ and $k_n(\cdot)$ with respect to their arguments.

Substituting the reflection factors $\hat{A}_n$ in the expression for the reflected wave field of Eq. (191) and taking $\mathbf{x} = \mathbf{x}^R$, we obtain

$$\hat{p}^{rf}(\mathbf{x}^R | \mathbf{x}^S) = \frac{\hat{\gamma}_0^2 \hat{Q}}{2\pi^2} \sum_{n=0}^{\infty} (2n+1) \hat{A}_n k_n(\hat{\gamma}_0 r^R) k_n(\hat{\gamma}_0 r^S) P_n[\cos(\mathbf{x}^R, \mathbf{x}^S)]. \quad (199)$$

With this expression we are able to compute the synthetic data pertaining to the present example of a domain scatterer. For the computation of the Bessel functions and the Legendre polynomials we refer to the pertaining section of Chapter 1.

Similar as in Chapter 2, for $r \leq r^S$, we write the pressure of the incident wave field as the infinite series

$$\hat{p}^{inc}(\mathbf{x}_T | \mathbf{x}_T^S) = \hat{\gamma}_0 \frac{\hat{Q}}{2\pi} \sum_{m=0}^{\infty} \epsilon_m I_m(\hat{\gamma}_0 r) K_m(\hat{\gamma}_0 r^S) \cos[m(\phi - \phi^S)], \quad (200)$$

where $\epsilon_m = 1$ if $m = 0$ and $\epsilon_m = 2$ if $m \neq 0$. Here, I_m and K_m are the modified Bessel functions of the first and second kind, and integer order m .

Taking into account the causality condition for the reflected wave field, we write the reflected wave field outside the scattering cylinder as

$$\hat{p}^{rf}(\mathbf{x}_T | \mathbf{x}_T^S) = \hat{\gamma}_0 \frac{\hat{Q}}{2\pi} \sum_{m=0}^{\infty} \epsilon_m \hat{A}_m K_m(\hat{\gamma}_0 r) K_m(\hat{\gamma}_0 r^S) \cos[m(\phi - \phi^S)]. \quad (201)$$

The wave field inside the cylinder has to be bounded at $r = 0$, hence, we write this interior wave field as

$$\hat{p}^{int}(\mathbf{x}_T | \mathbf{x}_T^S) = \hat{\gamma}_0 \frac{\hat{Q}}{2\pi} \sum_{m=0}^{\infty} \epsilon_m \hat{B}_m I_m(\hat{\gamma}_{sct} r) K_m(\hat{\gamma}_0 r^S) \cos[m(\phi - \phi^S)]. \quad (202)$$

The unknown expansion factors $\hat{A}_m$ and $\hat{B}_m$ follow from the interface conditions at $r = a$, where the pressure is continuous and the inverse of the mass density times the radial derivative of the pressure is continuous. This leads to

$$\lim_{r \downarrow a} [\hat{p}^{inc}(\mathbf{x}_T | \mathbf{x}_T^S) + \hat{p}^{rf}(\mathbf{x}_T | \mathbf{x}_T^S)] = \lim_{r \uparrow a} \hat{p}^{int}(\mathbf{x}_T | \mathbf{x}_T^S), \quad (203)$$

$$\frac{1}{s\rho_0} \lim_{r \downarrow a} \partial_r [\hat{p}^{inc}(\mathbf{x}_T | \mathbf{x}_T^S) + \hat{p}^{rf}(\mathbf{x}_T | \mathbf{x}_T^S)] = \frac{1}{s\hat{\rho}_{sct}} \lim_{r \uparrow a} \partial_r \hat{p}^{int}(\mathbf{x}_T | \mathbf{x}_T^S). \quad (204)$$

Substitution of the wave-field expressions into these interface conditions leads, for each m to the following set of equations:

$$I_m(\hat{\gamma}_0 a) + \hat{A}_m K_m(\hat{\gamma}_0 a) = \hat{B}_m I_m(\hat{\gamma}_{sct} a), \quad (205)$$

$$\frac{1}{s\rho_0} \left[\partial_r I_m(\hat{\gamma}_0 r) + \hat{A}_m \partial_r K_m(\hat{\gamma}_0 r) \right]_{r=a} = \frac{1}{s\hat{\rho}_{sct}} \hat{B}_m \partial_r I_m(\hat{\gamma}_{sct} r) \Big|_{r=a}. \quad (206)$$

Solving these equations for the coefficients A_m and B_m leads to

$$\hat{A}_m = -\frac{\hat{Z}_{sct}^{-1} \partial I_m(\hat{\gamma}_{sct} a) I_m(\hat{\gamma}_0 a) - Z_0^{-1} \partial I_m(\hat{\gamma}_0 a) I_m(\hat{\gamma}_{sct} a)}{\hat{Z}_{sct}^{-1} \partial I_m(\hat{\gamma}_{sct} a) K_m(\hat{\gamma}_0 a) - Z_0^{-1} \partial K_m(\hat{\gamma}_0 a) I_m(\hat{\gamma}_{sct} a)} \quad (207)$$

and

$$\hat{B}_m = \left(I_m(\hat{\gamma}_0 a) + \hat{A}_m K_m(\hat{\gamma}_0 a) \right) / I_m(\hat{\gamma}_{sct} a), \quad (208)$$

where $\partial I_m(\cdot)$ and $\partial K_m(\cdot)$ denote the derivatives of the functions I_m and K_m with respect to their arguments. Substituting the factors $\hat{A}_m$ in the expression for the reflected wave field of Eq. (201) and taking $\mathbf{x}_T = \mathbf{x}_T^R$, we obtain

$$\hat{p}^{rfl}(\mathbf{x}_T^R | \mathbf{x}_T^S) = \hat{\gamma}_0 \frac{\hat{Q}}{2\pi} \sum_{m=0}^{\infty} \epsilon_m \hat{A}_m K_m(\hat{\gamma}_0 r^R) K_m(\hat{\gamma}_0 r^S) \cos[m(\phi^R - \phi^S)]. \quad (209)$$

With this expression we are able to compute the synthetic data pertaining to the present example of a domain scatterer.

End of excerpt from the book van den Berg (2017).

A Source and directory structure

```

/
├─ Make_include .....File with system specific setting and can be adapted for specific Unix systems
├─ Makefile .....Controls the compilation and linking of the programs in the subdirectories
├─ README
├─ SU_LEGAL_STATEMENT.txt
├─ bin .....Directory for the binaries compiled and linked using the Makefile
├─ FFTlib .....Library for FFT transformation routines used by the programs
├─ fdelmodc ..... This directory contains all source code for the program fdelmodc
│   └─ FiguresPaper .....The bash-script to generate the Figures from in Geophysics manuscript
│   └─ demo ..... Bash-script which demonstrate the possibilities of fdelmodc
├─ doc .....where you can find this manual
├─ include .....Directory for the include file from the FFT library
├─ lib .....Directory where the FFT library is placed
├─ utils .....All source code for programs to generate models and wavelets can be found in here
└─ /
    └─ bin/
        ├── basop ..... Executable for basic operations (shift, envelope, ..) on seismic data
        ├── extendModel Executable to extends the edges of a file with first and last trace and/or sample
        ├── fconv .....Executable for auto-, cross-correlation, deconvolution or convolution computation
        ├── fdelmodc ..... Executable for elastic acoustic finite-difference wavefield modeling
        ├── green ..... Executable for the calculation of 2D Greens function in homogeneous media
        ├── makemod .....Executable for building gridded subsurface models
        └─ makewave .....Executable to generate wavelets
    └─ /include
        └─ genfft.h .....Include file for the FFT library
    └─ /lib
        └─ libgenfft.a .....Library which contains the objects of the FFT routines
└─ /
    └─ fdelmodc/
        ├── Makefile ..... controls the compilation and linking of the program fdelmodc
        ├── fdelmodc.h ..... header file which defines structures used modeling
        ├── par.h .....header file from SU for reading in program parameters
        ├── SUsegy.h ..... adjusted segy header file, which defines ns as an integer
        ├── segy.h ..... original segy header from SU
        ├── acoustic2.c ..... Kernel of acoustic FD using 2'nd order derivatives
        ├── acoustic4.c ..... Kernel of acoustic FD using 4'th order derivatives
        ├── acoustic6.c ..... Kernel of acoustic FD using 6'th order derivatives
        ├── applySource.c .....Routine which adds source amplitude(s) to the wavefield grids
        ├── atopkge.c ..... converts ascii to arithmetic from SU
        └─ CMWC4096.c ..... random number generator

```

```

defineSource.c ..... computes, or read from file, the source signature
docpkge.c .....function for self-documentation, from SU
elastic4.c ..... Kernel of elastic FD using 4'th order derivatives
fdelmodc.c .....main FD modeling program, contains self-doc
fileOpen.c .....file handling routines to open SU files
gaussGen.c .....generate a Gaussian distribution of random numbers
getBeamTimes.c .....stores energy fields (beams) in arrays at certain time steps
getModelInfo.c .....reads gridded model file to compute min/max and sampling intervals
getParameters.c .....reads in all parameters to set up a FD modeling
getRecTimes.c .....stores the wavefield at the receiver positions
getWaveletInfo.c reads source wavelet file and computes maximum frequency and sampling
getpars.c .....functions to get parameters from the command line, from SU
name_ext.c .....inserts a character string after the filename, before the extension
readModel.c ....reads gridded model files and computes medium parameters used in the FD
kernels
recvPar.c .....calculates the receiver positions based on the input parameters
spline3.c .....computes interpolation based on third order splines
taperEdges.c .....tapers the wavefield to suppress unwanted reflections from the edges
verbosepkg.c .....functions to print out verbose, error and warning messages to stderr
viscoacoustic4.c .....Kernel of visco-acoustic FD using 4'th order derivatives
viscoelastic4.c .....Kernel of visco-elastic FD using 4'th order derivatives
wallclock_time.c .....function used to calculate wallclock time
writeRec.c .....writes the receiver array(s) to output file(s)
writeSnapTimes.c .....writes gridded wavefield(s) at a desired time to output file(s)
writeSrcRecPos.c .....writes the source and receiver positions into a gridded file
writesufile.c .....writes an 2D array to a SU file

/
fdelmodc/
├── FiguresPaper/ ..... scripts to reproduce Figures in Thorbecke and Draganov (2011)
│   ├── README ..... briefly describes the runtimes of the scripts
│   ├── clean ..... removes all *.su *.bin *.txt *.eps in the current directory
│   ├── Figure2.scr ..... starts fdelmodc only to compute the source positions, 1 s.
│   ├── Figure3.scr ..... calls Simple_model_base, and Simple_model_sides.scr, 122 hours!
│   ├── Figure3_ref.scr ..... direct modeled reference result Figure 3d, 500 s.
│   ├── Figure4.scr ..... 5 different source signature lengths, 5x3.5 hours
│   ├── Figure5.scr ..... simulates 8000 short (2.5 s) sources, 3.5 hours
│   ├── Figure6.scr ..... 5 different number of random sources, 5x3.5 hours
│   ├── Figure6f.scr ..... make postscript file of middle trace after Figure6.scr, 1 s.
│   ├── Figure6length.scr ... alternative not used in paper, fixed source signature length, 5x3.5
│   |   hours
│   ├── Figure7.scr ..... as Figure 6, but with 1000 deep sources, 3.5 hours
│   ├── Figure7fmax.scr alternative not used in paper, varying maximum frequency, 5x3.5 hours
│   ├── Figure7length.scr ... alternative not used in paper, fixed length deep sources, 3.5 hours
│   ├── Figure8-9.scr .... for random and ricker wavelet deep, volume and plane sources, 6x3.5
│   |   hours
│   ├── Figure8-9Hom.scr .. for reviewer, same as Fig. 8-9 in homogeneous medium, 6x3.5 hours
│   ├── Figure10.scr .. reference and 2 SI results for visco-acoustic media, 2x200 s. + 2x1 hours
│   ├── Figure11.scr .... calls fdelmodc_long.scr, can not be reproduced; software in test phase
│   ├── Figure12.scr ..... calls fdelmodc_amplitude.scr, can not be reproduced; software in test
│   |   phase
│   ├── Figure13.scr ..... amplitude variations on source strength, 3x1500 s.
│   ├── Figure13Amp.scr ..... computes only the amplitude distributions pictures, 5 s.
│   ├── Figure14-15.scr ..... receivers and source placed on model with topography, 161 hours
│   ├── FigureSourcesAppendixA.scr .... source construction shown in Figure A2-A3-A4, 150 s.
│   ├── FigureGreenAppendixA.scr compares FD result with analytical result, used in Figure 23
│   └── FigureGreenDxAppendixA.scr ..... difference with analytical result, used in Figure 24

```

```

└─ SIrand.scr  middle trace is correlated with all the output traces to compute the SI result
└─ Simple_model_base.scr  .... models sequential 900 shots at level  $z = 3600$ , 70 hours
└─ Simple_model_sides.scr  .... models sequential 2x360 shots at the sides  $x=1000, 9000$ , 50
    hours
└─ fdelmodc_amplitude.scr  ... models along recording on 3600 s. used in Fig 12, 100 hours
└─ fdelmodc_long.scr  .... models along recording on 3600 s. used in Fig 11, 100 hours
└─ FigurePres.scr  .... snapshots for movie usage in presentation to explain SI, 2x800 s.
└─ MakeGifMovie.scr  .... attempt to make movie from FigurePres.scr snapshots, imageJ is
    better
└─ cross.scr  .... calls FigureCCsources.scr and compute cross-correlation used in Figure 12,
    1600 s.
└─ FigureCCsources.scr  .... to compute source signature used in cross.scr, 1600 s.

/
└─ fdelmodc/
    └─ demo/
        └─ clean  .... removes all *.su *.bin *.txt *.eps in the current directory
        └─ eps_for_manual.scr  .... the results of fdelmodc_rand.scr in eps, used in Figure 8, 9, 10
        └─ fdelmodc_rand.scr  .. generation of random source signatures placed at random positions
        └─ fdelmodc_srcrec.scr  .... illustrates source and receiver positions, used in Figure 14
        └─ fdelmodc_taper.scr  .... the effect of (absorbing) tapering of the edges, used in Figure 6
        └─ fdelmodc_visco.scr  .... wave propagation in visco-elastic medium, used in Figure 19
        └─ fdelmodc_circ.scr  .... receivers placed on a circle, used in Figure 21
        └─ fdelmodc_sourcepos.scr  .... different source distributions, used in Figure 20
        └─ fdelmodc_plane.scr  plane wave at depth to receivers at the surface, including snapshots,
            17
        └─ fdelmodc_stab.scr  ... illustrates dispersion and instability in snapshots, used in Figure 3
        └─ fdelmodc_topography.scr  .... source and receivers on topography, used in Figure 22
        └─ fdelmodc_obc.scr  .... same as fdelmodc_topography, but receivers on topography of
            sea-bottom
        └─ model_flank.scr  .... builds a steep flank model, used in fdelmodc_srcrec.scr

/
└─ utils/
    └─ Makefile  .... to compile and link the code
    └─ par.h  .... header file from SU for reading in program parameters
    └─ segy.h  .... original segy header from SU
    └─ allocs.c  .... allocate 2D arrays as pointer list
    └─ atopkge.c  .... converts ascii to arithmetic from SU
    └─ basop.c  .... main program for basic operations on seismic data
    └─ diffraction.c  .... insert diffractor in the model used, in makemod
    └─ docpkge.c  .... function for self-documentation, from SU
    └─ ellipse.c  .... ellipse shaped contrast used in makemod
    └─ extendModel.c  .... main program to extend the edges of a gridded model
    └─ fconv.c  main program for auto-, cross-correlation, deconvolution or convolution computation
    └─ fractint.c  .... compute fractal shaped interface used in makemod
    └─ freqwave.c  .... compute wavelets in frequency domain, used in makewave
    └─ getFileInfo.c  .... gets sizes, sampling and min/max values of a SU file
    └─ getModelInfo.c  .... reads gridded model file to compute min/max and sampling intervals
    └─ getpars.c  .... functions to get parameters from the command line, from SU
    └─ getrecpos.c  .... read receiver positions used in green
    └─ green.c  .... main program for calculation of (exact) 2D Greens function in hom. medium
    └─ grid.c  .... fills the gridded model below the interface  $z_p$  used in makemod
    └─ gridabove.c  .... fills the gridded model above the interface  $z_p$  used in makemod
    └─ interpolation.c  interpolates the interface defined by the input parameters to all grid points
    └─ linearint.c  .... compute piecewise linear interface used in makemod
    └─ makemod.c  .... main program of gridded subsurface model builder
    └─ makewave.c  .... main program for the generation of wavelets

```

```

└ name_ext.c .....inserts a character string after the filename, before the extension
└ plotexample.c .....prints an example parameter file for makemod
└ polint.c ..... compute polynomial shaped interface used in makemod
└ readData.c .....reads SU file and returns header and 2D array
└ roughint.c ..... compute rough shaped interface used in makemod
└ sinusint.c ..... compute sinus shaped interface used in makemod
└ spline.c .....compute spline shaped interface used in makemod
└ verbosepkg.c .....functions to print out verbose, error and warning messages to stderr
└ wallclock_time.c .....function used to calculate wallclock time
└ writeData.c ..... writes an 2D array to a SU file
└ xwgreen.c .. calculation of di/mono-pole response in 2D homogeneous medium used in green

```

B Differences in parameter use compared with DELPHI's fdacmod

fdelmodc uses many similar parameters as the DELPHI application fdacmod. The differences are:

- acoustic modeling → ischeme=1
- file_vel= → file_cp=
- file_att= → file_qp= or Qp=
- dipsrc=0 → src_orient=1
- dipsrc=1 → src_orient=2
- diprcv=0 → rcv_type_p=1
- diprcv=1 → rcv_type_vz=1
- xrcv= → xrcva=
- zrcv= → zrcva=

In fdelmodc there is no dxsrc dzsrc dxspread dzspread tapfact.

C Makewave

Makewave makes different types of waves which can be used as input wavelet for a modelling scheme. Five type of wavelets can be defined, each type has is own set of characteristic parameters.

C.1 Parameters

makewave - generation of wavelets

makewave file= [optional parameters]

Required parameters:

file_out= output array-file (empty is SU pipe)

Optional parameters:

```

nt=256 ..... number of samples
dt=0.004 ..... stepsize in time-direction(s)
fmin=10 ..... minimum frequency in wavelet(Hz)
flef=20 ..... left attenuation point in freq. domain(Hz)

```

```

frig=50 ..... right attenuation point in freq. domain(Hz)
fmax=60 ..... maximum frequency in wavelet(Hz)
db=-20 ..... attenuation at the maximum frequency fm in dB
fp=30 ..... frequency peak in wavelet
t0=0.0 ..... position of peak of wavelet
shift=0 ..... shift wavelet until it's causal (overrides t0)
scale=1 ..... 1: sets value of maximum time-peak to scale
scfft=1 ..... scale factor in fft^-1; 0-> 1/N, 1-> = df
cn=1 ..... cn integer and 1 < cn < 3 (see Neidell)
cm=10 ..... cm integer and 7 < cm < 25 (see Neidell)
w=g2 ..... type of wavelet (g2 gives a Ricker Wavelet)
inverse=0 ..... compute 1.0/(S(w)+eps)
eps=1.0 ..... stabilization in inverse
verbose=0 ..... silent option; >0 display info

```

Options for w :

- g0 = Gaussian wavelet
- g1 = derivative of a Gaussian wavelet
- g2 = second derivative of a Gaussian wavelet(=Ricker)
- sqrtg2 = sqrt of second derivative of a Gaussian wavelet(=Ricker)
- fw = wavelet defined by fmin, flef, frig and fmax
- mon = monochromatic wavelet defined by fp
- cs = suite of wavelets determined by cn and cm
(see Neidell: Geophysics 1991, p.681-690)

The parameters fmax or fp characterizes the wavelet. If both fmax and fp are given fmax is used. For the Gaussian wavelet (w=g0) only the parameter fmax has a meaning (the peak lies always at 0, fp=0). Note that fmin, flef and frig are only used when the option fw is chosen. If scale is chosen to be zero no scaling is done.

```

author   : Jan Thorbecke : 27-09-1993 (janth@xs4all.nl)
product  : Originates from DELPHI software
          : revision 2010

```

The wavelets generated with makewave are

- g0: Gaussian wavelet

$$G_0(f) = \exp\left(-\frac{f^2}{f_p^2}\right) \quad (210)$$

- g1: first derivative of a Gaussian wavelet

$$G_1(f) = \frac{f}{\sqrt{2}f_p} \exp\left(-\frac{f^2}{2f_p^2}\right) \quad (211)$$

- g2: second derivative of a Gaussian wavelet

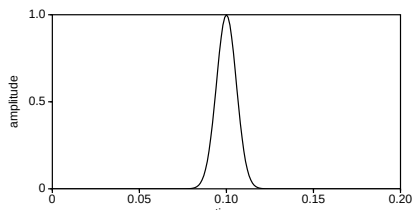
$$G_2(f) = \frac{f^2}{f_p^2} \exp\left(-\frac{f^2}{f_p^2}\right) \quad (212)$$

- fw: flat wavelet spectrum

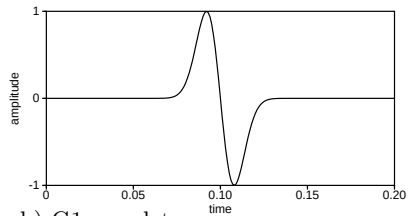
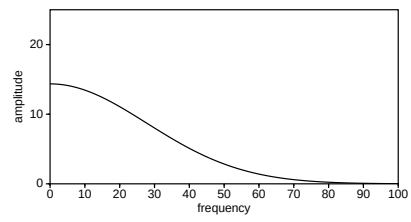
$$FW(f) = \begin{cases} 0 & f < f_{min} \\ \cos\left(\pi * \frac{f-f_{min}}{f_{min}-f_{lef}}\right) + 1.0)/2.0; & f_{min} \leq f < f_{lef} \\ 1 & f_{lef} < f < f_{rig} \\ \cos\left(\pi * \frac{f-f_{rig}}{f_{max}-f_{rig}}\right) + 1.0)/2.0; & f_{rig} \leq f < f_{max} \\ 0 & f > f_{max} \end{cases} \quad (213)$$

where f_p is the peak in the spectrum of the wavelet. If both fmax and fp are given fmax is used. For the Ricker wavelet (w=g2) a frequency peak is most convenient to choose. For the Gaussian wavelet (w=g0) only the parameter fmax has a meaning, because the peak lies always at fp=0. Note that fmin, flef and frig are only used when the option fw is chosen.

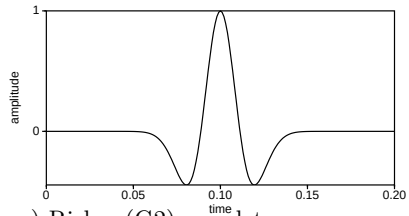
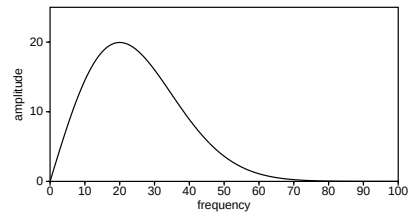
If shift=1 then a time shift is calculated in such a way that the t=0 sample has a value smaller than 1e-3.



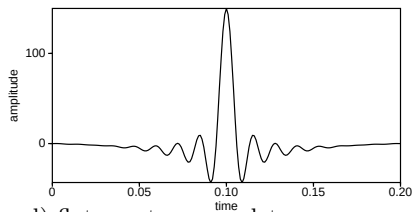
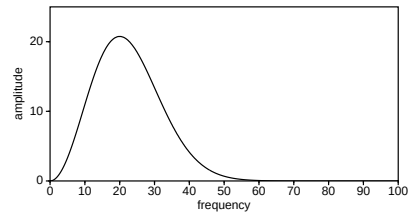
a) G0 wavelet



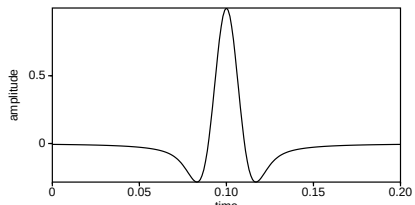
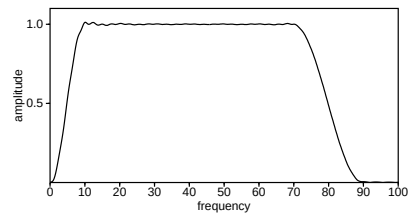
b) G1 wavelet



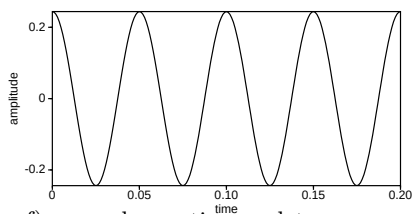
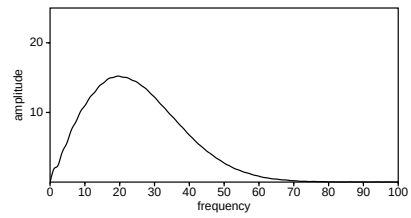
c) Ricker (G2) wavelet



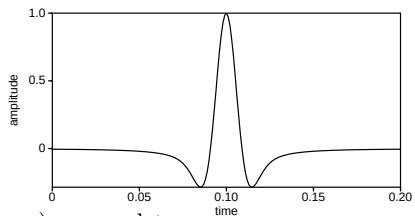
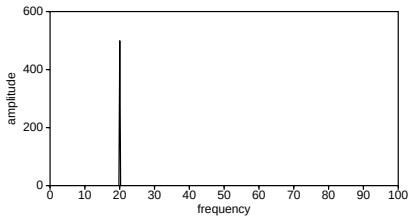
d) flat-spectrum wavelet



e) 'sqrt-of-G2' wavelet



f) monochromatic wavelet



g) cs wavelet

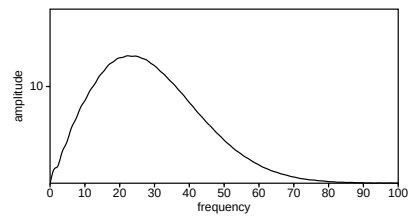


Figure 30: Time and amplitude spectrum of the wavelet types generated by the program `makewave`.

A weak vertical line in the zero-offset receiver recording of V_x and V_z components can be observed if a G_1 wavelet is chosen. This vertical line is caused by the shape of the input wavelet. In fdelmodc the wavelet is integrated over time (see section 6.5 in the manual) and causes a G_1 wavelet to become a wavelet with only positive values. This is not a physical source and the modelling scheme can not completely deal with that correctly. See also "Numerical Techniques for Conservation Laws with Source Terms" by Justin Hudson. To overcome the problems with the G_1 wavelet you can set `src_injectionrate=1`.

C.2 Examples

A simple example of a Ricker wavelet (g2) with a maximum frequency of 60 Hz (-20dB attenuation), a time sampling rate of 0.004 seconds and a time shift calculated by the program is obtained by typing:

```
makewave shift=1 verbose=1 | suxgraph
makewave shift=1 verbose=1 | sufft | suamp | suxgraph
```

The script in `utils/demo/wavelet.scr` reproduces the wavelets in Figure 30.

D Makemod

This program generates 2D gridded velocity and density files based on user defined interfaces. The interfaces are defined by several (x, z) points. Through these points a linear, a polynomial or a splined curve is calculated in order to define the whole range of x positions between the minimum and maximum defined x positions. A sinusoidal or rough interface can be superpositioned on this defined interface. The sinusoidal and rough interface variations are defined by some extra parameters.

The subsurface files makemod generates can be used in a migration or finite difference program. As a result three (or two for the acoustic case) SU files are produced with extensions `_cp.su`, `_cs.su` and `_ro.su`.

D.1 Parameters

`makemod` - gridded subsurface model builder

`makemod file_base= cp0= sizex= sizez= dx= dz= [optional parameters]`

Required parameters:

```
file_base= ..... base name of the output file(s).
cp0= ..... Cp for background medium.
sizex= ..... x-size of the model in meters.
sizez= ..... z-size of the model in meters.
dx= ..... grid distance in x in meters.
dz= ..... grid distance in z in meters.
```

Optional parameters:

```
orig=0,0 (x,z) ..... origin at the top left corner of the model.
MEDIUM
cs0=0 ..... Cs for background medium (0 is none).
ro0=0 ..... Rho for background medium (0 is none).
gradt=1 ..... type of boundary gradient (1=linear, 2=cos)
cp=none ..... P-wave velocities below the interface
cs=none ..... S-wave velocities below the interface
ro=none ..... Density below the interface
above=0 ..... define model below interface
          =1: define model above interface
INTERFACE
intt=none ..... Type of interface
```

```

var=none ..... variables to describe the interface
grad=0.0 ..... gradient(m) of the boundary
gradunit=0 ..... gradient unit (m/s per gradunit)
gradcp=0.0 ..... gradient(m/s per grad-unit) in the layer
gradcs=0.0 ..... gradient(m/s per grad-unit) in the layer
gradro=0.0 ..... gradient(kg/m3 per grad-unit) in the layer
poly=0 ..... polynominal interpolation through (x,z) points
x=none ..... x-positions for the interface
z=none ..... z-positions for the interface
dtype=0 ..... diffractor type for diffr and randdf

```

OUTPUT

```

writeint=0 ..... interfaces as function of x (ext: _int)
rayfile=0 ..... interfaces as function of x in ASCII file.mod
skip=5 ..... number of x position to skip in file_ray
example=0 ..... makes an example parameter file
verbose=0 ..... silent option; >0 display info

```

Options for intt:

```

- def      = default interface through the points(Xi, Zi)
- sin      = sinus shaped interface
- rough    = rough interface with beta(smoothness)
- fract    = cosinus fractal shaped interface
- random   = define random velocities in layer
- ellipse  = define ellipse shaped body
- diffr    = point diffractions
- randdf   = define random diffractors

```

Options for var in case of intt =:

```

- sin(2)   = wavelength,amplitude
- rough(3) = amplitude,beta,seed
- fract(6) = Nsinus,amplitude,dim,k0,freqscale,seed
- random(1) = min-max variation around cp
- ellipse(2) = r1, r2: vertical and horizontal radius
- diffr(1)  = width of each point, type(optional)
- randdf(2) = number of points, width of each point

```

Options for poly in default interface:

```

- 0      = linear
- 1      = polynomial
- 2      = cubic spline

```

Options for dtype value in var=width,dtype for diffr:

```

- -1     = random (0, 1, or 2) diffractor type
- 0      = cubic diffractor
- 1      = diamond diffractor
- 2      = circular diffractor

```

Option for gradunit, gradient unit per layer:

```

- 0      = gradient unit per layer is m/s per dz (default)
- 1      = gradient unit per layer is m/s per m

```

makemod builds a gridded subsurface file which can be used in a migration or finite difference program. The gridded model is stored in files with extensions _cp, _cs, _ro. The extensions _int and .mod are used for the interface files. The output format of the file(s) depends on the .(dot) extension of file_base.

```

author   : Jan Thorbecke : 18-01-1994 (janth@xs4all.nl)
product  : Originates from DELPHI software
          : revision 2010

```

If the parameter `grad` is omitted then there is no gradient calculated. Not defining the parameter `poly` will give stepwise linear interfaces. The rough interface is defined by an amplitude, a seed value for the random generator and `beta`. The parameter `beta` defines the roughness of the interface for `beta=1.0` a very rough interface is defined while `beta=3.0` gives a smooth interface. The interface is calculated by using the equation(after: Korvin, G., 1992, *Fractal models in earth sciences*, Elsevier, Amsterdam)

$$S(x) = F^{-1} \{k_x^{-2\beta-1}\}$$

The fractal interface is defined by 6 parameters The number of sinuses, amplitude, the fractal dimension `D` ($1 < D < 2$), the fundamental spatial wavenumber `k0`, the spatial frequency scaling parameter `b` and a seed value for the random generator. The interface is calculated by using the equation

$$f_r(x) = \sigma C \sum_{n=0}^N (D-1)^n \sin(k_0 b^n x + \phi_n)$$

with `D`: roughness fractal parameter `D = 1` smooth periodic curve `D = 2` rough areafilling curve `σ`: rms height `b`: frequency scaling (`b>1`) `N`: number of tones with

$$C = \frac{2D(2-D)}{[1-(D-1)^{2N}]^{\frac{1}{2}}}$$

such that `σ` is rms height. The given equation is an approximation of a fractal surface, with good control over the wavenumber content (Jaggard, D.L. and Sun, X., 1990, *Scattering from fractally corrugated surfaces*, Journal of the Optical Society of America, Vol. **7**, p. 1131-1139.).

Vertical and horizontal gradients can be defined in the model.

For an example of the different possibilities in `makemod` see the example subsection. Special attention is given to the implementation of a lateral velocity gradient. If the user gives only one `cp` value for each interface a homogeneous layer is calculated. Two values defines a lateral velocity gradient from the minimum x-position with the first value to the maximum x-position with the last value. For more complicated lateral gradients the user has to define for every x,z position the medium parameters (`cp` (`cs`) and `ro`). For more complicated medium changes it is also possible to define a lateral and vertical gradient together.

D.2 Examples

By typing `makemod example=1 > parmodel` a sample file is generated which contains all type of interface definitions. Typing `makemod par=parmodel` generates the subsurface files `example_cp.su`, `example_cs.su` and `example_ro.su`.

E fdemmodc

The program `fdemmodc` uses the same scheme as the acoustic mode of `fdelmodc`. For `fdelmodc` the linearized equation of motion (Newton's second law) and equation of deformation (Hook's law) are given by:

$$\frac{\partial V_x}{\partial t} = -\frac{1}{\rho} \frac{\partial P}{\partial x}, \quad (214)$$

$$\frac{\partial V_z}{\partial t} = -\frac{1}{\rho} \frac{\partial P}{\partial z}, \quad (215)$$

$$\frac{\partial P}{\partial t} = -\frac{1}{\kappa} \left\{ \frac{\partial V_x}{\partial x} + \frac{\partial V_z}{\partial z} \right\}. \quad (216)$$

In `fdemmodc` the electromagnetic field propagation equations are

$$\frac{\partial H_z}{\partial t} = -\frac{1}{\mu_0} \frac{\partial E_y}{\partial x}, \quad (217)$$

$$\frac{\partial H_x}{\partial t} = -\frac{1}{\mu_0} \frac{\partial E_y}{\partial z}, \quad (218)$$

$$\frac{\partial E_y}{\partial t} = -\frac{1}{\varepsilon} \left\{ \frac{\partial H_z}{\partial x} + \frac{\partial H_x}{\partial z} \right\} + k_\sigma E_y, \quad (219)$$

with

$$\mu_0 = 4.0 * \pi * 10^{-7} \quad \text{a constant}, \quad (220)$$

$$c_0 = 299792458.0 \quad \text{a constant}, \quad (221)$$

$$\varepsilon_0 = \frac{1}{\mu_0 * c_0^2} \quad \text{a constant}, \quad (222)$$

$$\varepsilon = \varepsilon_r * \varepsilon_0 \quad \text{a constant}. \quad (223)$$

The input medium files in `fdemmodc` are `file_er` ($= \varepsilon_r$) and `file_ks` ($= k_\sigma$).

References

- Alford, R., Kelly, K., and Boore, D. (1974). Accuracy of finite-difference modeling of the acoustic wave equation. *Geophysics*, 39(6):834–842.
- Bauer, A. L., Loubère, R., and Wendroff, B. (2008). On stability of staggered schemes. *SIAM J. Numer. Anal.*, 46(2):996–1011.
- Berkhout, A. J. (1987). *Applied seismic wave theory*. Elsevier, Amsterdam.
- Blanch, J., Robertsson, J., and Symes, W. (1996). Modeling of a constant q: Methodology and algorithm for an efficient and optimally inexpensive viscoelastic technique. *Geophysics*, 60.
- Bohlen, T. (2002). Parallel 3-d viscoelastic finite difference seismic modelling. *Computer and Geosciences*, 28:887–899.
- Chew, W. C. and Liu, Q. H. (1996). Perfectly matched layers for elastodynamics: A new absorbing boundary condition. *Journal of Computational Acoustics*, pages 341–359.
- Courant, R., Friedrichs, K., and Lewy, H. (1967). On the partial difference equations of mathematical physics. *IBM Journal, English translation of the 1928 German original*, pages 215–234. Available as download <http://www.stanford.edu/class/cme324/classics/courant-friedrichs-lewy.pdf>.
- Drossaert, F. H. and Giannopoulos, A. (2007). A nonsplit complex frequency-shifted pml based on recursive integration for fdtd modeling of elastic waves. *Geophysics*, 72(2):T9–T17.
- Dutta, G. and Schuster, G. T. (2014). Attenuation compensation for least-squares reverse time migration using the viscoacoustic-wave equation. *GEOPHYSICS*, 79(6):S251–S262.
- Fornberg, B. (1988). Generation of finite difference formulas on arbitrarily spaced grids. *Mathematics of Computation*, 51(184):699–706.
- Pérez-Ruiz, J. A., Luzón, F., and García-Jerez, A. (2005). Simulation of an irregular free surface with a displacement finite-difference scheme. *Bulletin of the Seismological Society of America*, 95(6):2216–2231.
- Robertsson, J. O. A. (1996). A numerical free-surface condition for elastic/viscoelastic finite-difference modeling in the presence of topography. *Geophysics*, 61:1921–1934.
- Robertsson, J. O. A., Blanch, J. O., and Symes, W. W. (1994). Viscoelastic finite-difference modeling. *Geophysics*, 59(09):1444–1456.
- Saenger, E. H. and Bohlen, T. (2004). Finite-difference modeling of viscoelastic and anisotropic wave propagation using the rotated staggered grid. *Geophysics*, 69(2):583–591.
- Sei, A. (1995). A family of numerical schemes for the computation of elastic waves. *SIAM J. Sci. Comput.*, 16(4):898–916.
- Sei, A. and Symes, W. (1995). Dispersion analysis of numerical wave propagation and its computational consequences. *Journal of Scientific Computing*, 10(1):1–27.

- Thorbecke, J. and Draganov, D. (2011). Finite-difference modeling experiments for seismic interferometry. *Geophysics*, 76(6):H1–H18.
- van den Berg, P. M. (2017). *Forward and Inverse Scattering Algorithms Based on Contrast Source Integral Equations*. in preparation.
- van Vossen, R., Robertsson, J. O. A., and Chapman, C. (2002). Finite-difference modeling of wave propagation in a fluid-solid configuration. *Geophysics*, 67(2):618–624.
- Virieux, J. (1986). P-Sv wave propagation in heterogeneous media - Velocity-stress finite-difference method. *Geophysics*, 51(04):889–901.
- Wang, Y. (2008). *Seismic Inverse Q Filtering*.
- Wapenaar, K. (1998). Reciprocity properties of one-way propagators. *Geophysics*, 63(4):1795–1798.